

spintent

SPANISH PARSE INTENTS

V0.99 2026-10-04^{*}

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	8
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	8
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	10
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	19
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	35
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	36
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	37
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	40
			12	Índice de la Implementación	166

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con MathCAT[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/MathCAT para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/MathCAT.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de MathML. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

^{*}Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-10-04.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «A Work in Progress»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{comando}{key-uno = valor, key-dos = valor, key-tres = valor, ...}`

El comando `\setspintent` establece las llaves (*keys*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*keys*) definidas en el *argumento opcional* [*key = val*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*key = val*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*key = val*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{<número>}` `\spnum{<número,número>}` `\spnum{<número.número>}` `\spnum{<número;número>}`
`\spnum[key = val]{<número,número;número>}` `\spnum[key = val]{<número,número;número e exponente>}`
`\spnum[key = val]{<número.número;número>}` `\spnum[key = val]{<número.número;número e exponente>}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<número>}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma ';':

Si no se proporciona el *argumento opcional* [*key = val*] y se ejecuta `\$ \spnum{+23} \$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\$ \spnum{3,14} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\$ \spnum{3,1;4} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {<número>} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\$ \spnum{3,14e3} \$` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {<argumento>}. El {<argumento>} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {<true | false>}` default: true

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\$ \spnum{100000} \$` imprimirá '100 000' en la salida PDF y `\$ \spnum[cifras-sep=false]{10\,00\,00} \$` imprimirá '10 00 00' en la salida PDF.

`read-sign = {<terse | clear>}` default: terse

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en `terse`, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\$ \spnum{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\$ \spnum[read-sign=clear]{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {<coma | punto>}` default: coma

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ';' de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ',' en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá '.' en la

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

salida PDF y NVDA/MathCAT verbalizará «punto». Por ejemplo $\$ \backslash \text{spnum}\{3;4\}$ imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma cuatro periódico» y $\$ \backslash \text{spnum}[\text{read-period-sep=punto}]\{3;4\}$ imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «tres punto cuatro periódico»

- Esta llave es necesaria SOLO cuando se utilizan «decimales periódicos puros», es decir, cuando no se ha incluido en el $\{ \langle \text{argumento} \rangle \}$ una coma ‘,’ o un punto ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = { \langle times \rangle | \langle cdot \rangle }`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «notación científica» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «por»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por». Por ejemplo $\$ \backslash \text{spnum}\{3,14\text{e}3\}$ imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo» y $\$ \backslash \text{spnum}[\text{notation-sym=times}]\{3,14\text{e}3\}$ imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo».

- Decir «notación científica» aquí es un poco exagerado, en realidad es «multiplicación por potencias de 10», no se verifica si la entrada cumple o no con la condición de estar escrita en «notación científica».

3.2 El comando `\spunit`

<code>\spunit</code>	<code>\spunit\{ \langle unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>
	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>
	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número e exponente unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número e exponente unidad \rangle \}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle unidad \rangle \}$, el cual puede contener una «parte numérica» opcional la cual se procesa internamente por el comando `\spnum` al inicio del $\{ \langle argumento \rangle \}$ seguido de una $\{ \langle unidad \rangle \}$ o un $\{ \langle alias \rangle \}$ valido descritos en §3.2.2 o una $\{ \langle unidad \rangle \}$ creada por el comando `\spnewunit` (§3.2.3) o un $\{ \langle alias \rangle \}$ creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \backslash \text{spunit}\{23 \text{ m}\}$ imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros».

La multiplicación entre las $\{ \langle unidades \rangle \}$ se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^ $\{ \langle exponente \rangle \}$ ’. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m}^* \text{s}^{\wedge}\{2\}\}$ se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros segundos al cuadrado».

Se pueden usar *fracciones lineales* con las $\{ \langle unidades \rangle \}$ utilizando una «única barra» ‘/’ para la separación entre las $\{ \langle unidades \rangle \}$ del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo: $\$ \backslash \text{spunit}\{23 \text{ m}/\text{s}^{\wedge}\{2\}\}$ imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros por segundos al cuadrado».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El $\{ \langle argumento \rangle \}$ puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm\{ \langle #1 \rangle \}` o `\mathbf\{ \langle #1 \rangle \}` por ejemplo, NO puede contener más de una «única barra» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para `\spunit`

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de $\{ \langle unidades \rangle \}$. Si se establece en `true`, se imprimirá ‘·’ entre las $\{ \langle unidades \rangle \}$ en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las $\{ \langle unidades \rangle \}$ en la salida PDF.

`read-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de $\{ \langle unidades \rangle \}$. Si se establece en `true`, NVDA/MathCAT verbalizará «por» entre las $\{ \langle unidades \rangle \}$; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las $\{ \langle unidades \rangle \}$.

`read-slash = { \langle por \rangle | \langle partido-por \rangle }`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «barra» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «partido por».

3.2.2 Argumentos para `\spunit`

Resumen de $\{ \langle unidades \rangle \}$ y $\{ \langle alias \rangle \}$ soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»

Unidades directas (sin alias):

L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{⟨nueva unidad⟩}{⟨verbalización NVDA⟩}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\langle \textit{nueva unidad} \rangle$ que establece lo que se imprimirá en la salida PDF y $\langle \textit{verbalización NVDA} \rangle$ que establece como NVDA/MathCAT verbalizará la $\langle \textit{nueva unidad} \rangle$.

La `{\nueva unidad}` no debe estar registrada como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\{\langle\textit{nueva unidad}\rangle\}$ es «global» y se almacena en el módulo `spintent.lua` como una $\{\langle\textit{unidad}\rangle\}$ valida para pasar como $\{\langle\textit{argumento}\rangle\}$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\{\langle\textit{nueva unidad}\rangle\}$ ‘px’ y cuando se utilice `\$ \spunit{100 px} \$` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{⟨nuevo alias⟩}{⟨expresión de unidades⟩}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nuevo alias} \rangle\}$ que establece es el nuevo $\{\langle \text{alias} \rangle\}$ de $\{\langle \text{argumento} \rangle\}$ para `\spunit` y $\{\langle \text{expresión de unidades} \rangle\}$ que puede contener multiplicación o división de $\{\langle \text{unidades} \rangle\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{\nuevo alias}` es «global» y se almacena en el módulo `spintent`. Lua como una `{\alias}` valido para pasar como `{\argumento}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^{\textstyle 3}}/s}` creará el `{\alias}` ‘caudal’ y cuando se utilice `\$ \spunit{50 caudal} \$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<u>\spmoney</u>	<code>\spmoney{<i>{cifra}</i>}</code>	<code>\spmoney[<i>{key = val}</i>]{<i>{cifra}</i>}</code>
	<code>\spmoney{<i>{cifra símbolo moneda}</i>}</code>	<code>\spmoney[<i>{key = val}</i>]{<i>{cifra símbolo moneda}</i>}</code>
	<code>\spmoney{<i>{cifra alias moneda}</i>}</code>	<code>\spmoney[<i>{key = val}</i>]{<i>{cifra alias moneda}</i>}</code>

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \textit{cifra} \rangle \}$ que contiene la «*cifra numérica*» positiva o negativa la cual se procesa internamente por el comando `\spnum` acompañada de «*símbolo moneda*» o «*alias moneda*» opcionales descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\$500` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase «*símbolo moneda*» o «*alias moneda*» en el argumento *obligatorio* donde tienen prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dolares}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dólares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estándar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá 'CLP500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el «*símbolo de moneda*» correspondiente. Por ejemplo: `\spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo moneda*» como '\$', '€', '£' o un «*alias moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT. Los «*alias moneda*» se pueden pasar con o sin tilde, por ejemplo dolares o dólares.

- Los «*alias moneda*» se deben escribir en minúsculas y los códigos ISO en minúsculas o todo en mayúsculas. Cualquier otra escritura, por ejemplo PESO o Usd, devolverá un “error” y se detendrá la compilación del documento.
- La posición del «*símbolo moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando (o del *modo texto* si se establece `money-math=false`), se debe tener presente en caso de querer usar un «*símbolo moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, con la única excepción de `\$`. La coma ‘,’ o el punto ‘.’ como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: peso

Establece el *símbolo moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo moneda*» o el «*alias moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

- Si la «*cifra numérica*» no tiene parte decimal, la llave `sub-units` no tiene efecto. Si la moneda en curso no admite *subunidades* se emite una advertencia y la llave tampoco tiene efecto.

`money-math = {(true | false)}` default: true

Establece si el *símbolo moneda* se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir el *símbolo moneda* en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir el *símbolo moneda* en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los «*símbolos moneda*» y «*alias moneda*» soportados en el `{(argumento)}` de `\spmoney`.

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	« <i>peso / pesos</i> »
dolar	dolares, dólar, dólares	dólar	\$	« <i>dólar / dólares</i> »
€	euro, euros	euro	€	« <i>euro / euros</i> »
£	libra, libras	libra	£	« <i>libra / libras</i> »
¥	yen, yenes	yen	¥	« <i>yen / yenes</i> »
yuan	yuanes	yuan	¥	« <i>yuan / yuanes</i> »
₩	won, wons	won	₩	« <i>won / wons</i> »
₪	shekel, shekels, sequel, sequeles	séquel	₪	« <i>séquel / séqueles</i> »
₹	rupia, rupias	rupia	₹	« <i>rupia / rupias</i> »
₰	rublo, rublos	rublo	₰	« <i>rublo / rublos</i> »
₺	lira, liras	lira	₺	« <i>lira / liras</i> »
₴	grivna, grivnas	grivna	₴	« <i>grivna / grivnas</i> »
¢	centavo, centavos	centavo	¢	« <i>centavo / centavos</i> »
Divisas en formato ISO (Mayúsculas)				
CLP	—	peso chileno	CLP	« <i>peso chileno / pesos chilenos</i> »
MXN	—	peso mexicano	MXN	« <i>peso mexicano / pesos mexicanos</i> »
USD	—	dólar	USD	« <i>dólar estadounidense / dólares estadounidenses</i> »

Continúa en la siguiente página...

Cuadro 2 – Continuación de la página anterior

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	—	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4 El comando \spshort

```
\spshort{<abreviaturas | acrónimos | números ordinales>}
\spshort[<key = val>]{<abreviaturas | acrónimos | números ordinales>}
\spshort[<read-arg={<verbalización NVDA>}]{<comandos LATEX>}
```

El comando `\spshort` trabaja solo en modo texto y recibe un argumento obligatorio del tipo `{<abreviaturas>}`, `{<acrónimos>}` o `{<números ordinales>}` descritos en §3.4.2.

Si no se proporciona el argumento opcional `[<key = val>]` y se ejecuta `\spshort{dr.a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{<números ordinales>}` se puede ejecutar `\spshort{1.a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1.o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear un nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` que no se encuentre registrada por spintent dentro del módulo Lua spintent.lua descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del argumento obligatorio que puede contener `{<comandos LATEX>}` en modo texto los cuales se imprimirán en salida PDF.

- Para la escritura de `{<números ordinales>}` se debe tener cuidado y no confundir la «letra o voladita» ‘o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del modo texto está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para `shorthands` y el comando `\sptext` provistos por el paquete `babel` en español compatible con `tagged` PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja solo en modo texto. Internamente utiliza la llave `E` (expansion text) proveída por el paquete `tagpdf`[29].

3.4.1 Llaves para \spshort

```
read-arg = {<verbalización NVDA>} default: no usado
Establece la verbalización del argumento {<comandos LATEX>} de la nueva {<abreviatura>}, {<acrónimo>}
o {<número ordinal>} para NVDA. El valor de esta llave siempre debe pasarse entre ‘{ }’. Por ejemplo:
\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}} creará una nueva {<abreviatura>}
que imprimirá ‘8.avo’ en la salida PDF y NVDA verbalizará «octavo».
```

```
small-caps = {<true | false>} default: false
```

Establece si se usarán versalitas para imprimir `{<acrónimos>}` y para la «letra volada» en `{<abreviaturas>}` y `{<números ordinales>}` respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas

con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^A’ en la salida PDF y NVDA verbalizará «*doctora*».

3.4.2 Argumentos para `\spshort`

El cuadro 3 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	« <i>doctora</i> »
dr.as	—	Dr. ^{as}	« <i>doctoras</i> »
prof.a	—	Prof. ^a	« <i>profesora</i> »
d.a	d. ^a	D. ^a	« <i>doña</i> »
m.a	—	M. ^a	« <i>maría</i> »
n.o	n. ^o	N. ^o	« <i>número</i> »
n.os	—	N. ^{os}	« <i>números</i> »
c.ia	—	C. ^{ia}	« <i>compañía</i> »
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	« <i>página</i> »
vol.	—	vol.	« <i>volumen</i> »
etc.	—	etc.	« <i>etcétera</i> »
et al.	et. al.	et al.	« <i>y otros</i> »
ibíd.	ibid.	ibíd.	« <i>ibídem</i> »
op. cit.	op.cit.	op. cit.	« <i>obra citada</i> »
loc. cit.	loc.cit.	loc. cit.	« <i>lugar citado</i> »
v. gr.	v.gr.	v. gr.	« <i>verbigracia</i> »
i. e.	i.e.	i. e.	« <i>esto es</i> »
e. g.	e.g.	e. g.	« <i>por ejemplo</i> »
p. ej.	p.ej.	p. ej.	« <i>por ejemplo</i> »
Títulos, profesiones y cargos			
sr.	—	Sr.	« <i>señor</i> »
sra.	—	Sra.	« <i>señora</i> »
srta.	—	Srta.	« <i>señorita</i> »
dr.	—	Dr.	« <i>doctor</i> »
dra.	—	Dra.	« <i>doctora</i> »
dres.	—	Dres.	« <i>doctores</i> »
dras.	—	Dras.	« <i>doctoras</i> »
prof.	—	Prof.	« <i>profesor</i> »
profa.	—	Profa.	« <i>profesora</i> »
profs.	—	Profs.	« <i>profesores</i> »
ing.	—	Ing.	« <i>ingeniero</i> »
ings.	—	Ings.	« <i>ingenieros</i> »
lic.	—	Lic.	« <i>licenciado</i> »
v. b.	v.b.	V. B.	« <i>visto bueno</i> »
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	« <i>sociedad anónima</i> »
ee. uu.	ee.uu.	EE. UU.	« <i>estados unidos</i> »
dd. hh.	dd.hh.	DD. HH.	« <i>derechos humanos</i> »
jj. oo.	jj.oo.	JJ. OO.	« <i>juegos olímpicos</i> »
ff. aa.	ff.aa.	FF. AA.	« <i>fuerzas armadas</i> »
rr. ee.	rr.ee.	RR. EE.	« <i>relaciones exteriores</i> »
cc. aa.	cc.aa.	CC. AA.	« <i>comunidades autónomas</i> »
p. d.	p.d.	P. D.	« <i>posdata</i> »
a. c.	a.c.	a. C.	« <i>antes de Cristo</i> »
d. c.	d.c.	d. C.	« <i>después de Cristo</i> »
d. n. i.	d.n.i.	D. N. I.	« <i>documento nacional de identidad</i> »
r. u. t.	r.u.t.	R. U. T.	« <i>rol único tributario</i> »
r. u. n.	r.u.n.	R. U. N.	« <i>rol único nacional</i> »

El cuadro 4 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	—	ONG	«organización no gubernamental»
ongs	—	ONGs	«organizaciones no gubernamentales»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 5 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 5: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

Los números ordinales se verbalizan con palabras desde el 1 hasta el 99. Desde el 100 el ordinal no se convierte en palabras y se entrega a NVDA/MathCAT tal como se imprime, por ejemplo 100. o como «100^o».

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>} \spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el argumento obligatorio `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El argumento obligatorio $\{\langle siglo \rangle\}$ «*siempre*» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {\ac | dc | none} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el argumento $\{\langle siglo \rangle\}$. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xxi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno antes de cristo*»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xxi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno después de cristo*»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno*».

El comando `\sptime`

```
\sptime \sptime{\horas : minutos} \sptime{\horas : minutos : segundos}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle horas : minutos \rangle\}$ o $\{\langle horas : minutos : segundos \rangle\}$. Por ejemplo: `\sptime{23:45}` imprimirá en «*modo texto*» la salida PDF ‘23:45’ y NVDA/MathCAT verbalizará «*las veintitrés y cuarenta y cinco minutos*».

El comando `\spdate`

```
\spdate \spdate{\DD/MM/YYYY} \spdate{\YYYY/MM/DD} \spdate{\YYYY-MM-DD} \spdate{\DD-MM-YYYY}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle DD/MM/YYYY \rangle\}$, $\{\langle DD-MM-YYYY \rangle\}$ o $\{\langle YYYY-MM-DD \rangle\}$. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en «*modo texto*» la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «*dos de enero de mil novecientos ochenta y uno*».

- Si se pasa el argumento $\{\langle YYYY/MM/DD \rangle\}$, internamente se convertirá en $\{\langle DD/MM/YYYY \rangle\}$ y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee cuatro comandos `\spnewread`, `\spread`, `\spdsym` y `\spmsym` para trabajar con «*símbolos matemáticos*» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «*símbolo*» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «*símbolos matemáticos*», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spnewread`

```
\spnewread \spnewread{\símbolo}{\verbalización NVDA}
```

El comando `\spnewread` trabaja en *modo texto* y recibe dos argumentos obligatorios: $\{\langle símbolo \rangle\}$ y $\{\langle verbalización NVDA \rangle\}$ los cuales se almacenan de forma interna para ser utilizados como $\{\langle argumento \rangle\}$ en `\spread`. Solo se establece como NVDA/MathCAT verbalizará el $\{\langle símbolo \rangle\}$ al utilizarlo con `\spread` sin modificar la impresión de este.

El argumento $\{\langle símbolo \rangle\}$ no debe estar registrado. Si ya está registrado se devolverá un “error” y se detendrá la compilación del documento. La declaración es «global» y se almacena en el módulo `spintent.lua`. Por ejemplo: `\spnewread{\cong}{es-congruente-con}` registrará ‘\cong’ y cuando se utilice `\spread{\cong}` se imprimirá ‘≅’ en la salida PDF y NVDA/MathCAT verbalizará «*es congruente con*».

- El $\{\langle símbolo \rangle\}$ debe ser un único comando, pero puede ser uno definido por el usuario. Por ejemplo, con `\newcommand{\xcuad}{x^2}`, `\spnewread{\xcuad}{x-al-cuadrado}` registrará ‘\xcuad’ y cuando se utilice `\spread{\xcuad}` se imprimirá ‘x²’ en la salida PDF y NVDA/MathCAT verbalizará «*x al cuadrado*».
- Los espacios y los guiones bajos de $\{\langle verbalización NVDA \rangle\}$ se convierten en guiones ‘-’, por lo que {es congruente con} equivale a {es-congruente-con}. Los guiones repetidos se reducen a uno y se eliminan los del inicio y del final. Si el resultado queda vacío se devolverá un “error” y se detendrá la compilación del documento.

El comando `\spread`

```
\spread <{símbolo}> <spread[<key = val>]>{<símbolo>}
\spread[<read-arg={<verbalización NVDA>}>]{<comandos LATEX>}
```

El comando `\spread` trabaja en *modo matemático* y recibe el *argumento obligatorio* de la forma `{<símbolo>}` previamente registrado por el comando `\spnewread` o `{<comandos LATEX>}` para ser usado junto a la llave `read-arg`.

Si el argumento es `{<símbolo>}`, `$\spread{<cong>}$` imprimirá ‘ $\cong$ ’ en la salida PDF y el argumento `{<verbalización NVDA>}` establecido al registrar `{<símbolo>}` con `\spnewread` se verbalizará en NVDA/MathCAT.

Si el argumento es `{<comandos LATEX>}` y no se establece la llave `read-arg` se devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spread`

`read-arg = {<verbalización NVDA>}` default: no usado

Establece la *verbalización* del argumento `{<comandos LATEX>}`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y sobrescribe la verbalización del `{<símbolo>}` si está registrado previamente. Por ejemplo: `$\spread[read-arg={es-paralela-a}]{\parallel}$` imprimirá ‘ $\parallel$ ’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela a».

`silent = {<true | false>}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del argumento `{<símbolo>}` o `{<comandos LATEX>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

- Al utilizar `silent` no es necesario que el argumento `{<símbolo>}` esté registrado o usar la llave `read-arg` con `{<comandos LATEX>}`. Dependiendo de que contenga `{<argumento>}` puede que `silent` no sea suficiente, en ese caso utilice `read-arg={_}`.

El comando `\spusep`

```
\spusep <{\{ | [ | ( | ) | ] | \} }> <spusep[<key = val>]>{\{ | [ | ( | ) | ] | \} }
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{\{ | [ | ( | ) | ] | \} }`, el cual debe ser «solo uno» de los paréntesis soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spusep{< >}$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «abrir paréntesis» acorde a su propia heurística.

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por Lua^T_E^X crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

`size = {<none | big | Big | bigg | Bigg>}` default: none

Establece el *tamaño del paréntesis* pasado como `{<argumento>}` que se imprimirá en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en `none`, se imprime el `{<argumento>}` en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr` antes del `{<argumento>}`; si se establece en `Big`, se usa `\Bigl/\Bigr` antes del `{<argumento>}`; si se establece en `bigg`, se usa `\biggl/\biggr` antes del `{<argumento>}`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr` antes del `{<argumento>}`.

`silent = {<true | false>}` default: false

Establece si se *silencia* en NVDA/MathCAT la verbalización del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spdsym`

```
\spdsym <spdsym[<key = val>]>
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spdsym$` imprimirá ‘:’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para \spdsym

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym = {⟨old-style | two-dots | slash⟩}` default: two-dots

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá '÷'; si se establece en `two-dots`, imprimirá '⋅' y si se establece `slash`, imprimirá '/'.

`read-sym = {⟨dividido | dividido-por | dividido-entre⟩}` default: dividido

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando \spmsym

`\spmsym` `\spmsym` `\spmsym[⟨key = val⟩]`

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* [⟨key = val⟩] mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spmsym$` imprimirá '·' en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para \spmsym

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym = {⟨cross | dot | asterisk⟩}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá '×'; si se establece en `dot`, imprimirá '⋅' y si se establece en `asterisk`, imprimirá '*'.

`read-sym = {⟨por | multiplicado-por | veces⟩}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {⟨true | false⟩}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando \spdiv

`\spdiv` `\spdiv{⟨dividendo⟩}{⟨divisor⟩}` `\spdiv[⟨key = val⟩]{⟨dividendo⟩}{⟨divisor⟩}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨dividendo⟩} y {⟨divisor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spdiv{20}{26}$` imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veintiséis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spdsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul`{*factor*}{*factor*} `\spmul`[*key* = *val*]{*factor*}{*factor*}

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* { *factor* } y { *factor* }. Si no se proporciona el *argumento opcional* [*key* = *val*], `\spmul{20}{26}` imprimirá ‘20 · 26’ en la salida PDF y NVDA/MathCAT verbalizará «veinte por veintiséis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

`\spnD` `\spnD` (*número*) `\spnD`[*key* = *val*](*número*) `\spnD`(*letra*) `\spnD`[*key* = *val*](*letra*)

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número*) que se procesa internamente por `\spnum` o (*letra*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key* = *val*] ni el *argumento opcional delimitado por paréntesis*, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de ene».

Si se usa el *argumento* (*número*), `\spnD(6)` imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Si se usa el *argumento* (*letra*), `\spnD(m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de eme».

🚫 El número 0 no es válido: `\spnD(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnD`

`concept` = { *true* | *false* } default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «divisores de» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`prefix` = { *prefijo* } default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «divisores de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { *true* | *false* } default: *false*

Establece si se *imprime* el resultado de calcular los *divisores* del *argumento* (*número*). Si se establece en *true*, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis son uno, dos, tres y seis»; si se establece en *false*, no se imprimirá ni verbalizará el resultado de calcular los *divisores*.

`result-num` = { *número natural* } default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» y del (⟨*argumento*⟩) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnM`

`\spnM` `\spnM` `\spnM(⟨número⟩)` `\spnM[⟨key = val⟩](⟨número⟩)` `\spnM(⟨letra⟩)` `\spnM[⟨key = val⟩](⟨letra⟩)`

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (⟨*número*⟩) que se procesa internamente por `\spnum` o (⟨*letra*⟩) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] ni el *argumento opcional delimitado por paréntesis*, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de ene».

Si se usa el argumento (⟨*número*⟩), `\spnM(6)` imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Si se usa el argumento (⟨*letra*⟩), `\spnM(m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de eme».

🚫 El número **0** no es válido: `\spnM(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnM`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT verbalizará «múltiplos de» y el (⟨*argumento*⟩); si se establece en **false**, NVDA/MathCAT solo verbalizará el (⟨*argumento*⟩).

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «múltiplos de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular los *múltiplos* del argumento (⟨*número*⟩). Si se establece en **true**, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en **false**, no se imprimirá ni verbalizará el resultado de calcular los *múltiplos*.

`result-num = {⟨número natural⟩}` default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» y del (⟨*argumento*⟩) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmcD`

`\spmcD` `\spmcD` `\spmcD(⟨número, número, ...⟩)` `\spmcD(⟨letra, letra, ...⟩)` `\spmcD[⟨key = val⟩](⟨número, número, ...⟩)`

El comando `\spmcD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (⟨*número, número, ...*⟩) que se procesa internamente por `\spnum` o (⟨*letra, letra, ...*⟩) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] ni el *argumento opcional delimitado por paréntesis*, `\spmcD` imprimirá ‘mcd’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor».

Si se usa el argumento (⟨*número, número, ...*⟩), `\spmcD(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis».

Si se usa el argumento (⟨*letra, letra, ...*⟩), `\spmcD(m, n)` imprimirá ‘mcd(*m, n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre eme y ene».

🚫 Los argumentos (⟨*número, número, ...*⟩) o (⟨*letra, letra, ...*⟩) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {\langle true \rangle | \langle false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*máximo común divisor ...*» o «*m c d ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {\langle terse \rangle | \langle clear \rangle}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c d*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*máximo común divisor*».

`upper-case = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCD' en la salida PDF; si se establece en *false*, se imprimirá 'mcd' en la salida PDF.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se *imprime* el resultado de calcular el «*máximo común divisor*» del argumento (*número, número, ...*). Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá 'mcd(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis es igual a uno*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*máximo común divisor*».

`silent = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*máximo común divisor ...*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmc`

`\spmc` `\spmc` `\spmc(\langle número, número, ... \rangle)` `\spmc(\langle letra, letra, ... \rangle)` `\spmc[\langle key = val \rangle](\langle número, número, ... \rangle)`

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spmc` imprimirá 'mcm' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

Si se usa el argumento (*número, número, ...*), `\spmc(2, 3, 6)` imprimirá 'mcm(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), `\spmc(m, n)` imprimirá 'mcm(m, n)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre eme y ene*».

Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {\langle true \rangle | \langle false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*mínimo común múltiplo ...*» o «*m c m ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {\langle terse \rangle | \langle clear \rangle}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`upper-case = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCM' en la salida PDF; si se establece en *false*, se imprimirá 'mcm' en la salida PDF.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del argumento ($\langle \text{número}, \text{número}, \dots \rangle$). Si se establece en *true*: $\$ \backslash \text{spmcm}[\text{result}=\text{true}](2, 3, 6) \$$ imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*mínimo común múltiplo*».

`silent = { \langle true | false \rangle }` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*mínimo común múltiplo ...*» y del ($\langle \text{argumento} \rangle$) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnZ`

`\spnZ` $\backslash \text{spnZ} \{ \langle \text{número entero} \rangle \}$ $\backslash \text{spnZ} [\langle \text{key} = \text{val} \rangle] \{ \langle \text{número entero} \rangle \}$

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{ \langle \text{número entero} \rangle \}$ el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las $\langle \text{meta-keys} \rangle$ *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = { \langle true | false \rangle }` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento $\{ \langle \text{número entero} \rangle \}$ pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del $\{ \langle \text{número entero} \rangle \}$; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = { \langle true | false \rangle }` default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* $\{ \langle \text{número entero} \rangle \}$ pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)»; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` $\backslash \text{spmQ} \{ \langle \text{entero} \rangle \} \{ \langle \text{numerador} \rangle \} \{ \langle \text{denominador} \rangle \}$ $\backslash \text{spmQ} [\langle \text{key} = \text{val} \rangle] \{ \langle \text{entero} \rangle \} \{ \langle \text{numerador} \rangle \} \{ \langle \text{denominador} \rangle \}$

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: $\{ \langle \text{número entero} \rangle \}$, $\{ \langle \text{numerador} \rangle \}$ y $\{ \langle \text{denominador} \rangle \}$, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spmQ} \{ 2 \} \{ 5 \} \{ 7 \} \$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de **spintent**. Por ejemplo, la entrada $\$ 3 \backslash \text{frac} \{ 5 \} \{ 7 \} \$$ con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = { \langle entero | enteros | y \rangle }` default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en *entero*, $\$ \backslash \text{spmQ} [\text{join-word}=\text{entero}] \{ 1 \} \{ 5 \} \{ 7 \} \$$ imprimirá ‘ $1\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*un entero cinco séptimos*»; si se establece en *enteros*, $\$ \backslash \text{spmQ} [\text{join-word}=\text{enteros}] \{ 2 \} \{ 5 \} \{ 7 \} \$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos enteros cinco séptimos*»; si se establece en *y*, $\$ \backslash \text{spmQ} [\text{join-word}=y] \{ 2 \} \{ 5 \} \{ 7 \} \$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

`use-spnZ = { \langle true | false \rangle }` default: *true*

Establece si el $\{ \langle \text{numerador} \rangle \}$ y el $\{ \langle \text{denominador} \rangle \}$ que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en *true*, $\{ \langle \text{numerador} \rangle \}$ y $\{ \langle \text{denominador} \rangle \}$ se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en *false*, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`use-dfrac = { \langle true | false \rangle }` default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\$ \backslash \text{spmQ} [\text{use-dfrac}=\text{true}] \{ 2 \} \{ 5 \} \{ 7 \} \$$ imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = { \langle true | false \rangle }` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en *true*, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = { \langle true | false \rangle }` default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\sprazon`

`\sprazon` `\sprazon{⟨antecedente⟩}{⟨consecuente⟩}` `\sprazon[⟨key = val⟩]{⟨antecedente⟩}{⟨consecuente⟩}`

El comando `\sprazon` trabaja en *modo matemático* y recibe *dos argumentos obligatorios*: `{⟨antecedente⟩}` y `{⟨consecuente⟩}`, los tipografía como una fracción y devuelve una «razón».

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `⟨$⟩\sprazon{3}{4}⟨$⟩` imprimirá ‘ $\frac{3}{4}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro».

- El `{⟨antecedente⟩}` y el `{⟨consecuente⟩}` se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\sprazon`

`silent = {⟨true | false⟩}` default: `false`

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *razón* pasada al comando sin modificar la impresión de esta. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {⟨true | false⟩}` default: `false`

Establece si la fracción de la *razón* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `⟨$⟩\sprazon[use-dfrac=true]{3}{4}⟨$⟩` imprimirá la fracción en tamaño *display* ‘ $\frac{3}{4}$ ’ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline = {⟨true | false⟩}` default: `false`

Establece si la *razón* se imprime en línea con dos puntos ‘:’ en lugar de una fracción. Si se establece en `true`, `⟨$⟩\sprazon[inline=true]{3}{4}⟨$⟩` imprimirá ‘3 : 4’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro»; si se establece en `false`, la razón se imprime como una fracción.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprime una fracción.

El comando `\spProp`

`\spProp` `\spProp{⟨a⟩}{⟨b⟩}{⟨c⟩}{⟨d⟩}` `\spProp[⟨key = val⟩]{⟨a⟩}{⟨b⟩}{⟨c⟩}{⟨d⟩}`

El comando `\spProp` trabaja en *modo matemático* y recibe *cuatro argumentos obligatorios*: `{⟨a⟩}` y `{⟨b⟩}` forman la primera *razón*, `{⟨c⟩}` y `{⟨d⟩}` forman la segunda, y devuelve una «proporción» con las dos razones unidas por el signo igual.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `⟨$⟩\spProp{3}{4}{6}{8}⟨$⟩` imprimirá ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho».

- Los cuatro argumentos se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\spProp`

`silent = {⟨true | false⟩}` default: `false`

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *proporción* pasada al comando sin modificar la impresión de esta. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {⟨true | false⟩}` default: `false`

Establece si las fracciones de la *proporción* se imprimen usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `⟨$⟩\spProp[use-dfrac=true]{3}{4}{6}{8}⟨$⟩` imprimirá las fracciones en tamaño *display* ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF; si se establece en `false`, las fracciones se imprimen con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline = {⟨true | false⟩}` default: `false`

Establece si las *razones* de la *proporción* se imprimen en línea con dos puntos ‘:’ en lugar de fracciones. Si se establece en `true`, `⟨$⟩\spProp[inline=true]{3}{4}{6}{8}⟨$⟩` imprimirá ‘3 : 4 = 6 : 8’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho»; si se establece en `false`, las razones se imprimen como fracciones.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprimen fracciones.

7 Utilidades para geometría

El paquete **spintent** provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como $\{\langle \text{argumento} \rangle\}$ base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben $\{\langle \text{argumento} \rangle\}$ único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘ $\prime$ ’ y los subíndices siempre deben escribirse usando llaves ‘ $A_{\{...\}}$ ’ y nunca sin estas ‘ A_n ’ que funciona solo por coincidencia. .

El comando `\spcoord`

<code>\spcoord</code>	<code>\spcoord{\langle abscisa, ordenada \rangle}</code>	<code>\spcoord[\langle key = val \rangle]{\langle abscisa, ordenada \rangle}</code>
	<code>\spcoord{\langle abscisa; ordenada \rangle}</code>	<code>\spcoord[\langle key = val \rangle]{\langle abscisa; ordenada \rangle}</code>

El comando `\spcoord` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle abscisa, ordenada \rangle\}$ o $\{\langle abscisa; ordenada \rangle\}$ el cual se procesa internamente bajo el comando `\spnum` cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\spcoord{-3, 4}` imprimirá ‘ $(-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «menos tres coma cuatro»; `\spcoord{x_{1}, y_{1}}` imprimirá ‘ (x_1, y_1) ’ y verbalizará «*exis subíndice uno y subíndice uno*».

- El comando `\spcoord` solo acepta un *separador* ‘,’ o ‘;’ para la abscisa y la ordenada cuando son valores numéricos no fraccionarios.

Llaves para `\spcoord`

`silent = {\langle true | false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoint`

<code>\spPoint</code>	<code>\spPoint{\langle punto \rangle}</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}</code>
	<code>\spPoint{\langle punto \rangle}(\langle abscisa, ordenada \rangle)</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}(\langle abscisa, ordenada \rangle)</code>
	<code>\spPoint{\langle punto \rangle}(\langle abscisa; ordenada \rangle)</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}(\langle abscisa; ordenada \rangle)</code>

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle punto \rangle\}$ el cual es una *única letra mayúscula* con prima ‘ $\prime$ ’ o subíndice ‘ $_{\{...\}}$ ’ y el *argumento opcional delimitado por paréntesis* $(\langle abscisa, ordenada \rangle)$ el cual se procesa internamente bajo el comando `\spcoord`.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\spPoint{A}` imprimirá ‘ A ’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A*»; `\spPoint{A_{1}}` imprimirá ‘ A_1 ’ y verbalizará «*punto A sub uno*».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spPoint{A}(-3, 4)` imprimirá ‘ $A(-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A menos tres coma cuatro*»; `\spPoint{A_{1}}(-3, 4)` imprimirá ‘ $A_1(-3, 4)$ ’ y verbalizará «*punto A sub uno menos tres coma cuatro*».

Llaves para `\spPoint`

`read-cmd = {\langle school | mathcat \rangle}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle punto \rangle\}$. Si se establece en *school*, NVDA/MathCAT verbalizará el argumento $\{\langle punto \rangle\}$ sin el prefijo «mayúscula»; `\spPoint{A}` se verbaliza «*punto A*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*punto*». El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

`concept = {\langle true | false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent = {\langle true | false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{⟨letra⟩}</code>	<code>\spvec{⟨punto, punto⟩}</code>
	<code>\spvec{⟨letra⟩} (⟨abscisa, ordenada⟩)</code>	<code>\spvec{⟨punto, punto⟩} (⟨abscisa, ordenada⟩)</code>
	<code>\spvec[⟨key = val⟩]{⟨letra⟩} (⟨abscisa, ordenada⟩)</code>	<code>\spvec[⟨key = val⟩]{⟨punto, punto⟩} (⟨abscisa, ordenada⟩)</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` (minúscula) o `{⟨punto, punto⟩}` junto al *argumento opcional delimitado por paréntesis* `(⟨abscisa, ordenada⟩)` el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma `{⟨punto, punto⟩}` con prima(s) ‘`’` o subíndice(s) ‘`_{}...`’ opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (minúscula) con subíndice ‘`_{}...`’ opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spvec{a}` imprimirá ‘ $\vec{a}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector a »; `\spvec{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ y verbalizará «vector $A B$ ».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spvec{a}(-3,4)` imprimirá ‘ $\vec{a} = (-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector a igual a menos tres coma cuatro»; `\spvec{AB}(-3,4)` imprimirá ‘ $\overrightarrow{AB} = (-3, 4)$ ’ y verbalizará «vector $A B$ igual a menos tres coma cuatro».

Llaves para `\spvec`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spvec{AB}` se verbaliza «vector $A B$ »; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «vector». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`autofit = {⟨true | false⟩}` default: *false*

Establece si se *utiliza* `\vphantom{b}` al ejecutar el comando cuando se utiliza el argumento `{⟨letra⟩}`. Si se establece en *true*, se aplicará `\vphantom{b}` al inicio del argumento `{⟨letra⟩}`; si se establece en *false*, no se aplicará `\vphantom{b}`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el `{⟨argumento⟩}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprecta`

<code>\sprecta</code>	<code>\sprecta{⟨punto, punto⟩}</code>	<code>\sprecta{⟨letra⟩}</code>
	<code>\sprecta[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sprecta[⟨key = val⟩]{⟨letra⟩}</code>

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` o `{⟨punto, punto⟩}` con prima(s) ‘`’` o subíndice(s) ‘`_{}...`’ opcionales. Si el *argumento* es de la forma `{⟨punto, punto⟩}` se aplica `\overleftarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (mayúscula o minúscula), se imprimirá solo la `{⟨letra⟩}` sin aplicar `\overleftarrow`.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\sprecta{AB}` imprimirá ‘ $\overleftarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «recta $A B$ »; `\sprecta{m}` imprimirá ‘ m ’ en la salida PDF y NVDA/MathCAT verbalizará «recta m ».

Llaves para `\sprecta`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto, punto⟩}` y el argumento `{⟨letra⟩}` cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{AB}` se verbaliza «recta $A B$ »; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprayo`

`\sprayo {⟨punto, punto⟩}`
`\sprayo [⟨key = val⟩] {⟨punto, punto⟩}`

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\sprayo{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\sprayo`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {⟨*punto, punto*⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{AB}` se verbaliza «rayo A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {⟨rayo | semirrecta⟩}`

default: *rayo*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{AB}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\sprayo[read-sty=semirrecta]{AB}`, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ‘ $\overrightarrow{AB}$ ’ en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside {⟨punto, punto⟩}`
`\spside [⟨key = val⟩] {⟨punto, punto⟩}`

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spside{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spside{AB}` se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «lado A B»; si se utiliza `\spside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza `\spside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «trazo A B»

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spside[read-arg={hipotenusa}]{AB}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` `\spmside{<punto, punto>}`
 `\spmside[<key = val>]{<punto, punto>}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) ' _{...}' opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* $\overline{AB}$; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmside{AB}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «medida del lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «medida del» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «medida del lado A B»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «medida del segmento A B» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «medida del trazo A B»

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «medida del», el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `$\spmside[concept=false,read-arg={medida de la cuerda}]{AB}$` imprimirá ‘AB’ en la salida PDF y NVDA/MathCAT verbalizará «medida de la cuerda A B».

`print-m = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* la notación visual ‘m’ antes aplicar `\overline` al {⟨argumento⟩}. Si se establece en *true*, `$\spmside[print-m=true]{AB}$` imprimirá ‘m $\overline{AB}$ ’ en la salida PDF, si se establece en *false* imprimirá ‘AB’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del lado A B».

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del «concepto» establecido por la llave `read-sty` y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<code>\spangle</code>	<code>{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
<code>\spangle</code>	<code>[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma {⟨punto | letra griega | número⟩} o {⟨punto, punto, punto⟩} aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma {⟨punto⟩} o {⟨punto, punto, punto⟩} admiten prima(s) ‘’ o subíndice(s) ‘_{}...’ opcionales.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spangle{A}$` imprimirá ‘∠A’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `$\spangle{\alpha}$` imprimirá ‘∠α’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa»; `$\spangle{1}$` imprimirá ‘∠1’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo uno» y `$\spangle{AOB}$` imprimirá ‘∠AOB’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

- Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {⟨punto⟩} o {⟨punto, punto, punto⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\spangle{AOB}$` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`recto = {⟨true | false⟩}`

default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `$\spangle[recto=true]{A}$` imprimirá ‘⊥A’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `$\spangle[recto=true]{AOB}$` imprimirá ‘⊥AOB’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto «ángulo» y el {⟨argumento⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨argumento⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

```
\spmangle <{punto | letra griega | número}> <{punto, punto, punto}>
\spmangle[key = val]{<punto | letra griega | número>} \spmangle[key = val]{<punto, punto, punto>}
```

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{<punto | letra griega | número>}` o `{<punto, punto, punto>}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{<punto>}` o `{<punto, punto, punto>}` admiten prima(s) ‘`’` o subíndice(s) ‘`_`’ opcionales.

Si no se proporciona el *argumento opcional* `[key = val]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto>}` o `{<punto, punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

```
recto = {<true | false>} default: false
```

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A»; `\spmangle[recto=true]{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

```
print-m = {<true | false>} default: false
```

Establece si se *imprime* la notación visual ‘m’ antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en `true`, `\spmangle[print-m=true]{AOB}` imprimirá ‘m $\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

```
concept = {<true | false>} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» seguido por el concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del ángulo» y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

```
silent = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del concepto «ángulo» y del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

```
\spang <{grados}>
\spang <{grados : minutos}>
\spang <{grados : minutos : segundos}>
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<grados>}`, `{<grados : minutos>}` o `{<grados : minutos : segundos>}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

<code>\sparc</code>	<code>\sparc{⟨punto, punto⟩}</code>	<code>\sparc{⟨punto, punto, punto⟩}</code>
	<code>\sparc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sparc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\sparc{AB}` imprimirá $\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{AB}` se verbaliza «arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

<code>\spmarc</code>	<code>\spmarc{⟨punto, punto⟩}</code>	<code>\spmarc{⟨punto, punto, punto⟩}</code>
	<code>\spmarc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\spmarc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales colocando `'m'` antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmarc{AB}` imprimirá $m\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* $\widehat{AB}$; `\spmarc` representa su *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{AB}` se verbaliza «medida del arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

```
\spPoly {<punto, punto, punto, ...>}
\spPoly[<key = val>]{<punto, punto, punto, ...>}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto, punto, ...>}`, una lista de al menos tres `{<puntos>}` con prima(s) `'_'` o subíndice(s) `'_{...}'` opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «*triángulo*» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el concepto «*cuadrado*» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo y se añade a la verbalización el nombre del polígono: «*pentágono*», «*hexágono*», «*heptágono*» u «*octágono*» de cinco a ocho puntos, y «*polígono*» desde nueve puntos.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spPoly{ABC}` imprimirá `'\triangle ABC'` en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly{ABCD}` imprimirá `'\square ABCD'` y verbalizará «*cuadrado A B C D*»; `\spPoly{ABCDE}` imprimirá `'ABCDE'` y verbalizará «*pentágono A B C D E*».

Llaves para `\spPoly`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto, punto, ...>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{ABC}` se verbaliza «*triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el `{<argumento>}` o el concepto «*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
concept = {<true | false>} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto («*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*») acorde al número de puntos en el `{<argumento>}` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

```
print-sym = {<true | false>} default: true
```

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión sin modificar la verbalización de este. Si se establece en `true`, `\spPoly[print-sym=true]{ABC}` imprimirá `'\triangle ABC'` en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly[print-sym=true]{ABCD}` imprimirá `'\square ABCD'` en la salida PDF y NVDA/MathCAT verbalizará «*cuadrado A B C D*»; si se establece en `false`, solo se imprimirá el `{<argumento>}` en la salida PDF y la verbalización será la misma.

🔊 Esta llave no tiene efecto con cinco o más puntos, porque no hay un símbolo definido para ellos.

```
silent = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

```
\spAfig {<punto, punto, punto, ...>}
\spAfig[<key = val>]{<punto, punto, punto, ...>}
\spAfig(<1 | 2 | 3 | ... | n>)
\spAfig[<key = val>](<1 | 2 | 3 | ... | n>)
\spAfig(<F | P | B | L | T>)
\spAfig[<key = val>](<F | P | B | L | T>)
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(<1 | 2 | 3 | ... | n>)`, `(<punto, punto, punto, ...>)` o `(<F | P | B | L | T>)` aplicando `\mathcal{A}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al `(<argumento>)` y la llave `concept`.

Si se proporciona el *argumento* `(<1 | 2 | 3 | ... | n>)`, `\spAfig(1)` imprimirá `'\mathcal{A}_1'` en la salida PDF y NVDA/MathCAT verbalizará «*área uno*»; `\spAfig(n)` imprimirá `'\mathcal{A}_n'` en la salida PDF y NVDA/MathCAT verbalizará «*área ene*».

Si se proporciona el *argumento* `(<punto, punto, punto, ...>)`, `\spAfig(ABC)` imprimirá `'\mathcal{A}_{\triangle ABC}'` en la salida PDF y NVDA/MathCAT verbalizará «*área del triángulo A B C*»; `\spAfig(ABCD)` imprimirá `'\mathcal{A}_{\square ABCD}'` en la salida PDF y NVDA/MathCAT verbalizará «*área del cuadrado A B C D*»;

Si se proporciona el *argumento* $((F|P|B|L|T))$, $\$ \backslash \text{spAfig}(F) \$$ imprimirá ' $\mathcal{A}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura»; $\$ \backslash \text{spAfig}(F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura uno».

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$ ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spAfig} \$$ imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área».

- El *argumento* $((\text{punto}, \text{punto}, \text{punto}, \dots))$ acepta prima(s) ' $'$ ' o subíndice(s) ' $_{\{ \dots \}}$ '. El *argumento* $((F|P|B|L|T))$ solo acepta subíndices ' $_{\{ \dots \}}$ ' y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para $\backslash \text{spAfig}$

`read-cmd` = $\{ \langle \text{school} \mid \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spAfig}(\text{ABC}) \$$ se verbaliza «área del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área del» y el concepto «nombre de la figura» acorde al $((\text{argumento}))$ pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves ' $\{ \}$ '.

`concept` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del» y el concepto «nombre de la figura» acorde al $((\text{argumento}))$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «área del», el concepto «nombre de la figura» y el $((\text{argumento}))$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $((\text{argumento}))$ acorde al valor establecido por la llave `read-cmd`.

`read-arg` = $\{ \langle \text{nombre} \rangle \}$ default: *no usado*

Sobrescribe el término «área del» y el concepto «nombre de la figura» cuando se utiliza el argumento $((\text{punto}, \text{punto}, \text{punto}, \dots))$ que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ' $\{ \}$ ' y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spAfig}[\text{read-arg}=\{\text{trapezio}\}](\text{ABCD}) \$$ imprimirá ' $\mathcal{A}_{\square \text{ABCD}}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapezio A B C D».

- Esta llave no se puede utilizar con los argumentos $((F|P|B|L|T))$ y $((1|2|3|\dots|n))$.

`concept*` $\langle \text{valor prohibido} \rangle$ default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al $((\text{argumento}))$ pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «área» y el $((\text{argumento}))$ acorde al valor establecido por la llave `read-cmd`.

`read-sub` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos $((1|2|3|\dots|n))$ y $((F|P|B|L|T))$ para NVDA/MathCAT. Si se utiliza $((1|2|3|\dots|n))$ y se establece en *true*, $\$ \backslash \text{spAfig}[\text{read-sub}=\text{true}](1) \$$ imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «área sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «área uno».

Si se utiliza $((F|P|B|L|T))$ y se establece en *true*, $\$ \backslash \text{spAfig}[\text{read-sub}=\text{true}](F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «área de la figura uno».

`print-sym` = $\{ \langle \text{auto} \mid \text{triangle} \mid \text{square} \mid \text{none} \rangle \}$ default: *auto*

Establece si se *usará* $\backslash \text{triangle}$ o $\backslash \text{mdlgwhtsquare}$ en la impresión para los argumentos $((\text{punto}, \text{punto}, \text{punto}, \dots))$ y $((1|2|3|\dots|n))$ pasado al comando sin modificar la verbalización de este. Si se utiliza $((\text{punto}, \text{punto}, \text{punto}, \dots))$ se infiere el *símbolo* acorde al número de puntos; si establece en *auto* se imprime y si se establece en *none* se omite: $\$ \backslash \text{spAfig}[\text{print-sym}=\text{auto}](\text{ABC}) \$$ y $\$ \backslash \text{spAfig}[\text{print-sym}=\text{none}](\text{ABC}) \$$ imprimirán ' $\mathcal{A}_{\triangle \text{ABC}}$ ' y ' $\mathcal{A}_{\text{ABC}}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «área del triángulo A B C».

Si se utiliza $((1|2|3|\dots|n))$ los valores *auto* y *none* quedan sin efecto; si se establece en *triangle* o *square* anida el argumento bajo el *símbolo*: $\$ \backslash \text{spAfig}[\text{print-sym}=\text{triangle}](1) \$$ imprimirá ' $\mathcal{A}_{\triangle 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del triángulo uno»; $\$ \backslash \text{spAfig}[\text{print-sym}=\text{square}](1) \$$ imprimirá ' $\mathcal{A}_{\square 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del cuadrado uno».

- Esta llave no se puede utilizar con el argumento $((F|P|B|L|T))$.

`silent` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del», el concepto «nombre de la figura» y el $((\text{argumento}))$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(\langle punto, punto, punto, \dots \rangle)</code>
<code>\spPfig[\langle key = val \rangle]</code>	<code>\spPfig[\langle key = val \rangle](\langle punto, punto, punto, \dots \rangle)</code>	<code>\spPfig[\langle key = val \rangle](\langle F P B L T \rangle)</code>
<code>\spPfig(\langle 1 2 3 \dots n \rangle)</code>	<code>\spPfig(\langle F P B L T \rangle)</code>	
<code>\spPfig[\langle key = val \rangle](\langle 1 2 3 \dots n \rangle)</code>	<code>\spPfig[\langle key = val \rangle](\langle F P B L T \rangle)</code>	

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle 1 | 2 | 3 | \dots | n \rangle)$, $(\langle punto, punto, punto, \dots \rangle)$ o $(\langle F | P | B | L | T \rangle)$ aplicando `\mathcal{P}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $(\langle argumento \rangle)$ y la llave `concept`.

Si se proporciona el *argumento* $(\langle 1 | 2 | 3 | \dots | n \rangle)$, `\spPfig(1)` imprimirá $\mathcal{P}_1$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro uno»; `\spPfig(n)` imprimirá $\mathcal{P}_n$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro ene».

Si se proporciona el *argumento* $(\langle punto, punto, punto, \dots \rangle)$, `\spPfig(ABC)` imprimirá $\mathcal{P}_{\triangle ABC}$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo A B C»; `\spPfig(ABCD)` imprimirá $\mathcal{P}_{\square ABCD}$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado A B C D»;

Si se proporciona el *argumento* $(\langle F | P | B | L | T \rangle)$, `\spPfig(F)` imprimirá $\mathcal{P}_F$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura»; `\spPfig(F_{1})` imprimirá $\mathcal{P}_{F_1}$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura uno».

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$ ni el *argumento opcional delimitado por paréntesis*, `\spPfig` imprimirá $\mathcal{P}$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro».

- El *argumento* $(\langle punto, punto, punto, \dots \rangle)$ acepta prima(s) $'_{'$ o subíndice(s) $'_{\{...\}}$. El *argumento* $(\langle F | P | B | L | T \rangle)$ solo acepta subíndices $'_{\{...\}}$ y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para `\spPfig`

`read-cmd` = $\{\langle school | mathcat \rangle\}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(ABC)` se verbaliza «perímetro del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{\langle prefijo \rangle\}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «perímetro del» y el concepto «nombre de la figura» acorde al $(\langle argumento \rangle)$ pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves $\{\}$.

`concept` = $\{\langle true | false \rangle\}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del» y el concepto «nombre de la figura» acorde al $(\langle argumento \rangle)$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «perímetro del», el concepto «nombre de la figura» y el $(\langle argumento \rangle)$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $(\langle argumento \rangle)$ acorde al valor establecido por la llave `read-cmd`.

`read-arg` = $\{\langle nombre \rangle\}$ default: *no usado*

Sobrescribe el término «perímetro del» y el concepto «nombre de la figura» cuando se utiliza el argumento $(\langle punto, punto, punto, \dots \rangle)$ que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves $\{\}$ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spPfig[read-arg=\langle trapecio \rangle](ABCD)` imprimirá $\mathcal{P}_{\square ABCD}$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos $(\langle F | P | B | L | T \rangle)$ y $(\langle 1 | 2 | 3 | \dots | n \rangle)$.

`concept*` $\langle valor prohibido \rangle$ default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al $(\langle argumento \rangle)$ pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «perímetro» y el $(\langle argumento \rangle)$ acorde al valor establecido por la llave `read-cmd`.

`read-sub` = $\{\langle true | false \rangle\}$ default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos $(\langle 1 | 2 | 3 | \dots | n \rangle)$ y $(\langle F | P | B | L | T \rangle)$ para NVDA/MathCAT. Si se utiliza $(\langle 1 | 2 | 3 | \dots | n \rangle)$ y se establece en *true*, `\spPfig[read-sub=true](1)` imprimirá $\mathcal{P}_1$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «perímetro uno».

Si se utiliza $(\langle F | P | B | L | T \rangle)$ y se establece en *true*, `\spPfig[read-sub=true](F_{1})` imprimirá $\mathcal{P}_{F_1}$ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «perímetro de la figura uno».

`print-sym` = $\{\langle auto | triangle | square | none \rangle\}$

default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos (`\punto`, `punto`, `punto`, ...) y (`\1` | `2` | `3` | ... | `n`) pasado al comando sin modificar la verbalización de este. Si se utiliza (`\punto`, `punto`, `punto`, ...) se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: `\spPfig[print-sym=auto](ABC)` y `\spPfig[print-sym=none](ABC)` imprimirán ' $\mathcal{P}_{\triangle ABC}$ ' y ' $\mathcal{P}_{ABC}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «perímetro del triángulo A B C»

Si se utiliza (`\1` | `2` | `3` | ... | `n`) los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: `\spPfig[print-sym=triangle](1)` imprimirá ' $\mathcal{P}_{\triangle_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo uno»; `\spPfig[print-sym=square](1)` imprimirá ' $\mathcal{P}_{\square_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado uno».

Esta llave no se puede utilizar con el argumento (`\F` | `\P` | `\B` | `\L` | `\T`).

`silent = {\true | false}`

default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del», el concepto «nombre de la figura» y el `\argumento` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprad` y `\spdiam`

<code>\sprad</code>	<code>\sprad</code>	<code>\sprad[\langle key = val \rangle]</code>
<code>\spdiam</code>	<code>\spdiam</code>	<code>\spdiam[\langle key = val \rangle]</code>

Los comandos `\sprad` y `\spdiam` trabajan en *modo matemático* y reciben el *argumento opcional* [`\langle key = val \rangle`] mediante el cual se establece la «forma» en la que se imprimirán en la salida PDF las letras '*r*' y '*d*' y como se verbalizarán en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`\langle key = val \rangle`], `\sprad` imprimirá '*r*' en la salida PDF y NVDA/MathCAT verbalizará «radio»; `\spdiam` imprimirá '*d*' en la salida PDF y NVDA/MathCAT verbalizará «diámetro».

Llaves para `\sprad` y `\spdiam`

`concept = {\true | false}`

default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «radio» o «diámetro» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\sprad` y `\spdiam` se verbalizan como «radio» y «diámetro» en NVDA/MathCAT; si se establece en `false`, se verbalizarán las letras '*r*', '*R*', '*d*' o '*D*' acorde al valor establecido por la llave `read-cmd`.

`read-cmd = {\school | mathcat}`

default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará las letras '*R*' o '*D*' cuando `concept=false` y `upper-case=true`. Si se establece en `school`, NVDA/MathCAT verbalizará las letras '*R*' o '*D*' sin el prefijo «mayúscula»: si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`upper-case = {\true | false}`

default: false

Establece si se *imprime* usando mayúsculas. Si se establece en `true`, `\sprad[upper-case=true]` y `\spdiam[upper-case=true]` imprimirán '*R*' y '*D*' en la salida PDF; si se establece en `false` se imprimirá usando minúsculas.

La verbalización al establecer esta llave es controlada por la llave `read-cmd`.

`print-sym = {\true | false}`

default: false

Establece si se usará `\diameter` antes de la letra '*d*' o '*D*' en la impresión del comando `\spdiam`. Si se establece en `true`, `\spdiam[print-sym=true]` imprimirá ' $\varnothing d$ ' en la salida PDF y NVDA/MathCAT verbalizará «diámetro».

La llave `print-sym` no tiene ningún efecto en `\sprad`.

`sub-index = {\subíndice}`

default: no usado

Establece un *subíndice* sobre las letras '*r*', '*R*', '*d*' o '*D*'. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\sprad[sub-index={1}]` imprimirá ' r_1 ' en la salida PDF y NVDA/MathCAT verbalizará «radio uno».

`read-sub = {\true | false}`

default: false

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\sprad[sub-index={1}, read-sub=true]` se verbalizará «radio sub uno» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «radio uno».

`read-arg = {\verbalización NVDA}`

default: no usado

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe

usar en conjunto con la llave `concept`. Por ejemplo: `\sprad[read-arg={medida exacta}]$` imprimirá ‘*r*’ en la salida PDF y NVDA/MathCAT verbalizará «*radio medida exacta*»; `\sprad[concept=false,read-arg={medida exacta}]$` imprimirá ‘*r*’ y NVDA/MathCAT verbalizará «*medida exacta*».

Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {\true | false}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spcirc`

<code>\spcirc</code>	<code>\spcirc</code>	<code>\spcirc[⟨key = val⟩]</code>
<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro⟩)</code>	
<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro, radio⟩)</code>	

El comando `\spcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨centro⟩)` o `(⟨centro, radio⟩)`, donde `⟨radio⟩` es opcional y debe pasarse junto al argumento `(⟨centro⟩)` separado por una coma ‘,’ obligatoria.

La forma en la que se procesa el `⟨radio⟩` junto a su impresión y verbalización se establece de manera interna: Si es un `⟨número⟩` entero o decimal se usa `\spnum`; si es una `⟨medida⟩` se usa `\spunit`; si es un segmento `⟨punto punto⟩` se usa `\spside` o `\spmside` acorde al valor establecido por la llave `radio-sty`; y si es una letra se imprime tal cual. Cualquier otra forma devuelve un “error” y se detiene la compilación del documento.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]` ni el *argumento opcional delimitado por paréntesis*, `\spcirc$` imprimirá ‘ $\odot$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia*».

Si se proporciona el *argumento* `(⟨centro⟩)`, `\spcirc(0)$` imprimirá ‘ $\odot(O)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O*».

Si se proporciona el *argumento* `(⟨centro, radio⟩)`, `\spcirc(0,3)$` imprimirá ‘ $\odot(O,3)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O y radio tres*».

El argumento `(⟨centro⟩)` y el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩` aceptan prima ‘`’` o subíndice ‘`_{\dots}`’.

Llaves para `\spcirc`

El comando `\spcirc` posee las *meta-keys* `spside` pasada al comando `\spside` (v. pág. 21) y `spmside` pasada al comando `\spmside` (v. pág. 22).

`read-cmd = {\school | mathcat}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `(⟨centro⟩)`. Si se establece en `school`, NVDA/MathCAT verbaliza el argumento `(⟨centro⟩)` sin el prefijo «mayúscula»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

Esta llave tiene efecto sobre el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩`.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`print-sym = {\usual | odot}` default: odot

Establece si se *usará* `\bigodot` o la letra ‘*C*’ en la impresión del comando sin modificar la verbalización de este. Si se establece en `usual`, `\spcirc[print-sym=usual](0)$` imprimirá ‘*C*(*O*)’ en la salida PDF; si se establece en `odot`, `\spcirc[print-sym=odot](0)$` imprimirá ‘ $\odot(O)$ ’ en la salida PDF.

`read-sty = {\circun | circle}` default: circun

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se establece en `circun`, `\spcirc[read-sty=circun](0)$` se verbalizará «*circunferencia de centro en O*» en NVDA/MathCAT; si se establece en `circle`, `\spcirc[read-sty=circle](0)$` se verbalizará «*círculo de centro en O*» en NVDA/MathCAT.

`diameter = {\true | false}` default: false

Establece *como* se verbaliza el argumento `⟨radio⟩` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spcirc[diameter=true](0,3)$` se verbalizará «*circunferencia de centro en O y diámetro tres*» en NVDA/MathCAT; si se establece en `false`, `\spcirc[diameter=false](0,3)$` se verbalizará «*circunferencia de centro en O y radio tres*» en NVDA/MathCAT.

`radio-sty = {\side | mside}` default: mside

Establece el *comando* que procesa internamente el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩`. Si se establece en `side` se usará `\spside{⟨punto punto⟩}`, `\spcirc[radio-sty=side](O,AB)$` imprimirá ‘ $\odot(O,AB)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O y radio lado A B*»; si se

establece en `m-side` se usará `\spmside{⟨punto punto⟩}`, `\spcirc[radio-sty=m-side](O, AB)` imprimirá $\odot(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio medida del lado A B».

- Las *meta-keys* `spside` y `spmside` permiten modificar la impresión y verbalización del argumento *⟨radio⟩* cuando tiene la forma *⟨punto punto⟩*.

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el «concepto» y el argumento (*⟨centro⟩*) o (*⟨centro, radio⟩*); si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento (*⟨centro⟩*) o (*⟨centro, radio⟩*) acorde al valor establecido por la llave `read-cmd`.

`sub-index = {⟨subíndice⟩}` default: no usado

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spcirc[sub-index={1},radio-sty=m-side](O, AB)` imprimirá $\odot_1(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia uno de centro en O y radio medida del lado A B».

`read-sub = {⟨true | false⟩}` default: false

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spcirc[sub-index={1},read-sub=true](O)` se verbalizará «circunferencia sub uno de centro en O» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «circunferencia uno de centro en O».

`read-arg = {⟨verbalización NVDA⟩}` default: no usado

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spcirc[read-arg={disco}](O)` imprimirá $\odot(O)$ en la salida PDF y NVDA/MathCAT verbalizará «disco de centro en O».

`silent = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty`, del argumento (*⟨centro⟩*) o (*⟨centro, radio⟩*) pasados al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAcirc`

<code>\spAcirc</code>	<code>\spAcirc</code>	<code>\spAcirc[⟨key = val⟩]</code>
	<code>\spAcirc(⟨centro⟩)</code>	<code>\spAcirc[⟨key = val⟩](⟨centro⟩)</code>
	<code>\spAcirc(⟨1 2 3 ... n⟩)</code>	<code>\spAcirc[⟨key = val⟩](⟨1 2 3 ... n⟩)</code>

El comando `\spAcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*⟨centro⟩*) o (*⟨1 | 2 | 3 | ... | n⟩*), aplicando `\mathcal{A}` a la impresión.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] ni el *argumento opcional delimitado por paréntesis*, `\spAcirc` imprimirá 'A' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia».

Si se proporciona el *argumento* (*⟨centro⟩*), `\spAcirc(O)` imprimirá $\mathcal{A}_{\odot(O)}$ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia con centro en O».

Si se proporciona el *argumento* (*⟨1 | 2 | 3 | ... | n⟩*), `\spAcirc(1)` imprimirá $\mathcal{A}_{\odot_1}$ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno».

- El *argumento* (*⟨centro⟩*) acepta prima ' ' o subíndice ' _{...}' y no acepta un *⟨radio⟩*. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spAcirc`

`read-cmd = {⟨school | mathcat⟩}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (*⟨centro⟩*). Si se establece en `school`, NVDA/MathCAT verbaliza el argumento (*⟨centro⟩*) sin el prefijo «mayúscula»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty = {⟨circun | circle⟩}` default: circun

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «área de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en `circun`, `\spAcirc[read-sty=circun](O)`

se verbalizará «área de la circunferencia con centro en O » en NVDA/MathCAT; si se establece en `circle`, `$\spAcirc[read-sty=circle](0)$` se verbalizará «área del círculo con centro en O » en NVDA/MathCAT.

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`⟨centro⟩`) o (`⟨1 | 2 | 3 | ... | n⟩`); si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento (`⟨centro⟩`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`⟨centro⟩`) o con el argumento (`⟨1 | 2 | 3 | ... | n⟩`) es equivalente a usar `silent=true`.

`read-arg = {⟨verbalización NVDA⟩}` default: no usado

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `$\spAcirc[read-arg={esfera}](0)$` se verbalizará «área de la esfera con centro en O » en NVDA/MathCAT; `$\spAcirc[concept=false,read-arg={esfera}](0)$` se verbalizará «esfera con centro en O » en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`⟨1 | 2 | 3 | ... | n⟩`) es equivalente a usar `silent=true`.

`print-sym = {⟨usual | odot⟩}` default: odot

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en `usual`, `$\spAcirc[print-sym=usual](0)$` imprimirá ' $\mathcal{A}_{C(O)}$ ' en la salida PDF; si se establece en `odot`, `$\spAcirc[print-sym=odot](0)$` imprimirá ' $\mathcal{A}_{O(O)}$ ' en la salida PDF.

`sub-index = {⟨subíndice⟩}` default: no usado

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`⟨centro⟩`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$\spAcirc[sub-index={1}](0)$` imprimirá ' $\mathcal{A}_{O_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno con centro en O ».

`read-sub = {⟨true | false⟩}` default: false

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`⟨1 | 2 | 3 | ... | n⟩`) al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `$\spAcirc[read-sub=true](1)$` se verbalizará «área de la circunferencia sub uno» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «área de la circunferencia uno».

`silent = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...», del concepto establecido por la llave `read-sty` y del argumento (`⟨centro⟩`) o (`⟨1 | 2 | 3 | ... | n⟩`) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT si verbalizará la salida del comando.

El comando `\spPcirc`

<code>\spPcirc</code>	<code>\spPcirc</code>	<code>\spPcirc[⟨key = val⟩]</code>
	<code>\spPcirc(⟨centro⟩)</code>	<code>\spPcirc[⟨key = val⟩](⟨centro⟩)</code>
	<code>\spPcirc(⟨1 2 3 ... n⟩)</code>	<code>\spPcirc[⟨key = val⟩](⟨1 2 3 ... n⟩)</code>

El comando `\spPcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`⟨centro⟩`) o (`⟨1 | 2 | 3 | ... | n⟩`), aplicando `\mathcal{P}` a la impresión.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`] ni el *argumento opcional delimitado por paréntesis*, `$\spPcirc$` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia».

Si se proporciona el *argumento* (`⟨centro⟩`), `$\spPcirc(0)$` imprimirá ' $\mathcal{P}_{O(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia con centro en O ».

Si se proporciona el *argumento* (`⟨1 | 2 | 3 | ... | n⟩`), `$\spPcirc(1)$` imprimirá ' $\mathcal{P}_{O_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

- El *argumento* (`⟨centro⟩`) acepta prima '' o subíndice '`_{\dots}`' y no acepta un *radio*. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spPcirc`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (`<centro>`). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento (`<centro>`) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*perímetro de ...*» seguido por el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty` = { `<circun | circle>` }

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «*perímetro de ...*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spPcirc[read-sty=circun](0)` se verbalizará «*perímetro de la circunferencia con centro en O*» en NVDA/MathCAT; si se establece en *circle*, `\spPcirc[read-sty=circle](0)` se verbalizará «*perímetro del círculo con centro en O*» en NVDA/MathCAT.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro de ...*» y del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento (`<centro>`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`<centro>`) o con el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`read-arg` = { `<verbalización NVDA>` }

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spPcirc[read-sty=circle,read-arg={disco}](0)` se verbalizará «*perímetro del disco con centro en O*» en NVDA/MathCAT; `\spPcirc[concept=false,read-arg={disco}](0)` se verbalizará «*disco con centro en O*» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`print-sym` = { `<usual | odot>` }

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spPcirc[print-sym=usual](0)` imprimirá ' $\mathcal{P}_{C(O)}$ ' en la salida PDF; si se establece en *odot*, `\spPcirc[print-sym=odot](0)` imprimirá ' $\mathcal{P}_{\odot(O)}$ ' en la salida PDF.

`sub-index` = { `<subíndice>` }

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`<centro>`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spPcirc[sub-index={1}](0)` imprimirá ' $\mathcal{P}_{\odot_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*perímetro de la circunferencia uno con centro en O*».

`read-sub` = { `<true | false>` }

default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`<1 | 2 | 3 | ... | n>`) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\spPcirc[read-sub=true](1)` se verbalizará «*perímetro de la circunferencia sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*perímetro de la circunferencia uno*».

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro de ...*», del concepto establecido por la llave `read-sty` y del argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT si verbalizará la salida del comando.

El comando `\spVol`

`\spVol` `\spVol` `\spVol[<key = val>]`

El comando `\spVol` trabaja en *modo matemático* y recibe el *argumento opcional* [`<key = val>`] mediante el cual se establece la «*forma*» en la que se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spVol` imprimirá ' $\mathcal{V}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*volumen*».

Llaves para `\spVol`

`concept` = { `(true | false)` }

default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*volumen*» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spVol` se verbaliza como «*volumen*» en NVDA/MathCAT; si se establece en `false`, se verbalizará la letra ‘*V*’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd` = { `(school | mathcat)` }

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará la letra ‘*V*’ cuando `concept=false`. Si se establece en `school`, NVDA/MathCAT verbalizará únicamente la letra ‘*V*’; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas sobre ‘*V*’.

`sub-index` = { `(subíndice)` }

default: `no usado`

Establece un *subíndice* sobre ‘*V*’. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\spVol[sub-index={1}]` imprimirá ‘*V*₁’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen uno*».

`read-sub` = { `(true | false)` }

default: `false`

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spVol[sub-index={1}, read-sub=true]` se verbalizará «*volumen sub uno*» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «*volumen uno*».

`read-arg` = { `(verbalización NVDA)` }

default: `no usado`

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spVol[read-arg={medida exacta}]` imprimirá ‘*V*’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen medida exacta*»; `\spVol[concept=false, read-arg={medida exacta}]` imprimirá ‘*V*’ y NVDA/MathCAT verbalizará «*medida exacta*».

📌 Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent` = { `(true | false)` }

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de ‘*V*’ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-10-04 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAcirc	31
\spAfig	26
\spPcirc	32
\spPfig	28
\spPoint	19
\spPoly	26
\spProp	18
\spVol	33
\spangle	23
\spang	24
\sparc	25
\spcirc	30
\spcoord	19
\spdate	11
\spdiam	29
\spdiv	13
\spdsym	12
\spmQ	17
\spmangle	24
\spmarc	25
\spmcd	15
\spmcm	16
\spmoney	6
\spmside	22
\spmsym	13
\spmul	14
\spnD	14
\spnM	15
\spnZ	17
\spnewread	11
\spnewunit	6
\spnum	3
\sprad	29
\sprayo	21
\sprazon	18
\spread	12
\sprecta	20
\spshort	8
\spside	21
\spsiglo	10
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
\spvec	20
read-sty	31
read-sub	32
silent	32
sub-index	32
Keys for \spAfig provide by spintent :	
concept*	27
concept	27
prefix	27
print-sym	27
read-arg	27
read-cmd	27
read-sub	27
silent	27
Keys for \spangle provide by spintent :	
concept	23
prefix	23
read-cmd	23
recto	23
silent	23
Keys for \sparc provide by spintent :	
concept	25
prefix	25
read-cmd	25
silent	25
Keys for \spcirc provide by spintent :	
concept	31
diameter	30
prefix	30
print-sym	30
radio-sty	30
read-arg	31
read-cmd	30
read-sty	30
read-sub	31
silent	31
spmside	30, 31
spside	30, 31
sub-index	31
Keys for \spcoord provide by spintent :	
silent	19
Keys for \spdiam provide by spintent :	
concept	29
print-sym	29
read-arg	29
read-cmd	29
read-sub	29
silent	30
sub-index	29
upper-case	29
Keys for \spdiv provide by spintent :	
cifras-sep	14
notation-sym	14
read-parens	14
read-period-sep	14
read-sign	14
read-sym	14
set-sym	14
wrap-parens	14
Keys for \spdsym provide by spintent :	
prefix	13
read-sym	13

K	
Keys for \spAcirc provide by spintent :	
concept	32
prefix	31
print-sym	32
read-arg	32
read-cmd	31

set-sym	13	result-num	14
suffix	13	result	14
Keys for \spmangle provide by spintent :		silent	15
concept	24	Keys for \spnM provide by spintent :	
prefix	24	concept	15
print-m	24	prefix	15
read-cmd	24	result-num	15
recto	24	result	15
silent	24	silent	15
Keys for \spmarc provide by spintent :		Keys for \spnum provide by spintent :	
concept	25	cifras-sep	3
prefix	25	notation-sym	4
read-cmd	25	read-period-sep	3
silent	25	read-sign	3
Keys for \spmcd provide by spintent :		Keys for \spnZ provide by spintent :	
concept	16	cifras-sep	17
prefix	16	read-parens	17
read-cmd	16	read-sign	17
result	16	wrap-parens	17
silent	16	Keys for \spPcirc provide by spintent :	
upper-case	16	concept	33
Keys for \spmcm provide by spintent :		prefix	33
concept	16	print-sym	33
prefix	16	read-arg	33
read-cmd	16	read-cmd	33
result	16	read-sty	33
silent	17	read-sub	33
upper-case	16	silent	33
Keys for \spmoney provide by spintent :		sub-index	33
cifras-sep	7	Keys for \spPfig provide by spintent :	
currency	7	concept*	28
money-math	7	concept	28
sub-units	7	prefix	28
Keys for \spmQ provide by spintent :		print-sym	28
join-word	17	read-arg	28
read-parens	17	read-cmd	28
use-dfrac	17	read-sub	28
use-spN	17	silent	29
wrap-parens	17, 18	Keys for \spPoint provide by spintent :	
Keys for \spmside provide by spintent :		concept	19
concept	22	prefix	19
prefix	22	read-cmd	19
print-m	23	silent	19
read-arg	23	Keys for \spPoly provide by spintent :	
read-cmd	22	concept	26
read-sty	22	prefix	26
silent	23	print-sym	26
Keys for \spmsym provide by spintent :		read-cmd	26
not-print	13	silent	26
prefix	13	Keys for \spProp provide by spintent :	
read-sym	13	inline	18
set-sym	13	silent	18
suffix	13	use-dfrac	18
Keys for \spmul provide by spintent :		Keys for \sprad provide by spintent :	
cifras-sep	14	concept	29
notation-sym	14	read-arg	29
read-parens	14	read-cmd	29
read-period-sep	14	read-sub	29
read-sign	14	silent	30
read-sym	14	sub-index	29
set-sym	14	upper-case	29
wrap-parens	14	Keys for \spray provide by spintent :	
Keys for \spnD provide by spintent :		concept	21
concept	14	prefix	21
prefix	14	read-cmd	21

read-sty	21	Keys for \spvec provide by spintent :	
silent	21	autofit	20
Keys for \sprazon provide by spintent :		concept	20
inline	18	prefix	20
silent	18	read-cmd	20
use-dfrac	18	silent	20
Keys for \spread provide by spintent :		Keys for \spVol provide by spintent :	
read-arg	12	concept	34
silent	12	read-arg	34
Keys for \sprecta provide by spintent :		read-cmd	34
concept	21	read-sub	34
prefix	20	silent	34
read-cmd	20	sub-index	34
silent	21		
Keys for \spshort provide by spintent :			
read-arg	8		
small-caps	8		
Keys for \spside provide by spintent :			
concept	22		
prefix	22		
read-arg	22		
read-cmd	22		
read-sty	22		
silent	22		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
cifras-sep	4		
notation-sym	4		
print-cdot	4		
read-cdot	4		
read-period-sep	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		
size	12		

M

Lua module provide by **spintent**:

spintent.lua

6–11

P

Packages :

babel-spanish

babel

documentmetadata-support

enumext

expl3

fontspec

fourier-otf

hyperref

l3keys

latex-lab-mathintent

libertinus-otf

luamml

scontents

sourcecodepro

tagpdf

unicode-math

2

2, 8

1

1

3

2

2

2

3

1

2

1

1

2

1, 8, 11

1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-10-04} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_LTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34     \IfPackageLoadedF { unicode-math }
35     {
36       \IfPackageLoadedF { lua-unicode-math }
37       {
38         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
39         \RequirePackage{unicode-math}
```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

```

\str_if_eq:nVTF \str_if_eq:nVF
\str_uppercase:V \str_lowercase:V
\seq_set_from_clist:Ne
\cs_generate_variant:Nn \str_if_eq:nnTF { V }
\cs_generate_variant:Nn \str_if_eq:nnF { V }
\cs_generate_variant:Nn \str_uppercase:n { V }
\cs_generate_variant:Nn \str_lowercase:n { V }
\cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal`, `\rightanglesqr` and `\attribute`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`), the string variable `\l__spintent_luafun_right_angle_sqr` used by the function `\__spintent_luafun_right_angle_sqr:` to copy `\rightanglesqr` alongside the string variable `\l__spintent_luaset_scope_attr_str` used by `\attribute` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str
69 \str_new:N \l__spintent_luaset_scope_attr_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str`, `\l__spintent_luaset_sym_fam_ok_str`, and `\l__spintent_luaset_scope_attr_str`.)

11.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

70 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for `tagged PDF` — `math-AF` mode does not want either ghost present.

```

71 \bool_if:NTF \l__spintent_debug_ghost_bool
72 { \__spintent_luafun_set_debug_ghost:n { true } }
73 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

74 \int_new:N \l__spintent_scope_attr_int
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_luaset_scope_attr_str }

```

(End of definition for `\__spintent_scope_attr_int`.)

`\__spintent_luafun_spunit_lookup_alias:n` Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_spmoney_lookup_data:n` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean_split_and_set:n` (used by `\spnum`).

```

76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:n` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

`mathcal-fallback`
`right-angle-sqr-fallback`

First, we'll define two error messages in case our fallback fails.

```

80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=` (`#2`). A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0" "0" "2063
127 }
```

🔗 Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

11.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     \__spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn`
`\__spintent_sanitize_affix:cn`

The function `\__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, it converts the $\{\langle argument \rangle\}$ into a string (so every character has category code 12 and it no longer matters how an underscore ‘_’ was typed), trims the bounds, converts spaces ‘ ’ and underscores ‘_’ to hyphens ‘-’, collapses multiple hyphens, and safely strips leading/trailing hyphens. Because underscores are converted *before* collapsing, any run mixing hyphens and underscores is reduced to a single hyphen, and the stored value never starts or ends with ‘-’ or ‘_’: the command that uses the value is the one that places the leading ‘_’ of the intent name. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\__spintent_spsomesym_prefix_tl`, ready to be conditionally appended. In the same way, `{ __foo__bar__ }` is stored as `foo-bar`. The only exception is the exact value ‘_-’ (once the bounds are trimmed), which is stored as a single hyphen ‘-’. After the command prepends its own leading ‘_’, this yields the *silent* literal ‘_-’ documented for `\spread`. Any other value made only of hyphens, underscores and spaces is stored as *empty*.

```

136 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Nc #1 { \tl_to_str:e {#2} }
142     \tl_trim_spaces:N #1
143     \str_if_eq:VnTF #1 { _- }
144     { \tl_set:Nn #1 { - } }
145     {
146       \tl_replace_all:Nnn #1 { ~ } { - }
147       \tl_replace_all:Nee #1 { \tl_to_str:n { _ } } { - }
148       \__spintent_collapse_hyphens:N #1
149       \tl_put_left:Nn #1 { \q_mark }
150       \tl_put_right:Nn #1 { \q_stop }
151       \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
152       \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
153       \tl_remove_all:Nn #1 { \q_mark }
154       \tl_remove_all:Nn #1 { \q_stop }
155     }
156   }
157 }
158 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\__spintent_setspintent_input_arg_cmd_tl` The variable `\__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```

159 \tl_new:N \__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\langle argument \rangle$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

160 \msg_new:nnnn { spintent } { unknown-command-setup }
161 {
162     Cannot~configure~unknown~command~'\exp_not:c{#1}'.
163 }
164 {
165     You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
166     command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
167 }

```

(End of definition for `\__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as $\langle argument \rangle$ exists in ‘#1’ and set the $\langle keys \rangle$ passed as $\langle argument \rangle$ in ‘#2’ to the correct path; otherwise, it will return an error.

```

168 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
169 {
170     \cs_if_exist:cTF { #1 }
171     {
172         \keys_set:nn { spintent / #1 } { #2 }
173     }
174     {
175         \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
176     }
177 }
178 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

179 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
180 {
181     \tl_if_single:nTF { #1 }
182     {
183         \token_if_cs:NTF #1
184         {
185             \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
186         }
187         {
188             \__spintent_setspintent_set_keys:nn { #1 } { #2 }
189         }
190     }
191     {
192         \__spintent_setspintent_set_keys:nn { #1 } { #2 }
193     }
194 }
195 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent` Now we define the user command `\setspintent`.

```

196 \NewDocumentCommand \setspintent { m m }
197 {
198     \tl_if_blank:nTF { #1 }
199     {
200         \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
201     }
202     {
203         \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
204         \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
205         \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
206     }
207 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```
\l__spintent_command_name_str
    unknown-choice
    cmd-key-unknown
    cmd-key-value-unknown
```

We define the variable `\l__spintent_command_name_str` which will *store* the “*name*” of each command within the implementation along with the messages `unknown-choice` used by *(keys)* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *(key)* `unknown` in commands that use `[key = val]`.

```
208 \str_new:N \l__spintent_command_name_str
209 \msg_new:nnn { spintent } { unknown-choice }
210 {
211     The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
212 }
213 \msg_new:nnnn { spintent } { cmd-key-unknown }
214 {
215     The~key~'#1'~is~unknown~by~command~
216     \c_backslash_str \l__spintent_command_name_str \c_space_tl
217     and~is~being~ignored~\msg_line_context:.
218 }
219 {
220     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
221     does~not~have~a~key~called~'#1'.\\
222     Check~that~you~have~spelled~the~key~name~correctly.
223 }
224 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
225 {
226     The~key~'#1=#2'~is~unknown~by~command~
227     \c_backslash_str \l__spintent_command_name_str \c_space_tl
228     and~is~being~ignored~\msg_line_context:.
229 }
230 {
231     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
232     does~not~have~a~key~called~'#1'.\\
233     Check~that~you~have~spelled~the~key~name~correctly.
234 }
```

(End of definition for `\l__spintent_command_name_str` and others.)

```
\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n
```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the *(key)* `unknown`.

```
235 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
236 {
237     \tl_if_blank:nTF {#2}
238     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
239     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
240 }
241 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
242 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
243 {
244     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
245 }
```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

11.12 Definition of core base variables

```
\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V
```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the *{(argument)}* passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “*units*”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘`_str`’ or ‘`_tl`’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```
\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str
```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘`+`’ or ‘`-`’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “*integer part*”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘`,`’ or ‘`.`’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “*finite decimal part*” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “*infinite periodic decimal part*” that follow a semicolon ‘`;`’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

246 \tl_new:N \l__spintent_luaset_sign_tl
247 \tl_new:N \l__spintent_luaset_part_int_tl
248 \tl_new:N \l__spintent_luaset_dec_sep_tl
249 \tl_new:N \l__spintent_luaset_part_dec_tl
250 \tl_new:N \l__spintent_luaset_part_period_tl
251 \str_new:N \l__spintent_luaset_has_integer_str
252 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

`\l__spintent_luaset_only_part_int_str` The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\l__spintent_luaset_only_part_int_str` for the “integer part”, in `\l__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\l__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

`\l__spintent_luaset_only_part_dec_str` Formatted versions (`cifras-sep=true`) are stored in `\l__spintent_luaset_format_part_int_str` for the “integer part” and `\l__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

`\l__spintent_luaset_format_part_int_str`

`\l__spintent_luaset_format_part_dec_str`

```

253 \str_new:N \l__spintent_luaset_only_part_int_str
254 \str_new:N \l__spintent_luaset_only_part_dec_str
255 \str_new:N \l__spintent_luaset_only_part_period_str
256 \str_new:N \l__spintent_luaset_format_part_int_str
257 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

`\l__spintent_luaset_millions_str` The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

`\l__spintent_luaset_dec_is_all_zeros_str`

`\l__spintent_luaset_has_exponent_str`

`\l__spintent_luaset_exponent_tl`

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

258 \str_new:N \l__spintent_luaset_millions_str
259 \str_new:N \l__spintent_luaset_decimal_period_str
260 \str_new:N \l__spintent_luaset_not_decimal_period_str
261 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
262 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
263 \str_new:N \l__spintent_luaset_has_exponent_str
264 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

`\l__spintent_luaset_has_units_str`

`\l__spintent_luaset_status_str`

`\l__spintent_luaset_arg_above_tl`

`\l__spintent_luaset_arg_below_tl`

`\l__spintent_luaset_denom_has_number_str`

Since the function `\l__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

265 \str_new:N \l__spintent_luaset_has_units_str
266 \str_new:N \l__spintent_luaset_status_str
267 \tl_new:N \l__spintent_luaset_arg_above_tl
268 \tl_new:N \l__spintent_luaset_arg_below_tl
269 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

270 \msg_new:nnnn { spintent } { need-math-mode }
271 {
272   Math~mode~required~for~'#1'~\msg_line_context:.
273 }
274 {
275   Use~inside~$.~$.~$~or~in~math~environment.
276 }
277 \msg_new:nnnn { spintent } { need-text-mode }
278 {
279   Text~mode~required~for~'#1'~\msg_line_context:.
280 }
281 {
282   Use~outside~of~$.~$.~$~or~math~environment.
283 }
```

11.14 The command \spnum

The user command `\spnum` is first of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for \spnum

The variable `\l__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “positivo” or “negativo” after reading the number; if set to `terse`, it will say “más” or “menos” before reading the number.

The variable `\l__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\l__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintent_spnum_intent_read_sign_str` will store the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` stores the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```

284 \bool_new:N \l__spintent_spnum_read_terse_bool
285 \str_new:N \l__spintent_spnum_read_period_sep_str
286 \tl_new:N \l__spintent_spnum_notation_sym_tl
287 \str_new:N \l__spintent_spnum_intent_read_sign_str
288 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

```

cifras-sep
read-sign
read-period-sep
notation-sym

Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.

289 \keys_define:nn { spintent/spnum }
290 {
291   cifras-sep .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
292   cifras-sep .initial:n = true,
293   cifras-sep .value_required:n = true,
294   read-sign .choices:nn =
295     { terse , clear }
296     {
297       \int_case:nn { \l_keys_choice_int }
298       {
299         {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
300         {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
301       }
302     },
303   read-sign .initial:n = terse,
304   read-sign .value_required:n = true,
305   read-sign / unknown .code:n =
```

```

306     \msg_error:nneee { spintent } { unknown-choice }
307     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
308 read-period-sep                .choices:nn =
309     { coma , punto }
310     {
311         \int_case:nn { \l_keys_choice_int }
312         {
313             {1} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { coma } }
314             {2} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { punto } }
315         }
316     },
317 read-period-sep                .initial:n = coma,
318 read-period-sep                .value_required:n = true,
319 read-period-sep / unknown .code:n =
320     \msg_error:nneee { spintent } { unknown-choice }
321     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
322 notation-sym                    .choices:nn =
323     { times , cdot }
324     {
325         \int_case:nn { \l_keys_choice_int }
326         {
327             {1} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \times } }
328             {2} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \cdot } }
329         }
330     },
331 notation-sym                    .initial:n = cdot,
332 notation-sym                    .value_required:n = true,
333 notation-sym / unknown .code:n =
334     \msg_error:nneee { spintent } { unknown-choice }
335     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
336 unknown                        .code:n =
337     {
338         \str_set:Nn \l__spintent_command_name_str { spnum }
339         \__spintent_unknown_keys_cmd:n {#1}
340     },
341 }

```

(End of definition for `cifras-sep` and others.)

11.14.2 Internal functions for `\spnum`

```

\__spintent_spnun_render_part:n
\__spintent_spnun_render_int:
\__spintent_spnun_render_dec:

```

The function `\__spintent_spnun_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_spnun_render_int:` and `\__spintent_spnun_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key `cifras-sep` is active or not.

```

342 \cs_new_protected:Nn \__spintent_spnun_render_part:n
343 {
344     \MathMLintent { \str_use:c { \l__spintent_luaset_only_part_#1_str } :number }
345     {
346         \bool_if:NTF \l__spintent_spnun_cifras_sep_bool
347         { \str_use:c { \l__spintent_luaset_format_part_#1_str } }
348         { \tl_use:c { \l__spintent_luaset_part_#1_tl } }
349     }
350 }
351 \cs_new_protected:Nn \__spintent_spnun_render_int:
352 {
353     \__spintent_spnun_render_part:n { int }
354 }
355 \cs_new_protected:Nn \__spintent_spnun_render_dec:
356 {
357     \__spintent_spnun_render_part:n { dec }
358 }

```

(End of definition for `\__spintent_spnun_render_part:n`, `\__spintent_spnun_render_int:`, and `\__spintent_spnun_render_dec:`.)

```

\__spintent_spnun_render_only_period:
\__spintent_spnun_print_period_bar:
\__spintent_spnun_render_period_sep:

```

The function `\__spintent_spnun_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_spnun_print_period_bar:`.

```

359 \cs_new_protected:Nn \__spintent_spnun_render_only_period:
360 {
361     \MathMLintent { \l__spintent_luaset_only_part_period_str :number }

```

```

362     {
363         \l__spintent_luaset_part_period_tl
364     }
365 }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “*infinite periodic decimal part*”.

```

366 \cs_new_protected:Nn \__spintent_snum_print_period_bar:
367 {
368     \MathMLintent { _periódico:postfix($dp) }
369     {
370         {
371             \overline
372             {
373                 \MathMLarg { dp } { \__spintent_snum_render_only_period: }
374             }
375         }
376     }
377 }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

378 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
379 {
380     \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
381     {
382         \MathMLintent { _coma:prefix($pp) }
383         {
384             \mathord{,}
385             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
386         }
387     }
388     {
389         \MathMLintent { _punto:prefix($pp) }
390         {
391             \mathord{.}
392             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
393         }
394     }
395 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

396 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
397 {
398     \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
399     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
400     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
401     \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
402     {
403         \mathord { \l__spintent_luaset_dec_sep_tl }
404         \MathMLarg { dp }
405         {
406             \__spintent_inner_sep:
407             \__spintent_snum_render_dec:
408         }
409     }
410     % period part
411     \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
412     {
413         \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
414         {
415             \__spintent_inner_sep:
416             \__spintent_snum_print_period_bar:

```

```

417     }
418     {
419         \MathMLintent { _con:prefix($pnum) }
420         {
421             \__spintent_inner_sep:
422             \MathMLarg { pnum }
423             {
424                 \__spintent_snum_print_period_bar:
425             }
426         }
427     }
428 }
429 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

430 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
431 {
432     \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
433     {
434         \__spintent_snum_render_decimal_sep:
435     }
436 }
437 \cs_new_protected:Nn \__spintent_snum_print_part_period:
438 {
439     \tl_if_empty:NF \l__spintent_luaset_part_period_tl
440     {
441         \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
442         { \__spintent_snum_render_period_sep: }
443         {
444             \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
445             {
446                 \__spintent_snum_render_period_sep:
447             }
448         }
449     }
450 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`.)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

451 \cs_new_protected:Nn \__spintent_snum_print_integer:
452 {
453     \str_if_eq:VnTF \l__spintent_luaset_not_decimal_not_period_str { true }
454     { \__spintent_snum_render_int: }
455     {
456         \MathMLintent { \l__spintent_luaset_only_part_int_str :number:prefix($next) }
457         {
458             \bool_if:NTF \l__spintent_snum_cifras_sep_bool
459             { \l__spintent_luaset_format_part_int_str }
460             { \l__spintent_luaset_part_int_tl }
461             \MathMLarg { next }
462         }

```

```

463         \tl_if_empty:NTF \l__spintent_luaset_part_dec_tl
464         { \__spintent_spnum_print_part_period: }
465         { \__spintent_spnum_print_part_dec: }
466     }
467 }
468 }
469 }
470 \cs_new_protected:Nn \__spintent_spnum_print_num_start:
471 {
472     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
473     {
474         \__spintent_invisible_sep: % need
475         \__spintent_spnum_print_integer:
476     }
477     {
478         \bool_if:NTF \l__spintent_spnum_read_terse_bool
479         {
480             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
481             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _más:prefix($n)} }
482             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _menos:prefix($n)} }
483         }
484         {
485             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
486             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _positivo:postfix($n) } }
487             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _negativo:postfix($n) } }
488         }
489         \__spintent_invisible_sep: % need
490         \MathMLintent { \l__spintent_spnum_intent_read_sign_str }
491         {
492             \l__spintent_luaset_sign_tl
493             \MathMLarg { n } { \__spintent_spnum_print_integer: }
494         }
495     }
496 }
497 \cs_new_protected:Nn \__spintent_spnum_print_number_core:
498 {
499     \__spintent_spnum_print_num_start:
500     \str_if_eq:VnT \l__spintent_luaset_has_exponent_str { true }
501     {
502         \MathMLintent { _por } { \l__spintent_spnum_notation_sym_tl }
503         10^{\l__spintent_luaset_exponent_tl }
504     }
505 }

```

(End of definition for `\__spintent_spnum_print_integer:`, `\__spintent_spnum_print_num_start:`, and `\__spintent_spnum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

506 \msg_new:nnnn { spintent } { spnum-has-units }
507 {
508     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(' #1').
509 }
510 {
511     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
512 }
513 \msg_new:nnnn { spintent } { spnum-no-digits }
514 {
515     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
516 }
517 {
518     You~provided~' #1',~which~contains~no~digits.
519 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[key = val]$ argument in ‘#1’ and a mandatory $\langle rational number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is

not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

520 \NewDocumentCommand \spnum { O{} m }
521 {
522   \mode_if_math:TF
523   {
524     \group_begin:
525       \keys_set:nn { spintent/spnum } { #1 }
526       \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
527       \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
528       { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
529       {
530         \tl_if_empty:NTF \__spintent_luaset_part_int_tl
531         { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
532         { \__spintent_spnum_print_number_core: }
533       }
534     \group_end:
535   }
536   { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
537 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the `{\argument}` to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the `{\argument}`.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

538 \str_new:N \__spintent_spunit_luaset_canonical_str
539 \str_new:N \__spintent_spunit_luaset_lookup_status_str
540 \str_new:N \__spintent_spunit_luaset_read_str
541 \str_new:N \__spintent_spunit_luaset_is_compact_str
542 \str_new:N \__spintent_spunit_luaset_compact_exp_str
543 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
544 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

545 \tl_new:N \__spintent_spunit_exponent_tl
546 \tl_new:N \__spintent_spunit_unit_name_tl
547 \str_new:N \__spintent_spunit_read_slash_str
548 \seq_new:N \__spintent_spunit_exp_seq
549 \seq_new:N \__spintent_spunit_mult_seq
550 \bool_new:N \__spintent_spunit_is_mult_bool
551 \bool_new:N \__spintent_spunit_read_cdod_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the `{\argument}`, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

552 \msg_new:nnnn { spintent } { spunit-invalid-unit }
553 {
554   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.

```

```

555 }
556 {
557   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
558 }
559 \msg_new:nnnn { spintent } { spunit-no-units }
560 {
561   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
562 }
563 {
564   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
565 }
566 \msg_new:nnnn { spintent } { inline-numeric-denominator }
567 {
568   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
569 }
570 {
571   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
572   does~not~allow~numbers~after~the~slash~(/).
573 }

```

(End of definition for spunit-invalid-unit, spunit-no-units, and inline-numeric-denominator.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

574 \keys_define:nn { spintent/spunit }
575 {
576   print-cdot      .bool_set:N = \l_spintent_spunit_print_cdot_bool,
577   print-cdot      .initial:n  = false,
578   print-cdot      .value_required:n = true,
579   read-cdot       .bool_set:N = \l_spintent_spunit_read_cdot_bool,
580   read-cdot       .initial:n  = false,
581   read-cdot       .value_required:n = true,
582   read-slash      .choices:nn =
583     { por , partido-por }
584     {
585       \int_case:nn { \l_keys_choice_int }
586       {
587         {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { _por } }
588         {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { _partido-por } }
589       }
590     },
591   read-slash      .initial:n  = por,
592   read-slash      .value_required:n = true,
593   read-slash / unknown .code:n =
594     \msg_error:nneee { spintent } { unknown-choice }
595     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
596   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
597   cifras-sep      .value_required:n = true,
598   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
599   read-period-sep .value_required:n = true,
600   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
601   notation-sym    .value_required:n = true,
602   unknown         .code:n =
603     {
604       \str_set:Nn \l__spintent_command_name_str { spunit }
605       \__spintent_unknown_keys_cmd:n {#1}
606     },
607 }

```

(End of definition for read-cdot and others.)

11.15.4 Internal functions for \spunit

The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ℓ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

608 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
609 {
610   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
611   {
612     { ℓ } { \ell }

```

```

613     }
614     { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
615 }

```

(End of definition for `\l__spintent_spunit_format_canonical:`)

`\l__spintent_spunit_process_base_exp:n`

The function `\l__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘#1’. It first looks up the unit in the dictionary; if not found, flags it as invalid.

Otherwise, unless this is the first unit in a product (`\l__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

616 \cs_new_protected:Nn \l__spintent_spunit_process_base_exp:n
617 {
618   \tl_clear:N \l__spintent_spunit_exponent_tl
619   \l__spintent_luafun_spunit_lookup_alias:n { #1 }
620   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
621   {
622     \str_set:Nn \l__spintent_luaset_status_str { invalid }
623   }
624   {
625     \bool_if:NF \l__spintent_spunit_is_mult_bool
626     {
627       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
628       {
629         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
630         { \MathMLint { _por } { \cdot } }
631         { \MathMLint { :silent } { \cdot } }
632       }
633       {
634         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
635         { \MathMLint { _por } { \, } }
636         { \, }
637       }
638     }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spintent_spunit_luafun_spunit_lookup_status_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

639   \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
640   {
641     {
642       \MathMLint { \l__spintent_spunit_luaset_read_str }
643       {
644         \l__spintent_spunit_format_canonical:
645       }
646     }
647     ^ { \l__spintent_spunit_luaset_compact_exp_str }
648   }
649   {
650     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
651     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
652     \seq_if_empty:NF \l__spintent_spunit_exp_seq
653     {
654       \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
655     }
656     \MathMLint { \l__spintent_spunit_luaset_read_str }
657     {
658       \l__spintent_spunit_format_canonical:
659     }
660     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
661     {
662       ^ { \l__spintent_spunit_exponent_tl }
663     }
664   }
665   \bool_set_false:N \l__spintent_spunit_is_mult_bool
666 }
667 }

```

(End of definition for `\l__spintent_spunit_process_base_exp:n`)

`\l__spintent_spunit_process_mult:n`

`\l__spintent_spunit_process_mult:V`

The function `\l__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{ \langle argument \rangle \}$ at each “star symbol” and applies `\l__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

668 \cs_new_protected:Nn \l__spintent_spunit_process_mult:n

```

```

669   {
670     \bool_set_true:N \__spintent_spunit_is_mult_bool
671     \seq_set_split:Nnn \__spintent_spunit_mult_seq { * } { #1 }
672     \seq_map_function:NN \__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
673   }
674   \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

675   \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
676   {
677     \tl_if_empty:NF \__spintent_luaset_arg_above_tl
678     {
679       \__spintent_spunit_process_mult:V \__spintent_luaset_arg_above_tl
680       \tl_if_empty:NF \__spintent_luaset_arg_below_tl
681       {
682         \MathMLintent { \__spintent_spunit_read_slash_str } { / }
683         \__spintent_spunit_process_mult:V \__spintent_luaset_arg_below_tl
684       }
685     }
686   }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{\langle argument \rangle}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

687   \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
688   {
689     \str_if_eq:VnTF \__spintent_luaset_has_units_str { false }
690     { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
691     {
692       \str_if_eq:VnTF \__spintent_luaset_status_str { valid }
693       {
694         \str_if_eq:VnTF
695         \__spintent_luaset_denom_has_number_str { true }
696         {
697           \msg_error:nnn { spintent }
698           { inline-numeric-denominator } { #1 }
699         }
700       }
701       \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaset_arg_above_tl
702       \str_if_eq:VnTF \__spintent_spunit_luaset_is_sexagesimal_str { true }
703       {
704         \tl_if_empty:NF \__spintent_luaset_part_int_tl
705         {
706           \__spintent_spnum_print_number_core:
707         }
708         \MathMLintent { \__spintent_spunit_luaset_read_str }
709         {
710           \mathord
711           {
712             \str_case:Vn \__spintent_spunit_luaset_canonical_str
713             {
714               { " } { \prime \! \prime }
715               { ' } { \prime }
716               { ° } { ° }
717             }
718           }
719         }
720       }
721     }
722     \tl_if_empty:NF \__spintent_luaset_part_int_tl
723     {

```

```

724             \__spintent_spnum_print_number_core: \,
725         }
726         \__spintent_print_spunit_hierarchy:
727     }
728 }
729 }
730 { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
731 }
732 }

```

(End of definition for `\__spintent_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [`⟨key = val⟩`] argument in ‘#1’ and a mandatory `{⟨unit expression⟩}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{⟨argument⟩}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

733 \NewDocumentCommand \spunit { O{} m }
734 {
735     \mode_if_math:TF
736     {
737         \group_begin:
738         \keys_set:nn { spintent/spunit } { #1 }
739         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
740         \__spintent_spunit_safe_exec:n { #2 }
741         \group_end:
742     }
743     { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
744 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to T_EX using the `\l__spintent_spunit_luaseta_status_s` variable. If a collision is detected or the `{⟨argument⟩}` is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

745 \msg_new:nnnn { spintent } { spnewunit-duplicate }
746 {
747     The~unit~symbol~or~alias~'#1'~is~already~defined!
748 }
749 {
750     You~cannot~redefine~with~\token_to_str:N \spnewunit.
751 }
752 \msg_new:nnnn { spintent } { spunitalias-duplicate }
753 {
754     The~alias~'#1'~is~already~defined~or~reserved!
755 }
756 {
757     You~cannot~redefine~an~existing~unit~or~another~alias.
758 }
759 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
760 {
761     The~expression~'#2'~for~alias~'#1'~is~invalid.
762 }
763 {
764     It~must~contain~only~defined~units,~exponents~and~one~'/'.
765 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit` The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

766 \NewDocumentCommand \spnewunit { m m }
767 {
768   \mode_if_math:TF
769   {
770     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
771   }
772   {
773     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
774     \str_if_eq:VnT \__spintent_spunit_luaset_status_str { duplicate }
775     {
776       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
777     }
778   }
779 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

780 \NewDocumentCommand \spunitalias { m m }
781 {
782   \mode_if_math:TF
783   {
784     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
785   }
786   {
787     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
788     \str_if_eq:VnT \__spintent_spunit_luaset_status_str { duplicate }
789     {
790       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
791     }
792     \str_if_eq:VnT \__spintent_spunit_luaset_status_str { invalid-expr }
793     {
794       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
795     }
796   }
797 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

\__spintent_spmoney_luaset_symbol_str
\__spintent_spmoney_luaset_position_str
\__spintent_spmoney_luaset_sing_str
\__spintent_spmoney_luaset_plur_str
\__spintent_spmoney_luaset_conde_str
\__spintent_spmoney_luaset_subsing_str
\__spintent_spmoney_luaset_subplur_str
\__spintent_spmoney_luaset_print_iso_str
\__spintent_spmoney_luaset_currency_arg_str
\__spintent_spmoney_luaset_status_str

```

```

798 \str_new:N \__spintent_spmoney_luaset_symbol_str
799 \str_new:N \__spintent_spmoney_luaset_position_str
800 \str_new:N \__spintent_spmoney_luaset_sing_str
801 \str_new:N \__spintent_spmoney_luaset_plur_str
802 \str_new:N \__spintent_spmoney_luaset_conde_str
803 \str_new:N \__spintent_spmoney_luaset_subsing_str
804 \str_new:N \__spintent_spmoney_luaset_subplur_str
805 \str_new:N \__spintent_spmoney_luaset_print_iso_str
806 \str_new:N \__spintent_spmoney_luaset_currency_arg_str
807 \str_new:N \__spintent_spmoney_luaset_status_str

```

(End of definition for `\__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

808 \bool_new:N \__spintent_spmoney_sub_units_bool
809 \str_new:N \__spintent_spmoney_intent_money_str
810 \str_new:N \__spintent_spmoney_intent_sign_str
811 \str_new:N \__spintent_spmoney_intent_sub_unit_str
812 \str_new:N \__spintent_spmoney_res_main_voice_str
813 \int_new:N \__spintent_spmoney_subnum_int
814 \str_new:N \__spintent_spmoney_extracted_curr_str
815 \tl_new:N \__spintent_spmoney_extracted_num_tl

```

(End of definition for `\__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database or is not written validly: an alias or symbol with any uppercase letter, or an ISO code that is neither all lowercase nor all uppercase.

```

816 \msg_new:nnnn { spintent } { currency-no-subunits }
817 {
818   The~currency~'#1'~does~not~support~subunits.~
819   The~option~'sub-units=true'~is~ignored.
820 }
821 {
822   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
823   do~not~have~fractional~subunits~registered~in~Lua~module.~
824 }
825 \msg_new:nnnn { spintent } { invalid-currency }
826 {
827   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
828 }
829 {
830   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
831   the~Lua~module.
832 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.17.2 Auxiliary functions for `\spmoney`

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a `notfound` status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

833 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
834 {
835   \__spintent_spmoney_normalize_key:n {#1}
836   \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
837   {
838     \msg_error:nnn { spintent } { invalid-currency } {#1}
839   }
840   { \str_set:Nn \__spintent_spmoney_luaset_currency_arg_str {#1} }
841 }
842 \cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
843 {
844   \__spintent_luafun_spmoney_normalize_key:n { \exp_not:n { #1 } }
845 }
846 \cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintent_spmoney_set_currency:n` and `\__spintent_spmoney_normalize_key:n`.)

11.17.3 Definition of keys for \spmoney

Now we add the keys `currency`, `sub-units`, `money-math` and `cifras-sep`.

```

currency
sub-units
money-math
cifras-sep
847 \keys_define:nn { spintent / spmoney }
848 {
849   currency      .code:n      = { \__spintent_spmoney_set_currency:n {#1} },
850   currency      .initial:n    = { pesos },
851   currency      .value_required:n = true,
852   sub-units     .bool_set:N   = \__spintent_spmoney_sub_units_bool,
853   sub-units     .value_required:n = true,
854   money-math    .bool_set:N   = \__spintent_spmoney_money_math_bool,
855   money-math    .initial:n    = { true },
856   money-math    .value_required:n = true,
857   cifras-sep    .meta:nn     = { spintent/spnum } { cifras-sep = {#1} },
858   cifras-sep    .value_required:n = true,
859   unknown       .code:n      =
860   {
861     \str_set:Nn \__spintent_command_name_str { spmoney }
862     \__spintent_unknown_keys_cmd:n {#1}
863   },
864 }
```

(End of definition for `currency` and others.)

11.17.4 Internal functions for \spmoney

`\__spintent_spmoney_validate_money:`

The function `\__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\l__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `\__spintent_luafun_spmoney_lookup_data:V`.

```

865 \cs_new_protected:Nn \__spintent_spmoney_validate_money:
866 {
867   \str_if_eq:VnT \l__spintent_luaset_has_units_str { true }
868   {
869     \__spintent_spmoney_normalize_key:V \l__spintent_luaset_arg_above_tl
870     \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
871     { \msg_error:nne { spintent } { invalid-currency } { \l__spintent_luaset_arg_above_tl }
872       {
873         \str_set:NV
874           \l__spintent_spmoney_luaset_currency_arg_str
875           \l__spintent_luaset_arg_above_tl
876       }
877     }
878     \str_set:Ne \l__spintent_spmoney_luaset_status_str { found }
879     \__spintent_luafun_spmoney_lookup_data:V \l__spintent_spmoney_luaset_currency_arg_str
880   }

```

(End of definition for `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:`

The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintent_spmoney_luaset_currency_arg_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”, directly in math mode when `\l__spintent_spmoney_money_math_bool` is `true`, or wrapped inside `\text` when it is `false`. For dollar signs ‘\$’, the symbol is typeset through the control sequence `\$` in both cases.

```

881 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
882 {
883   \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
884   { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
885   {
886     \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
887     {
888       \bool_if:Ntf \l__spintent_spmoney_money_math_bool
889       { \$ } { \text { \$ } }
890     }
891     {
892       \bool_if:Ntf \l__spintent_spmoney_money_math_bool
893       { \l__spintent_spmoney_luaset_symbol_str }
894       { \text { \l__spintent_spmoney_luaset_symbol_str } }
895     }
896   }
897 }
```

(End of definition for `\__spintent_spmoney_print_symbol:`.)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

898 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
899 {
900   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
901   {
902     \__spintent_spmoney_print_symbol:
903     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
904   }
905   {
906     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
907     \__spintent_spmoney_print_symbol:
908   }
909 }

```

(End of definition for `\__spintent_spmoney_render_layout:`.)

`\__spintent_spmoney_render_number_node:`
`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

910 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
911 {
912   \__spintent_spmoney_render_int:
913   \__spintent_spmoney_print_part_dec:
914 }
915 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
916 {
917   \__spintent_spmoney_render_int:
918 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘*más*’ or ‘*menos*’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

919 \cs_new_protected:Nn \__spintent_spmoney_print_int:
920 {
921   \tl_if_empty:NTF \l__spintent_luaset_sign_tl
922   {
923     \bool_if:NTF \l__spintent_spmoney_sub_units_bool
924     { \__spintent_spmoney_set_and_print_sub_units: }
925     { \__spintent_spmoney_intent_int: }
926   }
927   {
928     \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
929     {
930       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
931     }
932     {
933       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
934     }
935     \__spintent_invisible_sep:
936     \MathMLintent { \l__spintent_spmoney_intent_sign_str }
937     {
938       \l__spintent_luaset_sign_tl
939       \MathMLarg { n }
940       {
941         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
942         { \__spintent_spmoney_set_and_print_sub_units: }
943         { \__spintent_spmoney_intent_int: }
944       }
945     }
946   }
947 }

```

(End of definition for `\__spintent_spmoney_print_int:`.)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix MathML intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

948 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
949 {
950   \tl_if_empty:NTF \__spintent_luaset_dec_sep_tl
951   {
952     \str_if_eq:VnTF \__spintent_luaset_only_part_int_str { 1 }
953     {
954       \str_set:NV \__spintent_spmoney_intent_money_str \__spintent_spmoney_luaset_sing_str
955     }
956     {
957       \str_set:NV \__spintent_spmoney_intent_money_str \__spintent_spmoney_luaset_plur_str
958     }
959     \str_if_eq:VnT \__spintent_luaset_millions_str { true }
960     {
961       \str_set:NV \__spintent_spmoney_intent_money_str \__spintent_spmoney_luaset_conde_s
962     }
963   }
964   {
965     \str_set:NV \__spintent_spmoney_intent_money_str \__spintent_spmoney_luaset_plur_str
966     \str_if_eq:VnT \__spintent_luaset_millions_str { true }
967     {
968       \str_if_eq:VnTF \__spintent_luaset_dec_is_all_zeros_str { true }
969       {
970         \str_set:NV \__spintent_spmoney_intent_money_str \__spintent_spmoney_luaset_con
971       }
972       {
973         \str_set:NV \__spintent_spmoney_intent_money_str \__spintent_spmoney_luaset_plu
974       }
975     }
976   }
977   \__spintent_invisible_sep:
978   \MathMLintent { _ \__spintent_spmoney_intent_money_str :postfix($n) }
979   {
980     \__spintent_spmoney_render_layout:
981   }
982 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “subunits”. If the database returns empty subunit data, it triggers a warning and automatically disables the “subunit” rendering flag. The flag is also disabled, without a warning, when the amount has no decimal part, since there are no “subunits” to voice.

`spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “subunits” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘.5’ to ‘.50’), evaluates the correct singular or plural inflections for both the main “currency” and the “subunits”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

983 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
984 {
985   \str_if_empty:NT \__spintent_spmoney_luaset_subsing_str
986   {
987     \msg_warning:nne { spintent } { currency-no-subunits }
988     { \__spintent_spmoney_luaset_currency_arg_str }
989     \bool_set_false:N \__spintent_spmoney_sub_units_bool
990   }
991   \tl_if_empty:NT \__spintent_luaset_dec_sep_tl
992   { \bool_set_false:N \__spintent_spmoney_sub_units_bool }
993 }
994 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
995 {
996   \int_compare:nTF { \str_count:N \__spintent_luaset_only_part_dec_str == 1 }
997   {
998     \int_set:Nn \__spintent_spmoney_subnum_int
999     { \__spintent_luaset_only_part_dec_str * 10 }
1000     \str_set:Ne \__spintent_luaset_only_part_dec_str
1001     { \__spintent_luaset_only_part_dec_str 0 }
1002   }

```

```

1003     {
1004         \int_set:Nn \l__spintent_spmoney_subnum_int
1005         { \l__spintent_luaset_only_part_dec_str }
1006     }
1007     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
1008     {
1009         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1010         { _ \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1011     }
1012     {
1013         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1014         { _ \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1015     }
1016     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1017     {
1018         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1019         \l__spintent_spmoney_luaset_sing_str
1020     }
1021     {
1022         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1023         \l__spintent_spmoney_luaset_plur_str
1024     }
1025     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1026     {
1027         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1028         \l__spintent_spmoney_luaset_conde_str
1029     }
1030     \__spintent_spmoney_build_tree:
1031 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`.)

`\__spintent_spmoney_build_tree:`

`\__spintent_spmoney_print_sep_con:`

`\__spintent_spmoney_print_dec_sub_unit:`

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “currencies”. It arranges the integer component, the decimal separator, and the “subunit” component based on the layout position (pre or post) of the “currency symbol”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘_con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “subunit” intent to the decimal digits, ensuring accurate accessibility output.

```

1032 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1033 {
1034     \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1035     {
1036         \MathMLintent { _ \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1037         {
1038             \__spintent_spmoney_print_symbol:
1039             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1040         }
1041         \__spintent_spmoney_print_sep_con:
1042         \__spintent_spmoney_print_dec_sub_unit:
1043     }
1044     {
1045         \MathMLintent { _ \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1046         {
1047             \__spintent_inner_sep:
1048             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1049         }
1050         \__spintent_spmoney_print_sep_con:
1051         \__spintent_spmoney_print_dec_sub_unit:
1052         \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1053     }
1054 }
1055 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1056 {
1057     \MathMLintent { _con }
1058     {
1059         \mathord { \l__spintent_luaset_dec_sep_tl }
1060     }
1061 }
1062 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1063 {
1064     \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }

```

```

1065     {
1066         \__spintent_inner_sep:
1067         \MathMLLarg { sub }
1068     {
1069         \__spintent_spmoney_render_dec:
1070     }
1071 }
1072 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{\argument}` passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1073 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1074 {
1075     \__spintent_luafun_spmoney_prepare_input:n { \exp_not:n { #1 } }
1076     \__spintent_luafun_clean_split_and_set:n { \exp_not:V \__spintent_spmoney_extracted_num_tl }
1077     \str_if_empty:NF \__spintent_spmoney_extracted_curr_str
1078     {
1079         \str_set:Ne \__spintent_luaset_has_units_str { true }
1080         \str_set:NV \__spintent_luaset_arg_above_tl \__spintent_spmoney_extracted_curr_str
1081     }
1082     \__spintent_spmoney_validate_money:
1083     \bool_if:NT \__spintent_spmoney_sub_units_bool
1084     { \__spintent_spmoney_check_sub_units: }
1085     \__spintent_spmoney_print_int:
1086 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local T_EX group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1087 \NewDocumentCommand \spmoney { 0{} m }
1088 {
1089     \mode_if_math:TF
1090     {
1091         \group_begin:
1092         \keys_set:nn { spintent / spmoney } { #1 }
1093         \__spintent_spmoney_parse_and_route:n { #2 }
1094         \group_end:
1095     }
1096     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1097 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the E (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

`\__spintent_spsshort_luaset_status_str`
`\__spintent_spsshort_luaset_layout_str`
`\__spintent_spsshort_luaset_expand_str`
`\__spintent_spsshort_luaset_output_str`
`\__spintent_spsshort_luaset_base_str`
`\__spintent_spsshort_luaset_suffix_str`

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the E key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1098 \str_new:N \__spintent_spsshort_luaset_status_str
1099 \str_new:N \__spintent_spsshort_luaset_layout_str
1100 \str_new:N \__spintent_spsshort_luaset_expand_str
1101 \str_new:N \__spintent_spsshort_luaset_output_str
1102 \str_new:N \__spintent_spsshort_luaset_base_str
1103 \str_new:N \__spintent_spsshort_luaset_suffix_str

```

(End of definition for `\l__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```
1104 \msg_new:nnnn { spintent } { unknown-abbreviation }
1105 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1106 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }
```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`

`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```
1107 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1108 {
1109   \tag_mc_end_push:
1110   \tag_struct_begin:n { tag=Span, E={ #1 } }
1111   \tag_mc_begin:n { tag=Span }
1112   \tag_suspend:n { \spshort }
1113   #2
1114   \tag_resume:n { \spshort }
1115   \tag_mc_end:
1116   \tag_struct_end:
1117   \tag_mc_begin_pop:n {}
1118 }
1119 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }
```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.18.3 Definition of keys for `\spshort`

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```
1120 \keys_define:nn { spintent / spshort }
1121 {
1122   small-caps .bool_set:N = \l__spintent_spshort_smallcaps_bool,
1123   small-caps .initial:n = false,
1124   small-caps .value_required:n = true,
1125   read-arg .str_set:N = \l__spintent_spshort_read_arg_str,
1126   read-arg .initial:n = {},
1127   read-arg .value_required:n = true,
1128   unknown .code:n =
1129   {
1130     \str_set:Nn \l__spintent_command_name_str { spshort }
1131     \__spintent_unknown_keys_cmd:n {#1}
1132   },
1133 }
```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for `\spshort`

`\__spintent_spshort_print:nn`

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\l__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```
1134 \cs_new_protected:Nn \__spintent_spshort_print:nn
1135 {
1136   \str_clear:N \l__spintent_spshort_read_arg_str
1137   \keys_set:nn { spintent / spshort } {#1}
1138   \str_if_empty:NTF \l__spintent_spshort_read_arg_str
1139   {
1140     \__spintent_spshort_process_and_render:n {#2}
1141   }
1142   {
1143     \__spintent_spshort_render_bypass:n {#2}
1144   }
1145 }
```

(End of definition for `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```
1146 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1147 {
1148   \__spintent_luafun_spshort_process:n {#1}
1149   \str_case:VnF \__spintent_spshort_luaset_status_str
1150   {
1151     { success } { \__spintent_spshort_render_success: }
1152     { fallback } { \__spintent_spshort_render_fallback: }
1153     { error } { \__spintent_spshort_render_error:n {#1} }
1154   }
1155   { \__spintent_spshort_render_error:n {#1} }
1156 }
```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

`\__spintent_spshort_render_fallback:`

`\__spintent_spshort_render_bypass:n`

`\__spintent_spshort_render_error:n`

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “speech” to the `{⟨argument⟩}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `{⟨argument⟩}` to prevent fatal compilation.

```
1157 \cs_new_protected:Nn \__spintent_spshort_render_success:
1158 {
1159   \__spintent_spshort_tag_span:en
1160   { \__spintent_spshort_luaset_expand_str }
1161   { \__spintent_spshort_execute_layout: }
1162 }
1163 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1164 {
1165   \__spintent_spshort_tag_span:en
1166   { \__spintent_spshort_luaset_expand_str }
1167   {
1168     \bool_if:NTF \__spintent_spshort_smallcaps_bool
1169     { \textsc{ \__spintent_spshort_luaset_output_str } }
1170     { \__spintent_spshort_luaset_output_str }
1171   }
1172 }
1173 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1174 {
1175   \__spintent_spshort_tag_span:en { \__spintent_spshort_read_arg_str } { #1 }
1176 }
1177 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1178 {
1179   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1180 }
```

(End of definition for `\__spintent_spshort_render_success:` and others.)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “stored” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional `SMALL CAPS` based on the active boolean variables.

```
1181 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1182 {
1183   \str_case:VnF \__spintent_spshort_luaset_layout_str
1184   {
1185     { linear_regular }
1186     {
1187       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1188       { \textsc{ \__spintent_spshort_luaset_output_str } }
1189       { \__spintent_spshort_luaset_output_str }
1190     }
1191   }
```

```

1191 { linear_caps }
1192 { \l__spintent_spshort_luaset_output_str }
1193 { linear_mixed }
1194 { \l__spintent_spshort_luaset_output_str }
1195 { superscript }
1196 {
1197   \l__spintent_spshort_luaset_base_str
1198   \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1199   {
1200     \textsuperscript{
1201       \textsc{
1202         \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1203       }
1204     }
1205   }
1206   {
1207     \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1208   }
1209 }
1210 { small_caps_pure }
1211 {
1212   \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1213   {
1214     \textsc{
1215       \str_lowercase:V \l__spintent_spshort_luaset_output_str
1216     }
1217   }
1218   { \l__spintent_spshort_luaset_output_str }
1219 }
1220 { acronym_long }
1221 { \l__spintent_spshort_luaset_output_str }
1222 }
1223 { \l__spintent_spshort_luaset_output_str }
1224 }

```

(End of definition for `\l__spintent_spshort_execute_layout:`.)

`\spshort` The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\l__spintent_spshort_print:nn` for processing and printing.

```

1225 \NewDocumentCommand \spshort { 0{} m }
1226 {
1227   \mode_if_math:TF
1228   {
1229     \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1230   }
1231   {
1232     \mode_leave_vertical:
1233     \group_begin:
1234     \l__spintent_spshort_print:nn {#1} {#2}
1235     \group_end:
1236   }
1237 }

```

(End of definition for `\spshort`. This function is documented on page 8.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as `{\argument}`, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1238 \str_new:N \l__spintent_spsiglo_luaset_error_str

```

```

1239 \str_new:N \__spintent_spsiglo_luaset_arabic_str
1240 \str_new:N \__spintent_spsiglo_luaset_roman_min_str
1241 \str_new:N \__spintent_spsiglo_print_era_str

```

(End of definition for `\__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of **1** to **4000**, preventing incorrect speech synthesis.

```

1242 \msg_new:nnnn { spintent } { invalid-century-format }
1243 {
1244   Invalid~century~format~in~'#1'.
1245 }
1246 {
1247   The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1248   Ensure~the~value~is~between~1~and~4000.
1249 }

```

(End of definition for `invalid-century-format`.)

11.19.2 Definition of keys for `\spsiglo`

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1250 \keys_define:nn { spintent / spsiglo }
1251 {
1252   print-era .choices:nn =
1253     { none , ac , dc }
1254   {
1255     \int_case:nn { \l_keys_choice_int }
1256     {
1257       {1} { \str_set:Nn \__spintent_spsiglo_print_era_str { none } }
1258       {2} { \str_set:Nn \__spintent_spsiglo_print_era_str { ac } }
1259       {3} { \str_set:Nn \__spintent_spsiglo_print_era_str { dc } }
1260     }
1261   },
1262   print-era .initial:n = none,
1263   print-era .value_required:n = true,
1264   print-era / unknown .code:n =
1265     \msg_error:nneee { spintent } { unknown-choice }
1266     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1267   unknown .code:n =
1268     {
1269       \str_set:Nn \__spintent_command_name_str { spsiglo }
1270       \__spintent_unknown_keys_cmd:n {#1}
1271     },
1272 }

```

(End of definition for `print-era`.)

11.19.3 Internal functions for `\spsiglo`

__spintent_spsiglo_render_simple:nn

__spintent_spsiglo_render_simple:VV

__spintent_spsiglo_render_with_era:nnn

__spintent_spsiglo_render_with_era:VVV

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1273 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1274 {
1275   \MathMLintent { #1:number } { \text { \textsc {#2} } }
1276 }
1277 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1278 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1279 {
1280   \str_if_eq:nnTF { #1 } { ac }
1281   {
1282     \MathMLintent { _antes-de-cristo:postfix($z) }
1283     {
1284       \__spintent_inner_sep:
1285       \MathMLarg { z }
1286     }
1287     \MathMLintent { #2:number }

```

```

1288         {
1289             \text { \textsc {#3}~a.~C. }
1290         }
1291     }
1292 }
1293 }
1294 {
1295     \MathMLintent { _después-de-cristo:postfix($z) }
1296     {
1297         \__spintenc_inner_sep:
1298         \MathMLarg { z }
1299         {
1300             \MathMLintent { #2:number }
1301             {
1302                 \text { \textsc {#3}~d.~C. }
1303             }
1304         }
1305     }
1306 }
1307 }
1308 \cs_generate_variant:Nn \__spintenc_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for __spintenc_spsiglo_render_simple:nn and __spintenc_spsiglo_render_with_era:nnn.)

```

\__spintenc_spsiglo_tag_span:nn
\__spintenc_spsiglo_tag_span:en

```

The function `\__spintenc_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1309 \cs_new_protected:Nn \__spintenc_spsiglo_tag_span:nn
1310 {
1311     \tag_mc_end_push:
1312     \tag_struct_begin:n { tag=Span, alt={ #1 } }
1313     \tag_mc_begin:n { tag=Span }
1314     \tag_suspend:n { \spsiglo }
1315     #2
1316     \tag_resume:n { \spsiglo }
1317     \tag_mc_end:
1318     \tag_struct_end:
1319     \tag_mc_begin_pop:n {}
1320 }
1321 \cs_generate_variant:Nn \__spintenc_spsiglo_tag_span:nn { en }

```

(End of definition for __spintenc_spsiglo_tag_span:nn.)

```

\__spintenc_spsiglo_print_math:n
\__spintenc_spsiglo_print_text:n

```

The functions `\__spintenc_spsiglo_print_math:n` and `\__spintenc_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintenc_spsiglo_luaset_error_str` returned by the Lua module `spintenc.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintenc_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintenc_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1322 \cs_new_protected:Nn \__spintenc_spsiglo_print_math:n
1323 {
1324     \str_if_eq:VnTF \l__spintenc_spsiglo_luaset_error_str { true }
1325     {
1326         \msg_error:nnnn { spintenc } { invalid-century-format }
1327         { \spsiglo } { #1 }
1328     }
1329     {
1330         \str_if_eq:VnTF \l__spintenc_spsiglo_print_era_str { none }
1331         {
1332             \__spintenc_spsiglo_render_simple:VV
1333             \l__spintenc_spsiglo_luaset_arabic_str
1334             \l__spintenc_spsiglo_luaset_roman_min_str
1335         }
1336         {
1337             \__spintenc_spsiglo_render_with_era:VVV
1338             \l__spintenc_spsiglo_print_era_str
1339             \l__spintenc_spsiglo_luaset_arabic_str
1340             \l__spintenc_spsiglo_luaset_roman_min_str

```

```

1341     }
1342   }
1343 }
1344 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1345 {
1346   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1347   {
1348     \msg_error:nnnn { spintent } { invalid-century-format }
1349     { \spsiglo } { #1 }
1350   }
1351   {
1352     \str_case:Vn \l__spintent_spsiglo_print_era_str
1353     {
1354       { none }
1355       {
1356         \__spintent_spsiglo_tag_span:en
1357         { \l__spintent_spsiglo_luaset_arabic_str }
1358         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1359       }
1360       { ac }
1361       {
1362         \__spintent_spsiglo_tag_span:en
1363         { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1364         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1365       }
1366       { dc }
1367       {
1368         \__spintent_spsiglo_tag_span:en
1369         { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1370         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1371       }
1372     }
1373   }
1374 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional `\keys` passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1375 \NewDocumentCommand \spsiglo { 0{} m }
1376 {
1377   \group_begin:
1378     \keys_set:nn { spintent / spsiglo } { #1 }
1379     \__spintent_luafun_spsiglo_parse:n { #2 }
1380     \mode_if_math:TF
1381     { \__spintent_spsiglo_print_math:n { #2 } }
1382     {
1383       \mode_leave_vertical:
1384       \__spintent_spsiglo_print_text:n { #2 }
1385     }
1386   \group_end:
1387 }

```

(End of definition for `\spsiglo`. This function is documented on page 10.)

11.20 The command `\sptime`

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\l__spintent_sptime_luaset_seconds_str
\l__spintent_sptime_luaset_has_seconds_str
\l__spintent_sptime_luaset_error_str
\l__spintent_sptime_luaset_is_fraction_str
\l__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1388 \str_new:N \l__spintent_sptime_luaset_seconds_str
1389 \str_new:N \l__spintent_sptime_luaset_has_seconds_str
1390 \str_new:N \l__spintent_sptime_luaset_error_str
1391 \str_new:N \l__spintent_sptime_luaset_is_fraction_str
1392 \str_new:N \l__spintent_sptime_luaset_base_str

```

(End of definition for `\l__spintent_sptime_luaset_seconds_str` and others.)

11.20.1 Definition of error messages

invalid-time-format

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1393 \msg_new:nnnn { spintent } { invalid-time-format }
1394 {
1395   Invalid~time~format~in~#1.  \
1396   The~argument~'#2'~does~not~match~any~supported~time~format.
1397 }
1398 {
1399   Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss.  \
1400   Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1401 }

```

(End of definition for `invalid-time-format`.)

11.20.2 Internal functions for `\sptime`

`__spintent_sptime_build_seconds:nn`

The internal function `__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1402 \cs_new_protected:Nn __spintent_sptime_build_seconds:nn
1403 {
1404   \MathMLintent { #1:prefix($z) }
1405   {
1406     \text { : }
1407     \MathMLarg { z }
1408     {
1409       \MathMLintent { _segundos:postfix($m) }
1410       {
1411         __spintent_inner_sep:
1412         \MathMLarg { m }
1413         {
1414           \text { #2 }
1415         }
1416       }
1417     }
1418   }
1419 }

```

(End of definition for `__spintent_sptime_build_seconds:nn`.)

`__spintent_sptime_build_tree:`

The internal function `__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `__spintent_sptime_build_seconds:` with the appropriate prefix (‘con’ or ‘minutos-con’), or simply appends the ‘minutos’ intent tag if no seconds are present.

```

1420 \cs_new_protected:Nn __spintent_sptime_build_tree:
1421 {
1422   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1423   {
1424     \MathMLintent { :time }
1425     {
1426       \text { \l__spintent_sptime_luaset_base_str }
1427     }
1428     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1429     {
1430       __spintent_sptime_build_seconds:nn { con }
1431       { \l__spintent_sptime_luaset_seconds_str }
1432     }
1433     {
1434       __spintent_sptime_build_seconds:nn { _minutos-con }
1435       { \l__spintent_sptime_luaset_seconds_str }
1436     }
1437   }
1438   {
1439     \MathMLintent { :time }
1440     {
1441       \text { \l__spintent_sptime_luaset_base_str }
1442     }
1443   }

```

```

1443     \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1444     {
1445         \MathMLintent { _minutos }
1446         {
1447             \c__spintent_invisible_sep_int
1448         }
1449     }
1450 }
1451 }

```

(End of definition for `\__spintent_sptime_build_tree:`.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1452 \NewDocumentCommand \sptime { m }
1453 {
1454     \mode_if_math:TF
1455     {
1456         \group_begin:
1457         \__spintent_luafun_sptime_parse:n { #1 }
1458         \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1459         {
1460             \msg_error:nnnn { spintent } { invalid-time-format }
1461             { \sptime } { #1 }
1462         }
1463         {
1464             \__spintent_sptime_build_tree:
1465         }
1466     }
1467     \group_end:
1468     { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1469 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

```

\l__spintent_spdate_luaset_output_str
\l__spintent_spdate_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1470 \str_new:N \l__spintent_spdate_luaset_output_str
1471 \str_new:N \l__spintent_spdate_luaset_error_str

```

(End of definition for `\l__spintent_spdate_luaset_output_str` and `\l__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

`invalid-date-format`

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1472 \msg_new:nnnn { spintent } { invalid-date-format }
1473 {
1474     Invalid~date~or~unsupported~format~in~#1. \\\
1475     The~argument~'~#2'~does~not~match~a~supported~format.
1476 }
1477 {
1478     Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1479     Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1480 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for \spdate

__spintent_spdate_process:n

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```

1481 \cs_new_protected:Nn \__spintent_spdate_process:n
1482 {
1483   \__spintent_luafun_spdate_parse:n { #1 }
1484   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1485   {
1486     \msg_error:nnnn { spintent } { invalid-date-format }
1487     { \spdate } { #1 }
1488   }
1489   {
1490     \MathMLintent { :date }
1491     {
1492       \text { \__spintent_spdate_luaset_output_str }
1493     }
1494   }
1495 }

```

(End of definition for __spintent_spdate_process:n.)

\spdate

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```

1496 \NewDocumentCommand \spdate { m }
1497 {
1498   \mode_if_math:TF
1499   {
1500     \group_begin:
1501     \__spintent_spdate_process:n { #1 }
1502     \group_end:
1503   }
1504   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1505 }

```

(End of definition for \spdate. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command \spusep

For the use of parentheses, we will provide the command `\spusep` with only two *keys* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

spusep-empty-arg
spusep-invalid-arg

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank `{⟨argument⟩}`, the second this is when an input is invalid.

```

1506 \msg_new:nnnn { spintent } { spusep-empty-arg }
1507 {
1508   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1509 }
1510 {
1511   \token_to_str:N \spusep~requires~a~non-empty~argument.
1512 }
1513 \msg_new:nnn { spintent } { spusep-invalid-arg }
1514 {
1515   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1516 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

__spintent_spusep_arg_tl
__spintent_spusep_left_tl
__spintent_spusep_right_tl

The variable `\__spintent_spusep_arg_tl` stores the `{⟨argument⟩}`, the variable `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` stores the parentheses.

```

1517 \tl_new:N \__spintent_spusep_arg_tl
1518 \tl_new:N \__spintent_spusep_left_tl
1519 \tl_new:N \__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

```

size We define the (keys) size and silent.
silent
unknown
1520 \keys_define:nn { spintent / spusep }
1521 {
1522     size .choices:nn =
1523     { none , big , Big, bigg, Bigg }
1524     {
1525         \int_case:nn { \l_keys_choice_int }
1526         {
1527             { 1 }
1528             {
1529                 \tl_clear:N \l__spintent_spusep_left_tl
1530                 \tl_clear:N \l__spintent_spusep_right_tl
1531             }
1532             { 2 }
1533             {
1534                 \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1535                 \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1536             }
1537             { 3 }
1538             {
1539                 \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1540                 \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1541             }
1542             { 4 }
1543             {
1544                 \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1545                 \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1546             }
1547             { 5 }
1548             {
1549                 \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1550                 \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1551             }
1552         }
1553     },
1554     size .initial:n = none,
1555     size .value_required:n = true,
1556     size / unknown .code:n =
1557     \msg_error:nneee { spintent } { unknown-choice }
1558     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1559     silent .bool_set:N = \l__spintent_spusep_silent_bool,
1560     silent .initial:n = false,
1561     silent .value_required:n = true,
1562     unknown .code:n =
1563     {
1564         \str_set:Nn \l__spintent_command_name_str { spusep }
1565         \__spintent_unknown_keys_cmd:n {#1}
1566     },
1567 }
```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1568 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1569 {
1570     \tl_put_right:cn { \l__spintent_spusep_ #1 _tl } { #2 }
1571     \bool_if:NTF \l__spintent_spusep_silent_bool
1572     {
1573         \MathMLintert { :silent } { \tl_use:c { \l__spintent_spusep_ #1 _tl } }
1574     }
1575     {
1576         \tl_use:c { \l__spintent_spusep_ #1 _tl }
1577     }
1578 }
```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n` The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1579 \cs_new_protected:Nn \__spintent_spusep_convert:n
1580 {
1581   \tl_set:Nx \__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1582   \tl_if_empty:NT \__spintent_spusep_arg_tl
1583   {
1584     \msg_error:nnn { spintent } { spusep-empty-arg } {#1}
1585   }
1586   \str_case:VnF \__spintent_spusep_arg_tl
1587   {
1588     { ( }
1589     {
1590       \__spintent_spusep_set_parens:nn { left } { \lparen }
1591     }
1592     { ) }
1593     {
1594       \__spintent_spusep_set_parens:nn { right } { \rparen }
1595     }
1596     { [ }
1597     {
1598       \__spintent_spusep_set_parens:nn { left } { \lbrack }
1599     }
1600     { ] }
1601     {
1602       \__spintent_spusep_set_parens:nn { right } { \rbrack }
1603     }
1604     { \{ }
1605     {
1606       \__spintent_spusep_set_parens:nn { left } { \lbrace }
1607     }
1608     { \} }
1609     {
1610       \__spintent_spusep_set_parens:nn { right } { \rbrace }
1611     }
1612   }
1613   {
1614     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1615   }
1616 }

```

(End of definition for `\__spintent_spusep_convert:n`.)

`\spusep` The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1617 \NewDocumentCommand \spusep { O{} m }
1618 {
1619   \mode_if_math:TF
1620   {
1621     \group_begin:
1622     \keys_set:nn { spintent / spusep } {#1}
1623     \__spintent_spusep_convert:n {#2}
1624     \group_end:
1625   }
1626   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1627 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

spnewread-empty-args
spnewread-duplicate
spread-unknown-sym

The “error” `spnewread-empty-args` is raised when `\spnewread` receives a blank symbol or a reading that is empty once normalized. The “error” `spnewread-duplicate` is raised when it registers a symbol already in the dictionary. The “error” `spread-unknown-sym` is raised when `\spread` is given a symbol that is neither registered nor accompanied by `read-arg`.

```

1628 \msg_new:nnnn { spintent } { spnewread-empty-args }
1629 {

```

```

1630     Empty~argument~in~\token_to_str:N \spnewread.
1631   }
1632   {
1633     \token_to_str:N \spnewread~requires~two~non-empty~arguments.
1634   }
1635   \msg_new:nnnn { spintent } { spnewread-duplicate }
1636   {
1637     The~symbol~'#1'~is~already~defined!
1638   }
1639   {
1640     You~cannot~redefine~with~\token_to_str:N \spnewread.
1641   }
1642   \msg_new:nnnn { spintent } { spread-empty-sym }
1643   {
1644     Empty~symbol~argument~in~\token_to_str:N \spread.
1645   }
1646   {
1647     \token_to_str:N \spread~requires~a~non-empty~symbol.
1648   }
1649   \msg_new:nnnn { spintent } { spread-unknown-sym }
1650   {
1651     The~symbol~'#1'~is~not~registered~and~no~'read-arg'~was~given.
1652   }
1653   {
1654     Register~it~first~with~\token_to_str:N \spnewread,~or~pass~'read-arg'.
1655   }

```

(End of definition for `spnewread-empty-args`, `spnewread-duplicate`, and `spread-unknown-sym`.)

```

\l__spintent_spread_luaset_status_str
\l__spintent_spread_luaset_reading_str
\l__spintent_spread_read_arg_tl

```

The variables `\l__spintent_spread_luaset_status_str` and `\l__spintent_spread_luaset_reading_str` are set by `\__spintent_luafun_spread_new:n` / `\__spintent_luafun_spread_lookup:n`. The variable `\l__spintent_spread_read_arg_tl` stores `read-arg`.

```

1656 \str_new:N \l__spintent_spread_luaset_status_str
1657 \str_new:N \l__spintent_spread_luaset_reading_str
1658 \tl_new:N \l__spintent_spread_read_arg_tl

```

(End of definition for `\l__spintent_spread_luaset_status_str`, `\l__spintent_spread_luaset_reading_str`, and `\l__spintent_spread_read_arg_tl`.)

```

read-arg
silent
unknown

```

We define the `(keys)` `read-arg` and `silent`.

```

1659 \keys_define:nn { spintent / spread }
1660 {
1661   read-arg .code:n =
1662   {
1663     \__spintent_sanitiz_affix:Nn \l__spintent_spread_read_arg_tl {#1}
1664   },
1665   read-arg .initial:n = ,
1666   read-arg .value_required:n = true,
1667   silent .bool_set:N = \l__spintent_spread_silent_bool,
1668   silent .initial:n = false,
1669   silent .value_required:n = true,
1670   unknown .code:n =
1671   {
1672     \str_set:Nn \l__spintent_command_name_str { spread }
1673     \__spintent_unknown_keys_cmd:n {#1}
1674   },
1675 }

```

(End of definition for `read-arg`, `silent`, and `unknown`.)

```

\__spintent_spread_exec:nn

```

The function `\__spintent_spread_exec:nn` receives the optional `(keys)` in `'#1'` and the `{(symbol)}` in `'#2'`. With `silent` true, the symbol is typeset with a `:silent` intent, nothing else evaluated. Otherwise, with `read-arg` given, it is used directly, overriding any registered reading for this call alone. With `read-arg` empty, calls `\__spintent_luafun_spread_lookup:n`, using the registered reading when found, reporting `spread-unknown-sym` otherwise.

```

1676 \cs_new_protected:Nn \__spintent_spread_exec:nn
1677 {
1678   \keys_set:nn { spintent / spread } { #1 }
1679   \tl_if_blank:nTF { #2 }
1680   { \msg_error:nn { spintent } { spread-empty-sym } }
1681   {

```

```

1682     \__spintent_invisible_sep:
1683     \bool_if:NTF \l__spintent_spread_silent_bool
1684     {
1685         \MathMLint { :silent } { #2 }
1686     }
1687     {
1688         \tl_if_empty:NTF \l__spintent_spread_read_arg_tl
1689         {
1690             \__spintent_luafun_spread_lookup:n { \exp_not:n {#2} }
1691             \str_if_eq:VnTF \l__spintent_spread_luaset_status_str { found }
1692             {
1693                 \MathMLint { _\l__spintent_spread_luaset_reading_str } { #2 }
1694             }
1695             {
1696                 \msg_error:nne { spintent } { spread-unknown-sym } { \exp_not:n {#2} }
1697             }
1698         }
1699         {
1700             \MathMLint { _\l__spintent_spread_read_arg_tl } { #2 }
1701         }
1702     }
1703 }
1704 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

\spread The public command `\spread` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spread_exec:nn`.

```

1705 \NewDocumentCommand \spread { O{} m }
1706 {
1707     \mode_if_math:TF
1708     {
1709         \group_begin:
1710         \__spintent_spread_exec:nn { #1 } { #2 }
1711         \group_end:
1712     }
1713     { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1714 }

```

(End of definition for `\spread`. This function is documented on page 12.)

\spnewread The public command `\spnewread` registers a new symbol/reading pair. It first normalizes the reading ‘#2’ with `\__spintent_sanitizе_affix:Nn`, storing the result in the scratch variable `\l_tmpa_tl`, and reports `spnewread-empty-args` when the symbol ‘#1’ is blank or the normalized reading is empty. Otherwise it passes the symbol and the normalized reading to Lua via `\__spintent_luafun_spread_new:nn`, reporting `spnewread-duplicate` when the symbol is already registered.

```

1715 \NewDocumentCommand \spnewread { m m }
1716 {
1717     \mode_if_math:TF
1718     {
1719         \msg_error:nnn { spintent } { need-text-mode } { \spnewread }
1720     }
1721     {
1722         \__spintent_sanitizе_affix:Nn \l_tmpa_tl { #2 }
1723         \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_empty_p:N \l_tmpa_tl }
1724         { \msg_error:nn { spintent } { spnewread-empty-args } }
1725         {
1726             \__spintent_luafun_spread_new:nn { \exp_not:n {#1} } { \exp_not:V \l_tmpa_tl }
1727             \str_if_eq:VnT \l__spintent_spread_luaset_status_str { duplicate }
1728             {
1729                 \msg_error:nnn { spintent } { spnewread-duplicate } { #1 }
1730             }
1731         }
1732     }
1733 }

```

(End of definition for `\spnewread`. This function is documented on page 11.)

11.23.2 The command \spdsym

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```
\l__spintent_spdsym_read_sym_tl
\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

```
prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
```

```
1738 \keys_define:nn { spintent / spdsym }
1739 {
1740   prefix .code:n =
1741     { \l__spintent_sanitize_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1742   prefix .initial:n = ,
1743   prefix .value_required:n = true,
1744   suffix .code:n =
1745     { \l__spintent_sanitize_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1746   suffix .initial:n = ,
1747   suffix .value_required:n = true,
1748   set-sym .choices:nn =
1749     { old-style , two-dots , slash }
1750   {
1751     \int_case:nn { \l_keys_choice_int }
1752     {
1753       {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1754       {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1755       {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1756     }
1757   },
1758   set-sym .initial:n = two-dots,
1759   set-sym .value_required:n = true,
1760   set-sym / unknown .code:n =
1761     \msg_error:nneee { spintent } { unknown-choice }
1762     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1763   read-sym .choices:nn =
1764     { dividido , dividido-por , dividido-entre }
1765   {
1766     \int_case:nn { \l_keys_choice_int }
1767     {
1768       {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1769       {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1770       {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1771     }
1772   },
1773   read-sym .initial:n = dividido,
1774   read-sym .value_required:n = true,
1775   read-sym / unknown .code:n =
1776     \msg_error:nneee { spintent } { unknown-choice }
1777     { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1778     { \exp_not:n {#1} },
1779   unknown .code:n =
1780   {
1781     \str_set:Nn \l__spintent_command_name_str { spdsym }
1782     \l__spintent_unknown_keys_cmd:n {#1}
1783   },
1784 }
```

(End of definition for `prefix` and others.)

`\spdsym` Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```
1785 \NewDocumentCommand \spdsym { 0{} }
1786 {
1787   \mode_if_math:TF
1788   {
1789     \group_begin:
1790     \keys_set:nn { spintent / spdsym } { #1 }
1791     \MathMLintent
1792     {
```

```

1793         -
1794         \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1795         { \l__spintent_spdsym_prefix_tl - }
1796         \l__spintent_spdsym_read_sym_tl
1797         \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1798         { - \l__spintent_spdsym_suffix_tl }
1799     }
1800     { \l__spintent_spdsym_symbol_tl }
1801 \group_end:
1802 }
1803 { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1804 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1805 \tl_new:N \l__spintent_spmsym_read_sym_tl
1806 \tl_new:N \l__spintent_spmsym_symbol_tl
1807 \tl_new:N \l__spintent_spmsym_prefix_tl
1808 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

```

prefix We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym
read-sym
not-print
unknown
1809 \keys_define:nn { spintent / spmsym }
1810 {
1811     prefix .code:n =
1812     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
1813     prefix .initial:n = ,
1814     prefix .value_required:n = true,
1815     suffix .code:n =
1816     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1817     suffix .initial:n = ,
1818     suffix .value_required:n = true,
1819     set-sym .choices:nn =
1820     { cross , dot , asterisk }
1821     {
1822         \int_case:nn { \l_keys_choice_int }
1823         {
1824             {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1825             {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1826             {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1827         }
1828     },
1829     set-sym .initial:n = dot,
1830     set-sym .value_required:n = true,
1831     set-sym / unknown .code:n =
1832     \msg_error:nneee { spintent } { unknown-choice }
1833     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1834     read-sym .choices:nn =
1835     { por , multiplicado-por , veces }
1836     {
1837         \int_case:nn { \l_keys_choice_int }
1838         {
1839             {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1840             {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1841             {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1842         }
1843     },
1844     read-sym .initial:n = por,
1845     read-sym .value_required:n = true,
1846     read-sym / unknown .code:n =
1847     \msg_error:nneee { spintent } { unknown-choice }
1848     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1849     not-print .bool_set:N = \l__spintent_spmsym_not_print_bool,
1850     not-print .initial:n = false,
1851     not-print .value_required:n = true,
1852     unknown .code:n =
1853     {
1854         \str_set:Nn \l__spintent_command_name_str { spmsym }

```

```

1855     \__spintent_unknown_keys_cmd:n {#1}
1856   },
1857 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the **\spmsym** operator. Similar to **\spdsym**, it outputs the correct mathematical glyph based on the local *(keys)*, safely attaching prefixes and suffixes if present.

```

1858 \NewDocumentCommand \spmsym { 0{} }
1859 {
1860   \mode_if_math:TF
1861   {
1862     \group_begin:
1863     \keys_set:nn { spintent / spmsym } { #1 }
1864     \MathMLintent
1865     {
1866       -
1867       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1868       { \l__spintent_spmsym_prefix_tl - }
1869       \l__spintent_spmsym_read_sym_tl
1870       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1871       { - \l__spintent_spmsym_suffix_tl }
1872     }
1873     {
1874       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1875       {
1876         \invisibletimes
1877       }
1878       { \l__spintent_spmsym_symbol_tl }
1879     }
1880   \group_end:
1881 }
1882 { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1883 }

```

(End of definition for *\spmsym*. This function is documented on page 13.)

11.24 The command **\spdiv**

The user-level command **\spdiv** formats division representations. It takes two mandatory arguments: the *{(dividend)}* and the *{(divisor)}*. Both *{(arguments)}* are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *{(dividend)}* nor the *{(divisor)}* is *blank*.

11.24.1 Definition of keys for **\spdiv**

We define the *(keys)* **wrap-parens** and **read-parens** for **\spdiv** command. Number-related *(keys)* **cifras-sep**, **read-sign**, **read-period-sep** and **notation-sym** are forwarded to **\spnum** command using **.meta:nn**, while semantic operator *(keys)* **set-sym** and **read-sym** are forwarded to the **\spdsym** command using **.meta:nn**.

```

1884 \keys_define:nn { spintent / spdiv }
1885 {
1886   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1887   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1888   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1889   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1890   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1891   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1892   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1893   wrap-parens     .initial:n = false,
1894   wrap-parens     .value_required:n = true,
1895   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1896   read-parens     .initial:n = true,
1897   read-parens     .value_required:n = true,
1898   unknown         .code:n =
1899   {
1900     \str_set:Nn \l__spintent_command_name_str { spdiv }
1901     \__spintent_unknown_keys_cmd:n {#1}
1902   },
1903 }

```

(End of definition for *cifras-sep* and others.)

11.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\langle\textit{dividend}\rangle$ or the $\langle\textit{divisor}\rangle$ is missing or *blank*.

```

1904 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1905 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1906 {
1907   You~must~provide~both~a~dividend~and~a~divisor.\\
1908 }

```

(End of definition for `spdiv-empty-argument`.)

11.24.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1909 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1910 {
1911   \spnum {#1}
1912   \MathMLintent { _ \__spintent_spdsym_read_sym_tl }
1913   {
1914     \__spintent_spdsym_symbol_tl
1915   }
1916   \spnum {#2}
1917 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1918 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1919 {
1920   \bool_if:NTF \__spintent_spdiv_read_parens_bool
1921   {
1922     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1923   }
1924   {
1925     \MathMLintent { :silent } { ( }
1926     \__spintent_spdiv_render_op:nn {#1} {#2}
1927     \MathMLintent { :silent } { ) }
1928   }
1929 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn` The function `\__spintent_spdiv_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1930 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1931 {
1932   \keys_set:nn { spintent / spdiv } { #1 }
1933   \tl_if_blank:nTF { #2 }
1934   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1935   {
1936     \tl_if_blank:nTF { #3 }
1937     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1938     {
1939       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1940       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1941       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1942     }
1943   }
1944 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1945 \NewDocumentCommand \spdiv { 0{ } m m }
1946 {
1947   \mode_if_math:TF
1948   {
1949     \group_begin:
1950     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1951     \group_end:
1952   }
1953   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1954 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{\first factor}` and the `{\second factor}`. Both `{\arguments}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{\first factor}` nor the `{\second factor}` is *blank*.

11.25.1 Definition of keys for `\spmul`

We define the `(keys)` `wrap-parens` and `read-parens` for `\spmul` command. Number-related `(keys)` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `(keys)` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1955 \keys_define:nn { spintent / spmul }
1956 {
1957   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1958   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1959   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1960   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1961   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1962   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1963   wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1964   wrap-parens     .initial:n  = false,
1965   wrap-parens     .value_required:n = true,
1966   read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
1967   read-parens     .initial:n  = true,
1968   read-parens     .value_required:n = true,
1969   unknown         .code:n =
1970   {
1971     \str_set:Nn \l__spintent_command_name_str { spmul }
1972     \__spintent_unknown_keys_cmd:n {#1}
1973   },
1974 }

```

(End of definition for `cifras-sep` and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the `{\first factor}` or the `{\second factor}` is missing or *blank*.

```

1975 \msg_new:nnnn { spintent } { spmul-empty-argument }
1976 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1977 {
1978   You~must~provide~both~first~factor~and~second~factor.\\
1979 }

```

(End of definition for `spmul-empty-argument`.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both `{\arguments}` through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1980 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1981 {
1982   \spnum {#1}
1983   \MathMLintent { _ \l__spintent_spmsym_read_sym_tl }
1984   {
1985     \l__spintent_spmsym_symbol_tl
1986   }
1987   \spnum {#2}
1988 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1989 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1990 {
1991   \bool_if:NTF \l__spintent_spmul_read_parens_bool
1992   {
1993     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1994   }
1995   {
1996     \MathMLintent { :silent } { ( ) }

```

```

1997     \__spintent_spmul_render_op:nn {#1} {#2}
1998     \MathMLintent { :silent } { ) }
1999   }
2000 }

```

(End of definition for __spintent_spmul_render_parens:nn.)

__spintent_spmul_process:nnn

The function `\__spintent_spmul_process:nnn` sets `\keys`, validates `\{arguments\}`, and decides on parenthesis wrapping.

```

2001 \cs_new_protected:Nn \__spintent_spmul_process:nnn
2002 {
2003   \keys_set:nn { spintent / spmul } { #1 }
2004   \tl_if_blank:nTF { #2 }
2005   { \msg_error:nnn { spintent } { spmul-empty-argument } }
2006   {
2007     \tl_if_blank:nTF { #3 }
2008     { \msg_error:nnn { spintent } { spmul-empty-argument } }
2009     {
2010       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
2011       { \__spintent_spmul_render_parens:nn {#2} {#3} }
2012       { \__spintent_spmul_render_op:nn {#2} {#3} }
2013     }
2014   }
2015 }

```

(End of definition for __spintent_spmul_process:nnn.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```

2016 \NewDocumentCommand \spmul { O{} m m }
2017 {
2018   \mode_if_math:TF
2019   {
2020     \group_begin:
2021     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
2022     \group_end:
2023   }
2024   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
2025 }

```

(End of definition for `\spmul`. This function is documented on page 14.)

11.26 The commands `\spgcd` and `\spmcn`

The user-level commands `\spgcd` and `\spmcn` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for `\spgcd` and `\spmcn`

```

\__spintent_spmcm_spmcd_luaset_error_str
\__spintent_mc_luaset_types_str
\__spintent_mc_luaset_all_numeric_str
\__spintent_mc_args_seq
\__spintent_mc_raw_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl

```

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

2026 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
2027 \clist_new:N \__spintent_mc_luaset_types_clist
2028 \str_new:N \__spintent_mc_luaset_all_numeric_str
2029 \seq_new:N \__spintent_mc_args_seq
2030 \seq_new:N \__spintent_mc_raw_args_seq
2031 \seq_new:N \__spintent_mc_types_seq
2032 \int_new:N \__spintent_mc_idx_int
2033 \clist_map_inline:nn { mcd , mcm }
2034 {
2035   \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2036   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
2037   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2038 }

```

(End of definition for `\__spintent_spmcm_spmcd_luaset_error_str` and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

2039 \msg_new:nnnn { spintent } { mc-invalid-arguments }
2040 {
2041   Invalid~arguments~provided~to~#1.
2042 }
2043 {
2044   The~arguments~must~be~a~comma-separated~list. \\\
2045 }

```

(End of definition for mc-invalid-arguments.)

11.26.3 Keys for \spmcdd and \spmcm

result

We define the shared *(keys)* result, prefix, read-cmd, upper-case, concept and silent.

prefix

read-cmd

upper-case

concept

silent

unknown

```

2046 \clist_map_inline:nn { mcd , mcm }
2047 {
2048   \keys_define:nn { spintent / sp #1 }
2049   {
2050     result      .bool_set:c = { l__spintent_sp #1 _print_result_bool },
2051     result      .initial:n = false,
2052     result      .value_required:n = true,
2053     prefix      .code:n =
2054       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
2055     prefix      .initial:n = ,
2056     prefix      .value_required:n = true,
2057     read-cmd    .choices:nn =
2058       { terse , clear }
2059       {
2060         \int_case:nn { \l_keys_choice_int }
2061         {
2062           {1} { \bool_set_true:c { l__spintent_sp #1 _read_terse_bool } }
2063           {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
2064         }
2065       },
2066     read-cmd    .initial:n = clear,
2067     read-cmd    .value_required:n = true,
2068     read-cmd / unknown .code:n =
2069       \msg_error:nneee { spintent } { unknown-choice }
2070       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
2071     upper-case  .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
2072     upper-case  .initial:n = false,
2073     upper-case  .value_required:n = true,
2074     concept     .bool_set:c = { l__spintent_sp #1 _concept_bool },
2075     concept     .initial:n = true,
2076     concept     .value_required:n = true,
2077     silent      .bool_set:c = { l__spintent_sp #1 _silent_bool },
2078     silent      .initial:n = false,
2079     silent      .value_required:n = true,
2080     unknown     .code:n =
2081       {
2082         \str_set:Nn \l__spintent_command_name_str { sp #1 }
2083         \__spintent_unknown_keys_cmd:n {##1}
2084       },
2085   }
2086 }

```

(End of definition for result and others.)

11.26.4 Internal functions for \spmcdd and \spmcm

__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintent_mc_raw_args_seq` and `\l__spintent_mc_luaset_types_clist` into `\l__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintent_mc_args_seq`: each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2087 \cs_new_protected:Nn \__spintent_mc_process_args:nn
2088 {
2089   \__spintent_luafun_mc_classify_args:n {#2}
2090   \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#2}

```

```

2091 \seq_set_from_clist:NN \l__spintenc_mc_types_seq \l__spintenc_mc_luaset_types_clist
2092 \seq_clear:N \l__spintenc_mc_args_seq
2093 \int_zero:N \l__spintenc_mc_idx_int
2094 \seq_map_inline:Nn \l__spintenc_mc_raw_args_seq
2095 {
2096   \int_incr:N \l__spintenc_mc_idx_int
2097   \str_if_eq:eeTF
2098   { \seq_item:Nn \l__spintenc_mc_types_seq { \l__spintenc_mc_idx_int } } { number }
2099   {
2100     \seq_put_right:Nn \l__spintenc_mc_args_seq { \spnum {##1} }
2101   }
2102   { \seq_put_right:Nn \l__spintenc_mc_args_seq {##1} }
2103 }

```

Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`.

```

2104 \bool_if:cTF { l__spintenc_sp #1 _silent_bool }
2105 {
2106   \seq_set_map:NNn \l__spintenc_mc_args_seq \l__spintenc_mc_args_seq
2107   {
2108     \MathMLintent { _- } { \l__spintenc_inner_sep: ##1 }
2109   }
2110   \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent "por"
2111   \MathMLintent { :silent } { ( }
2112   \seq_use:Nnnn \l__spintenc_mc_args_seq
2113   { \MathMLintent { :silent } { , } }
2114   { \MathMLintent { :silent } { , } }
2115   { \MathMLintent { :silent } { , } }
2116 }
2117 {
2118   \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent doble "de"
2119   \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2120   {
2121     \MathMLintent { _entre } { ( }
2122   }
2123   {
2124     \MathMLintent { :silent } { ( }
2125   }
2126   \seq_use:Nnnn \l__spintenc_mc_args_seq
2127   {
2128     \MathMLintent { :silent } { , }
2129     \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" medio
2130   }
2131   {
2132     \MathMLintent { :silent } { , }
2133   }
2134   {
2135     \MathMLintent { :silent } { , }
2136     \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" final
2137   }
2138 }
2139 \MathMLintent { :silent } { ) }
2140 }

```

(End of definition for `\l__spintenc_mc_process_args:nn`)

`\l__spintenc_spmc_process:nnnn`

The internal function `\l__spintenc_spmc_process:nnnn` is the shared handler for both `\spmcld` and `\spmcm`. Takes the short identifier in ‘#1’, its full reading string in ‘#2’, the local *(keys)* in ‘#3’, and the optional arguments in ‘#4’.

The operator name is silenced with `silent=true` — `\l__spintenc_mc_process_args:nn` silences the arguments too, in that case, each wrapped in a literal `_-` intent, and the printed result is silenced as well — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal intent instead — `#1` or `#2` per `read-cmd`, prefixed with `prefix` when given.

With ‘#4’ given, classifies it through `\l__spintenc_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\l__spintenc_mc_process_args:n` prints the argument list itself either way.

- We should use `\mathrm` instead of `\operatorname`, which generates unwanted spacing when `\c__spintenc_invisible_sep_int` is applied.

```

2141 \cs_new_protected:Nn \l__spintenc_spmc_process:nnnn
2142 {

```

```

2143 \keys_set:nn { spintent / sp #1 } { #3 }
2144 \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2145 {
2146   \MathMLintent { :silent }
2147   {
2148     \mathrm
2149     {
2150       \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2151       { \text_uppercase:n {#1} } { #1 }
2152     }
2153   }
2154 }
2155 {
2156   \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2157   {
2158     \MathMLintent
2159     {
2160       -
2161       \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2162       {
2163         \tl_use:c { l__spintent_sp #1 _prefix_tl } -
2164       }
2165       \bool_if:cTF { l__spintent_sp #1 _read_terse_bool }
2166       { #1 } { #2 }
2167     }
2168     {
2169       \mathrm
2170       {
2171         \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2172         { \text_uppercase:n {#1} } { #1 }
2173       }
2174     }
2175   }
2176   {
2177     \MathMLintent { :silent }
2178     {
2179       \mathrm
2180       {
2181         \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2182         { \text_uppercase:n {#1} } { #1 }
2183       }
2184     }
2185   }
2186 }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2187 \tl_if_novalue:nF { #4 }
2188 {
2189   \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#4}
2190   \int_compare:nNnTF { \seq_count:N \l__spintent_mc_raw_args_seq } < { 2 }
2191   {
2192     \msg_error:nne { spintent } { mc-too-few-arguments } { \exp_not:c { sp #1 } }
2193   }
2194   {
2195     \__spintent_luafun_mc_classify_args:n {#4}
2196     \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2197     {
2198       \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2199       \str_if_eq:VnT \l__spintent_spmcm_spmcd_luaset_error_str { true }
2200       {
2201         \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2202       }
2203     }
2204     \__spintent_mc_process_args:nn {#1} {#4}
2205     \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2206     {
2207       \bool_if:cT { l__spintent_sp #1 _print_result_bool }

```

```

2208         {
2209             \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2210             {
2211                 \MathMLintent { :silent } { = }
2212                 \MathMLintent { _- }
2213                 {
2214                     \__spintent_inner_sep:
2215                     \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2216                 }
2217             }
2218             {
2219                 \MathMLintent { _es-igual-a } { = }
2220                 \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2221             }
2222         }
2223     }
2224 }
2225 }
2226 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmc` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2227 \NewDocumentCommand \spmc { 0{} d() }
2228 {
2229     \mode_if_math:TF
2230     {
2231         \group_begin:
2232         \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2233         \group_end:
2234     }
2235     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2236 }
2237 \NewDocumentCommand \spmc { 0{} d() }
2238 {
2239     \mode_if_math:TF
2240     {
2241         \group_begin:
2242         \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2243         \group_end:
2244     }
2245     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2246 }

```

(End of definition for `\spmc` and `\spmc`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Definition of variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\l__spintent_md_out_seq
\l__spintent_spmD_luaset_is_number_str
\l__spintent_spmD_luaset_error_str
\l__spintent_spmD_luaset_result_tl
\l__spintent_spmD_luaset_result_tl
\l__spintent_spmD_luaset_is_truncated_str
\l__spintent_spmD_luaset_is_truncated_str
\l__spintent_spmD_prefix_tl
\l__spintent_spmD_prefix_tl
\l__spintent_spmD_print_result_num_tl
\l__spintent_spmD_print_result_num_tl
2247 \seq_new:N \l__spintent_md_out_seq
2248 \str_new:N \l__spintent_spmD_luaset_error_str
2249 \str_new:N \l__spintent_spmD_luaset_is_number_str
2250 \clist_map_inline:nn { nM , nD }
2251 {
2252     \tl_new:c { l__spintent_sp #1 _luaset_result_tl }
2253     \str_new:c { l__spintent_sp #1 _luaset_is_truncated_str }
2254     \tl_new:c { l__spintent_sp #1 _prefix_tl }
2255 }

```

(End of definition for `\l__spintent_md_out_seq` and others.)

11.27.2 Keys for \spnM and \spnD

We define the shared *(keys)* `result`, `silent`, `result-num`, `prefix` and `concept`.

```

2256 \clist_map_inline:nn { nM , nD }
2257 {
2258   \keys_define:nn { spintent / sp #1 }
2259   {
2260     result      .bool_set:c = { l__spintent_sp #1 _print_result_bool },
2261     result      .initial:n = false,
2262     result      .value_required:n = true,
2263     silent      .bool_set:c = { l__spintent_sp #1 _silent_bool },
2264     silent      .initial:n = false,
2265     silent      .value_required:n = true,
2266     result-num  .tl_set:c = { l__spintent_sp #1 _print_result_num_tl },
2267     result-num  .initial:n = ,
2268     prefix      .code:n =
2269       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
2270     prefix      .initial:n = ,
2271     prefix      .value_required:n = true,
2272     concept     .bool_set:c = { l__spintent_sp #1 _concept_bool },
2273     concept     .initial:n = true,
2274     concept     .value_required:n = true,
2275     unknown     .code:n =
2276       {
2277         \str_set:Nn \__spintent_command_name_str { sp #1 }
2278         \__spintent_unknown_keys_cmd:n {##1}
2279       },
2280   }
2281 }

```

(End of definition for `result` and others.)

11.27.3 Definition of error messages for \spnM and \spnD

`md-invalid-argument` Exception triggered when the argument for multiples or divisors is invalid.

```

2282 \msg_new:nnnn { spintent } { md-invalid-argument }
2283 {
2284   Invalid~argument~provided~to~#1~\msg_line_context:.
2285 }
2286 {
2287   The~argument~must~be~a~single~positive~integer. \
2288 }

```

Exception triggered when fewer than two arguments are provided.

```

2289 \msg_new:nnnn { spintent } { mc-too-few-arguments }
2290 {
2291   Too~few~arguments~provided~to~#1~\msg_line_context:.
2292 }
2293 {
2294   At~least~two~arguments~are~required. \
2295 }

```

(End of definition for `md-invalid-argument` and `mc-too-few-arguments`.)

11.27.4 Internal functions for \spnM and \spnD

`\__spintent_spnM_spnD_process:nnnn` The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spnM_and_D_exec:nnnn`.

```

2296 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnn
2297 {
2298   \keys_set:nn { spintent / sp #1 } { #4 }
2299   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2300 }

```

(End of definition for `\__spintent_spnM_spnD_process:nnnn`.)

`\__spintent_spnM_spnD_intent:nn` The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends ‘-de’.

```

2301 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2302 {

```

```

2303 -
2304 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2305 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
2306 \str_case:nnF { #2 } { { multiplos } { múltiplos } }
2307 { #2 }
2308 -de
2309 }

```

(End of definition for `\__spintent_spnM_spnD_intent:nn`.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules. The argument is wrapped in a literal `_` intent, not in `:silent`, so that intents of its own (such as those of `\spnum`) are silenced too.

```

2310 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2311 {
2312   \MathMLintent { :silent } { \operatorname { #1 } }
2313   \MathMLintent { :silent } { ( }
2314   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2315   \MathMLintent { _- } { \__spintent_inner_sep: #2 }
2316   \MathMLintent { :silent } { ) }
2317 }

```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:nn`

The internal function `\__spintent_spnM_and_D_result:nn` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’ and the classified numeric argument ‘#2’. It invokes the corresponding Lua backend calculation, reporting `md-invalid-argument` if that fails — a safety net beyond `\__spintent_luafun_spnmd_classify:n`’s own check, since the Lua calculation never trusted that check blindly. On success, it maps the resulting clist into a sequence, and formats the final printed output utilizing custom intents for both separators and conjunctions.

```

2318 \cs_new_protected:Nn \__spintent_spnM_spnD_result:nn
2319 {
2320   \use:c { __spintent_luafun_calcular_ #1 :nn } { #2 }
2321   {
2322     \tl_use:c { l__spintent_sp #1 _print_result_num_tl }
2323   }
2324   \str_if_eq:VnTF \__spintent_spnM_spnD_luaset_error_str { true }
2325   {
2326     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2327   }
2328   {
2329     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2330     \MathMLintent { _son } { = \{ }
2331     \seq_set_from_clist:Ne
2332       \__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luaset_result_tl } }
2333     \seq_use:Nnnn \__spintent_md_out_seq
2334       { \MathMLintent { _y- } { , } }
2335       { \MathMLintent { :silent } { , } }
2336       {
2337         \MathMLintent { :silent } { , }
2338         \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { false }
2339         {
2340           \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" final
2341         }
2342       }
2343     \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { true }
2344     {
2345       \MathMLintent { :silent } { , } ~ \dots
2346     }
2347     \MathMLintent { :silent } { \} }
2348   }
2349 }

```

(End of definition for `\__spintent_spnM_spnD_result:nn`.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `\__spintent_luafun_spnmd_classify` decides whether it is wrapped through `\spnum`— and, when it is, whether `\__spintent_spnM_spnD_result:nn` also prints the computed set, per `result`— or left verbatim, with no result attempted. A zero is flagged as an error by `\__spintent_luafun_spnmd_classify:n` and rejected right away with `md-invalid-argument`. With `silent=true` the printed result is wrapped in a literal `- intent`, so it stays on the page but is not verbalized.

Either way, `silent` silences the whole thing through `\__spintent_spnM_spnD_silent:nn`; otherwise `concept=true` carries the literal `intent` from `\__spintent_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2350 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2351 {
2352   \tl_if_novalue:nTF { #4 }
2353   {
2354     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2355     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2356     {
2357       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2358       {
2359         \MathMLintent
2360         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2361         { \operatorname { #2 } }
2362       }
2363       { \MathMLintent { :silent } { \operatorname { #2 } } }
2364       \MathMLintent { :silent } { ( }
2365       \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2366       n
2367       \MathMLintent { :silent } { ) }
2368     }
2369   }
2370   {
2371     \__spintent_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2372     \str_if_eq:VnT \l__spintent_spnM_spnD_luaset_error_str { true }
2373     { \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } } }
2374     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2375     {
2376       \str_if_eq:VnTF \l__spintent_spnmd_luaset_is_number_str { true }
2377       { \__spintent_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2378       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2379     }
2380     {
2381       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2382       {
2383         \MathMLintent
2384         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2385         { \operatorname { #2 } }
2386       }
2387       { \MathMLintent { :silent } { \operatorname { #2 } } }
2388       \MathMLintent { :silent } { ( }
2389       \str_if_eq:VnTF \l__spintent_spnmd_luaset_is_number_str { true }
2390       { \spnum { #4 } }
2391       {
2392         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2393         #4
2394       }
2395       \MathMLintent { :silent } { ) }
2396     }
2397     \str_if_eq:VnT \l__spintent_spnmd_luaset_is_number_str { true }
2398     {
2399       \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2400       {
2401         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2402         {
2403           \MathMLintent { - }
2404           {
2405             \__spintent_inner_sep:
2406             \__spintent_spnM_spnD_result:nn { #1 } { #4 }
2407           }
2408         }
2409         {
2410           \__spintent_spnM_spnD_result:nn { #1 } { #4 }

```

```

2411         }
2412     }
2413 }
2414 }
2415 }

```

(End of definition for `\__spintent_spnM_spnD_exec:nnnn`.)

`\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintent_spnM_spnD_process:nnnnn`, passing along the optional *keys* ‘#1’ and the numeric *argument* ‘#2’.

```

2416 \NewDocumentCommand \spnM { O{} d() }
2417 {
2418     \mode_if_math:TF
2419     {
2420         \group_begin:
2421         \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } {#1} {#2}
2422         \group_end:
2423     }
2424     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2425 }
2426 \NewDocumentCommand \spnD { O{} d() }
2427 {
2428     \mode_if_math:TF
2429     {
2430         \group_begin:
2431         \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } {#1} {#2}
2432         \group_end:
2433     }
2434     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2435 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 15.)

11.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per `use-dfrac`), read natively — no Lua parsing involved.

11.28.1 Definition of keys for `\sprazon`

We define the *keys* `silent`, `use-dfrac` and `inline`.

```

silent
use-dfrac
inline
unknown
2436 \keys_define:nn { spintent / sprazon }
2437 {
2438     silent      .bool_set:N = \__spintent_sprazon_silent_bool,
2439     silent      .initial:n = false,
2440     silent      .value_required:n = true,
2441     use-dfrac   .bool_set:N = \__spintent_sprazon_use_dfrac_bool,
2442     use-dfrac   .initial:n = false,
2443     use-dfrac   .value_required:n = true,
2444     inline      .bool_set:N = \__spintent_sprazon_inline_bool,
2445     inline      .initial:n = false,
2446     inline      .value_required:n = true,
2447     unknown     .code:n =
2448     {
2449         \str_set:Nn \__spintent_command_name_str { sprazon }
2450         \__spintent_unknown_keys_cmd:n {#1}
2451     },
2452 }

```

(End of definition for `silent` and others.)

`\__spintent_sprazon_exec:nnn` The function `\__spintent_sprazon_exec:nnn` receives the optional *keys* in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under `silent`, the whole thing is silenced as one unit — `\frac/\dfrac` (per `use-dfrac`) normally, or `#2 : #3` under `inline`.

Otherwise, with `inline=false` (the default), numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside `\frac/\dfrac`, and the outer `\MathMLintent` reads `_es-a:infix($num,$denom)`. With `inline=true`, the same two references are tagged side by side instead — no `\frac/\dfrac` involved — with the `:` between them carrying the `_es-a:infix($num,$denom)` intent on its own.

```

2453 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2454 {

```

```

2455 \keys_set:nn { spintent / sprazon } { #1 }
2456 \__spintent_invisible_sep:
2457 \bool_if:NTF \l__spintent_sprazon_silent_bool
2458 {
2459   \MathMLintent { _- }
2460   {
2461     \__spintent_inner_sep:
2462     \bool_if:NTF \l__spintent_sprazon_inline_bool
2463       { #2 : #3 }
2464       {
2465         \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2466           { \dfrac { #2 } { #3 } }
2467           { \frac { #2 } { #3 } }
2468         }
2469       }
2470   }
2471 {
2472   \bool_if:NTF \l__spintent_sprazon_inline_bool
2473   {
2474     \MathMLintent { _es-a:infix($num,$denom) }
2475     {
2476       \MathMLarg { num } { #2 }
2477       :
2478       \MathMLarg { denom } { #3 }
2479     }
2480   }
2481   {
2482     \MathMLintent { _es-a:infix($num,$denom) }
2483     {
2484       \__spintent_inner_sep:
2485       \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2486         {
2487           \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2488         }
2489         { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2490       }
2491     }
2492   }
2493 }

```

(End of definition for `\__spintent_sprazon_exec:nnn`.)

`\sprazon` The public command `\sprazon` enforces *math mode* and delegates execution to `\__spintent_sprazon_exec:nnn`.

```

2494 \NewDocumentCommand \sprazon { O{} m m }
2495 {
2496   \mode_if_math:TF
2497   {
2498     \group_begin:
2499     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2500     \group_end:
2501   }
2502   { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2503 }

```

(End of definition for `\sprazon`. This function is documented on page 18.)

11.29 The command `\spProp`

The user command `\spProp` typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing `\__spintent_sprazon_exec:nnn` directly (not the public `\sprazon` command) for each ratio.

`\spProp` The public command `\spProp` enforces *math mode*, and forwards the same *(keys)* in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native `\MathMLintent` reading `_como` over the $=$ sign.

```

2504 \NewDocumentCommand \spProp { O{} m m m m }
2505 {
2506   \mode_if_math:TF
2507   {
2508     \group_begin:
2509     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2510     \bool_if:NTF \l__spintent_sprazon_silent_bool
2511       {
2512         \MathMLintent { :silent } { = }

```

```

2513         }
2514         { \MathMLintent { _como } { = } }
2515         \__spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2516     \group_end:
2517 }
2518 { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2519 }

```

(End of definition for `\spProp`. This function is documented on page 18.)

11.30 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

```

\l__spintent_spang_luaset_grado_str
\l__spintent_spang_luaset_minuto_str
\l__spintent_spang_luaset_segundo_str
\l__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2520 \str_new:N \l__spintent_spang_luaset_grado_str
2521 \str_new:N \l__spintent_spang_luaset_minuto_str
2522 \str_new:N \l__spintent_spang_luaset_segundo_str
2523 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l__spintent_spang_luaset_grado_str` and others.)

11.30.1 Definition of error messages for `\spang`

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2524 \msg_new:nnnn { spintent } { invalid-angle-format }
2525 { Invalid~angle~format~in~#1. }
2526 {
2527     The~argument~'#2'~could~not~be~parsed. \\\
2528 }

```

(End of definition for `invalid-angle-format`.)

11.30.2 Internal functions for `\spang`

`\__spintent_spang_process:n`

The internal function `\__spintent_spang_process:n` parses the angle input using `\__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT. Each value is expanded with `\exp_args:Ne` before it reaches `\spunit`, which protects its `{\argument}` from expansion.

```

2529 \cs_new_protected:Nn \__spintent_spang_process:n
2530 {
2531     \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2532     \__spintent_luafun_spang_parse:n { #1 }
2533     \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2534     {
2535         \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2536     }
2537     {
2538         \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2539         {
2540             \exp_args:Ne \spunit { \l__spintent_spang_luaset_grado_str ° }
2541         }
2542         \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2543         {
2544             \MathMLintent { _minutos:postfix($m) }
2545             {
2546                 \MathMLarg { m }
2547                 {
2548                     \exp_args:Ne \spunit { \l__spintent_spang_luaset_minuto_str ' }
2549                 }
2550             }
2551         }
2552         \str_if_empty:NF \l__spintent_spang_luaset_segundo_str

```

```

2553     {
2554         \MathMLIntant { _segundos:postfix($s) }
2555     {
2556         \MathMLarg { s }
2557     {
2558         \exp_args:Ne \spunit { \l__spintent_spang_luaset_segundo_str " }
2559     }
2560     }
2561 }
2562 }
2563 }

```

(End of definition for `\__spintent_spang_process:n`.)

\spang The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the `{\argument}`.

```

2564 \NewDocumentCommand \spang { m }
2565 {
2566     \mode_if_math:TF
2567     {
2568         \group_begin:
2569         \__spintent_spang_process:n { #1 }
2570         \group_end:
2571     }
2572     { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2573 }

```

(End of definition for `\spang`. This function is documented on page 24.)

11.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by `MathCAT`.

This command verifies that the `{\argument}` is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.31.1 Definition of keys for `\spnZ`

Now we define the `\keys` `cifras-sep` and `read-sign`. These are `.meta:nn` `\keys` passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by `MathCAT`.

```

2574 \keys_define:nn { spintent / spnZ }
2575 {
2576     cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2577     read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2578     wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2579     wrap-parens .initial:n = false,
2580     wrap-parens .value_required:n = true,
2581     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2582     read-parens .initial:n = true,
2583     read-parens .value_required:n = true,
2584     unknown .code:n =
2585     {
2586         \str_set:Nn \l__spintent_command_name_str { spnZ }
2587         \__spintent_unknown_keys_cmd:n {#1}
2588     },
2589 }

```

(End of definition for `cifras-sep` and others.)

11.31.2 Definition of error messages

spnz-not-integer Raised when the `{\argument}` contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2590 \msg_new:nnnn { spnZ } { spnz-not-integer }
2591 { The~value~!#1!~is~not~a~valid~integer. }
2592 {
2593     Command~\token_to_str:N \spnZ~only~accepts~integers.\\
2594 }

```

(End of definition for `spnz-not-integer`.)

11.31.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2595 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2596 {
2597   \bool_if:NTF \__spintent_spnz_read_parens_bool
2598   {
2599     ( \__spintent_spnnum_print_num_start: )
2600   }
2601   {
2602     \MathMLintent { :silent } { ( }
2603     \__spintent_spnnum_print_num_start:
2604     \MathMLintent { :silent } { ) }
2605   }
2606 }

```

(End of definition for __spintent_spnz_render_parens:.)

__spintent_spnz_process:nn

The function `\__spintent_spnz_process:nn` first processes the `<keys>` passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnnum_print_num_start:`.

```

2607 \cs_new_protected:Nn \__spintent_spnz_process:nn
2608 {
2609   \keys_set:nn { spintent / spnZ } {#1}
2610   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2611   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2612   {
2613     \tl_if_empty:NTF \__spintent_luaset_part_int_tl
2614     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2615     {
2616       \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2617       { \__spintent_spnz_render_parens: }
2618       { \__spintent_spnnum_print_num_start: }
2619     }
2620   }
2621   { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2622 }

```

(End of definition for __spintent_spnz_process:nn.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2623 \NewDocumentCommand \spnZ { O{} m }
2624 {
2625   \mode_if_math:TF
2626   {
2627     \group_begin:
2628     \__spintent_spnz_process:nn { #1 } { #2 }
2629     \group_end:
2630   }
2631   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2632 }

```

(End of definition for \spnZ. This function is documented on page 17.)

11.32 The command \spmQ

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct `MathML` intent for `MathCAT`.

11.32.1 Definition of variables for \spmQ

The variable `\l__spintrent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\l__spintrent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in the integer part, used by the function `\__spintrent_spmQ_print_scale_int:n`.

The variables `\l__spintrent_spmQ_wrap_parens_l_tl` and `\l__spintrent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintrent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\l__spintrent_saved_luamml_flag_int` temporarily stores the current value of `\l__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```
2633 \str_new:N \l__spintrent_spmQ_join_word_str
2634 \str_new:N \l__spintrent_spmQ_intent_read_sign_str
2635 \tl_new:N \l__spintrent_spmQ_wrap_parens_l_tl
2636 \tl_new:N \l__spintrent_spmQ_wrap_parens_r_tl
2637 \int_new:N \l__spintrent_saved_luamml_flag_int
```

(End of definition for `\l__spintrent_spmQ_join_word_str` and others.)

11.32.2 Definition of error messages for \spmQ

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\langle argument \rangle}` is not actually an integer.

```
2638 \msg_new:nnnn { spintrent } { spmQ-zero-denominator }
2639 {
2640   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2641 }
2642 {
2643   The~denominator~of~a~fraction~cannot~be~zero.\\
2644 }
2645 \msg_new:nnnn { spintrent } { spmQ-not-integer }
2646 {
2647   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2648 }
2649 {
2650   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2651 }
```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.32.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `use-dfrac`.

```
join-word
use-spnZ
wrap-parens
read-parens
use-dfrac
unknown
2652 \keys_define:nn { spintrent / spmQ }
2653 {
2654   join-word .choices:nn =
2655     { entero , enteros , y }
2656     {
2657       \int_case:nn { \l_keys_choice_int }
2658       {
2659         {1} { \str_set:Nn \l__spintrent_spmQ_join_word_str { entero } }
2660         {2} { \str_set:Nn \l__spintrent_spmQ_join_word_str { enteros } }
2661         {3} { \str_set:Nn \l__spintrent_spmQ_join_word_str { y } }
2662       }
2663     },
2664   join-word .initial:n = y,
2665   join-word .value_required:n = true,
2666   join-word/unknown .code:n =
2667     \msg_error:nnee { spintrent } { unknown-choice }
2668     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2669   use-spnZ .bool_set:N = \l__spintrent_spmQ_use_spnZ_bool,
2670   use-spnZ .initial:n = true,
2671   use-spnZ .value_required:n = true,
2672   use-dfrac .bool_set:N = \l__spintrent_spmQ_use_dfrac_bool,
2673   use-dfrac .initial:n = false,
2674   use-dfrac .value_required:n = true,
2675   wrap-parens .bool_set:N = \l__spintrent_spmQ_wrap_parens_bool,
2676   wrap-parens .initial:n = false,
2677   wrap-parens .value_required:n = true,
```

```

2678     read-parens      .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2679     read-parens      .initial:n = true,
2680     read-parens      .value_required:n = true,
2681     unknown          .code:n =
2682     {
2683         \str_set:Nn \l__spintent_command_name_str { spmQ }
2684         \__spintent_unknown_keys_cmd:n {#1}
2685     },
2686 }

```

(End of definition for join-word and others.)

11.32.4 Internal functions for \spmQ

__spintent_spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LaTeX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigr\lparen` and `\bigr\rparen`; any other style falls back to unsized `\lparen` and `\rparen`.

If the key `use-dfrac` is active, the size is always forced to `\Bigl/\Bigr` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2687 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2688 {
2689     \int_case:nnF { \mathstyle }
2690     {
2691         { 0 }
2692         {
2693             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2694             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2695         }
2696         { 1 }
2697         {
2698             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2699             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2700         }
2701         { 2 }
2702         {
2703             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigr \lparen }
2704             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2705         }
2706         { 3 }
2707         {
2708             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigr \lparen }
2709             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2710         }
2711     }
2712     {
2713         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2714         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2715     }
2716 }

```

If the key `use-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl/\Bigr` unconditionally.

```

2716     \bool_if:NT \l__spintent_spmQ_use_dfrac_bool
2717     {
2718         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2719         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2720     }
2721 }

```

(End of definition for __spintent_spmQ_set_wrap_parens_size:.)

__spintent_spmQ_start_wrap_parens:
__spintent_spmQ_stop_wrap_parens:

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2722 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2723 {

```

```

2724 \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2725 {
2726   \__spintent_spmQ_set_wrap_parens_size:
2727   \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2728     { \l__spintent_spmQ_wrap_parens_l_tl }
2729     { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_l_tl } }
2730 }
2731 }
2732 \cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
2733 {
2734   \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2735   {
2736     \__spintent_spmQ_set_wrap_parens_size:
2737     \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2738       { \l__spintent_spmQ_wrap_parens_r_tl }
2739       { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_r_tl } }
2740   }
2741 }

```

(End of definition for __spintent_spmQ_start_wrap_parens: and __spintent_spmQ_stop_wrap_parens:.)

__spintent_spmQ_set_join_word:

The function `\__spintent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2742 \cs_new_protected:Nn \__spintent_spmQ_set_join_word:
2743 {
2744   \str_case:Nn \l__spintent_spmQ_join_word_str
2745   {
2746     { entero } { \MathMLintent { _entero } { \c__spintent_invisible_sep_int } }
2747     { enteros } { \MathMLintent { _enteros } { \c__spintent_invisible_sep_int } }
2748     { y } { \MathMLintent { _y- } { \c__spintent_invisible_sep_int } }
2749   }
2750 }

```

(End of definition for __spintent_spmQ_set_join_word:.)

__spintent_spmQ_print_scale_int:

The function `\__spintent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintent_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintent_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `use-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2751 \cs_new_protected:Nn \__spintent_spmQ_print_scale_int:
2752 {
2753   \int_set_eq:Nc \l__spintent_saved_luamml_flag_int { \l__luamml_flag_int }
2754   \int_set:Nn \l__luamml_flag_int { 0 }
2755   \bool_if:NTF \l__spintent_spmQ_use_dfrac_bool
2756     { \__spintent_luafun_spmQ_scale_int:Nn \l__spintent_luaset_part_int_tl { dfrac } }
2757     { \__spintent_luafun_spmQ_scale_int:Nn \l__spintent_luaset_part_int_tl { frac } }
2758   \int_set_eq:cN { \l__luamml_flag_int } \l__spintent_saved_luamml_flag_int
2759 }

```

(End of definition for __spintent_spmQ_print_scale_int:.)

__spintent_spmQ_print_scale_int:n

The function `\__spintent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\__spintent_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\l__spintent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spintent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An

`\__spintent_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2760 \cs_new_protected:Nn \__spintent_spmQ_print_scale_int:n
2761 {
2762   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2763   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2764   {
2765     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
2766     {
2767       \__spintent_spmQ_print_scale_int:
2768     }
2769     {
2770       \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
2771       {
2772         \str_set:Nn
2773           \l__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2774       }
2775       {
2776         \str_set:Nn
2777           \l__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2778       }
2779       \MathMLintent { \l__spintent_spmQ_intent_read_sign_str }
2780       {
2781         \l__spintent_luaset_sign_tl \c__spintent_invisible_sep_int % \__spintent_invisible_s
2782         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2783       }
2784     }
2785   }
2786   { \msg_error:nnn { spintent } { spmQ-not-integer } { #1 } }
2787 }

```

(End of definition for `\__spintent_spmQ_print_scale_int:n`.)

`\__spintent_spmQ_print_spmZ_dfrac:nn`

The function `\__spintent_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘`#1`’ and the denominator in ‘`#2`’. It first calls `\__spintent_luafun_clean_split_and_set:` on ‘`#1`’ to validate that the numerator is an integer; if `\l__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintent_spmQ_use_spmZ_bool` and `\l__spintent_spmQ_use_dfrac_bool`, controlled respectively by the keys `use-spmZ` and `use-dfrac`. When `use-spmZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘`#1`’ and ‘`#2`’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `use-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2788 \cs_new_protected:Nn \__spintent_spmQ_print_spmZ_dfrac:nn
2789 {
2790   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2791   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { false }
2792   {
2793     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2794   }
2795   \bool_if:NTF \l__spintent_spmQ_use_spmZ_bool
2796   {
2797     \use:c { \bool_if:NT \l__spintent_spmQ_use_dfrac_bool { d } frac }
2798     { \spnZ { #1 } } { \spnZ { #2 } }
2799   }
2800   {
2801     \use:c { \bool_if:NT \l__spintent_spmQ_use_dfrac_bool { d } frac }
2802     { #1 } { #2 }
2803   }
2804 }

```

(End of definition for `\__spintent_spmQ_print_spmZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘`#1`’, the numerator ‘`#2`’, and the denominator ‘`#3`’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:.` Then it calls `\__spintent_spmQ_print_spmZ_dfrac:nn` to

typeset the fraction using the keys `use-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```
2805 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2806 {
2807   \__spintent_spmQ_start_wrap_parens:
2808   \__spintent_spmQ_print_scale_int:n { #1 }
2809   \__spintent_spmQ_set_join_word:
2810   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2811   \__spintent_spmQ_stop_wrap_parens:
2812 }
```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn`

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\l__spintent_luaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```
2813 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2814 {
2815   \keys_set:nn { spintent / spmQ } { #1 }
2816   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2817   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2818   {
2819     \int_compare:nNnTF { \l__spintent_luaset_part_int_tl } = { 0 }
2820     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2821     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2822   }
2823   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2824 }
```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```
2825 \NewDocumentCommand \spmQ { O{} m m m }
2826 {
2827   \mode_if_math:TF
2828   {
2829     \group_begin:
2830     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2831     \group_end:
2832   }
2833   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2834 }
```

(End of definition for `\spmQ`. This function is documented on page 17.)

11.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l__spintent_geo_luaset_error_str`, `\l__spintent_geo_luaset_intent_str`, `\l__spintent_geo_luaset_print_tl`, `\l__spintent_geo_luaset_sub_str` and `\l__spintent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\__spintent_mathmlarg_point:nnn`. Every command also activates `\l__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```
\l__spintent_geo_luaset_error_str
\l__spintent_geo_luaset_intent_str
\l__spintent_geo_luaset_print_tl
\l__spintent_geo_luaset_is_name_str
\l__spintent_geo_luaset_poly_sym_str
\l__spintent_geo_luaset_angulo_case_str
\l__spintent_geo_luaset_concept_str
\l__spintent_geo_luaset_letra_word_str
\l__spintent_geo_luaset_sub_spoken_str
\l__spintent_geo_luaset_nombre_es_num_str
```

```
2835 \str_new:N \l__spintent_geo_luaset_error_str
2836 \str_new:N \l__spintent_geo_luaset_intent_str
2837 \tl_new:N \l__spintent_geo_luaset_print_tl
2838 \clist_new:N \l__spintent_geo_luaset_points_clist
2839 \str_new:N \l__spintent_geo_luaset_is_name_str
2840 \str_new:N \l__spintent_geo_luaset_poly_sym_str
2841 \str_new:N \l__spintent_geo_luaset_angulo_case_str
2842 \str_new:N \l__spintent_geo_luaset_concept_str
```

```

2843 \str_new:N \l__spinttent_geo_luaset_letra_word_str
2844 \str_new:N \l__spinttent_geo_luaset_sub_spoken_str
2845 \str_new:N \l__spinttent_geo_luaset_nombre_es_num_str
2846 \str_new:N \l__spinttent_geo_luaset_letra_str
2847 \str_new:N \l__spinttent_geo_luaset_sub_str
2848 \str_new:N \l__spinttent_geo_luaset_tipo_str

```

(End of definition for `\l__spinttent_geo_luaset_error_str` and others.)

11.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x,y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as más/menos — never positivo/negativo, which is not how MathCAT reads a bare sign in this position.

11.34.1 Definition of variables for `\spcoord`

Variables set by the function `__spinttent_luafun_spcoord_parse_and_set:n`. The string variable `\l__spinttent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\l__spinttent_spcoord_luaset_#1_sign_str` is `+`, `-`, or empty; the variables `\l__spinttent_spcoord_luaset_#1_type_str` is `number` or `raw`; the variables `\l__spinttent_spcoord_luaset_#1_value_str` is the value with the sign already removed; and the variables `\l__spinttent_spcoord_luaset_#1_has_frac_str` is `none`, `frac`, or `dfrac`.

```

\l__spinttent_spcoord_luaset_error_str
\l__spinttent_spcoord_luaset_sep_str
\l__spinttent_spcoord_luaset_x_sign_str
\l__spinttent_spcoord_luaset_x_type_str
\l__spinttent_spcoord_luaset_x_value_str
\l__spinttent_spcoord_luaset_x_has_frac_str
\l__spinttent_spcoord_luaset_y_sign_str
\l__spinttent_spcoord_luaset_y_type_str
\l__spinttent_spcoord_luaset_y_value_str
\l__spinttent_spcoord_luaset_y_has_frac_str

2849 \str_new:N \l__spinttent_spcoord_luaset_error_str
2850 \str_new:N \l__spinttent_spcoord_luaset_sep_str
2851 \clist_map_inline:nn { x , y }
2852 {
2853   \str_new:c { \l__spinttent_spcoord_luaset_#1_sign_str }
2854   \str_new:c { \l__spinttent_spcoord_luaset_#1_type_str }
2855   \str_new:c { \l__spinttent_spcoord_luaset_#1_value_str }
2856   \str_new:c { \l__spinttent_spcoord_luaset_#1_has_frac_str }
2857 }

```

(End of definition for `\l__spinttent_spcoord_luaset_error_str` and others.)

The variable `\l__spinttent_spcoord_raw_arg_tl` holds the $\langle\textit{argument}\rangle$ exactly as given. The variable `\l__spinttent_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\l__spinttent_spcoord_wrap_parens_l_tl`/`\l__spinttent_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `__spinttent_spcoord_set_wrap_parens_size:.`

```

2858 \tl_new:N \l__spinttent_spcoord_raw_arg_tl
2859 \str_new:N \l__spinttent_spcoord_intent_str
2860 \tl_new:N \l__spinttent_spcoord_wrap_parens_l_tl
2861 \tl_new:N \l__spinttent_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spinttent_spcoord_raw_arg_tl` and others.)

11.34.2 Definition of error messages for `\spcoord`

`spcoord-invalid-arg`

```

2862 \msg_new:nnn { spinttent } { spcoord-invalid-arg }
2863 {
2864   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2865 }

```

(End of definition for `spcoord-invalid-arg`.)

11.34.3 Definition of keys for `\spcoord`

`silent` We define the key `silent`.

```

2866 \keys_define:nn { spinttent / spcoord }
2867 {
2868   silent .bool_set:N = \l__spinttent_spcoord_silent_bool,
2869   silent .initial:n = false,
2870   silent .value_required:n = true,
2871   unknown .code:n =
2872   {
2873     \str_set:Nn \l__spinttent_command_name_str { spcoord }
2874     \__spinttent_unknown_keys_cmd:n {#1}
2875   },
2876 }

```

(End of definition for `silent`.)

11.34.4 Internal functions for \spcoord

`\__spintrent_spcoord_set_wrap_parens_size:` With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```

2877 \cs_new_protected:Nn \__spintrent_spcoord_set_wrap_parens_size:
2878 {
2879   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \lparen }
2880   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \rparen }
2881   \bool_if:nT
2882     {
2883       \bool_lazy_or_p:nn
2884         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { none } } }
2885         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { none } } }
2886     }
2887   {
2888     \int_case:nnF { \mathstyle }
2889     {
2890       { 0 }
2891       {
2892         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2893         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2894       }
2895       { 1 }
2896       {
2897         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2898         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2899       }
2900       { 2 }
2901       {
2902         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2903         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2904       }
2905       { 3 }
2906       {
2907         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2908         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2909       }
2910     }
2911   }
2912   \bool_if:nT
2913   {
2914     \bool_lazy_or_p:nn
2915       { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { dfraction } }
2916       { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { dfraction } }
2917   }
2918   {
2919     \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2920     \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2921   }
2922 }
2923

```

(End of definition for `\__spintrent_spcoord_set_wrap_parens_size:`)

`\__spintrent_spcoord_value:n` With `\l__spintrent_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\l__spintrent_spcoord_luaset_ #1 _value_str`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in `#1`.

```

2924 \cs_new_protected:Nn \__spintrent_spcoord_value:n
2925 {
2926   \str_if_eq:eeTF
2927     { \str_use:c { \l__spintrent_spcoord_luaset_ #1 _type_str } } { number }
2928     {
2929       \exp_args:Ne \spnum { \str_use:c { \l__spintrent_spcoord_luaset_ #1 _value_str } }
2930     }
2931     { \str_use:c { \l__spintrent_spcoord_luaset_ #1 _value_str } }
2932 }

```

(End of definition for `\__spintrent_spcoord_value:n`)

`\__spintrent_spcoord_print:n` With `\l__spintrent_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintrent_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read` as `más/menos`. With no sign, delegates directly to `\__spintrent_spcoord_value:n`. Receives `x` or `y` in `#1`.

```

2933 \cs_new_protected:Nn \__spintrent_spcoord_print:n

```

```

2934 {
2935   \str_case:enF { \str_use:c { l__spintrent_spcoord_luaset_ #1 _sign_str } }
2936   {
2937     { + }
2938     {
2939       \MathMLintent { _más:prefix ($vp) }
2940       {
2941         \str_use:c { l__spintrent_spcoord_luaset_ #1 _sign_str }
2942         \MathMLarg{ vp } { \__spintrent_spcoord_value:n { #1 } }
2943       }
2944     }
2945     { - }
2946     {
2947       \MathMLintent { _menos :prefix($vn) }
2948       {
2949         \str_use:c { l__spintrent_spcoord_luaset_ #1 _sign_str }
2950         \MathMLarg{ vn } { \__spintrent_spcoord_value:n { #1 } }
2951       }
2952     }
2953   }
2954   { \__spintrent_spcoord_value:n { #1 } }
2955 }

```

(End of definition for `\__spintrent_spcoord_print:n`.)

`\__spintrent_spcoord_intent_infix:` Sets `\l__spintrent_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\l__spintrent_spcoord_intent_infix:choice`. Then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintrent_spcoord_print:n`'s own output.

```

2956 \cs_new_protected:Nn \__spintrent_spcoord_intent_infix:
2957 {
2958   \str_if_eq:VnTF \l__spintrent_spcoord_luaset_sep_str { ; }
2959   { \str_set:Nn \l__spintrent_spcoord_intent_str { _punto-y-coma:infix } }
2960   { \str_set:Nn \l__spintrent_spcoord_intent_str { _coma:infix } }
2961   \MathMLintent{ \l__spintrent_spcoord_intent_str ($x, $y)}
2962   {
2963     \MathMLarg{x}{ \__spintrent_spcoord_print:n { x } }
2964     \MathMLintent { :silent } { \l__spintrent_spcoord_luaset_sep_str }
2965     \MathMLarg{y}{ \__spintrent_spcoord_print:n { y } }
2966   }
2967 }

```

(End of definition for `\__spintrent_spcoord_intent_infix:`.)

`\__spintrent_spcoord_mathml:` With `silent=true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\l__spintrent_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintrent_spcoord_print:n/\__spintrent_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintrent_spcoord_intent_infix:` provides the reading in between.

```

2968 \cs_new_protected:Nn \__spintrent_spcoord_mathml:
2969 {
2970   \bool_if:NNTF \l__spintrent_spcoord_silent_bool
2971   {
2972     \MathMLintent { _- }
2973     {
2974       \l__spintrent_spcoord_wrap_parens_l_tl
2975       \l__spintrent_spcoord_raw_arg_tl
2976       \l__spintrent_spcoord_wrap_parens_r_tl
2977     }
2978   }
2979   {
2980     \MathMLintent { :silent } { \l__spintrent_spcoord_wrap_parens_l_tl }
2981     \__spintrent_spcoord_intent_infix:
2982     \MathMLintent { :silent } { \l__spintrent_spcoord_wrap_parens_r_tl }
2983   }
2984 }

```

(End of definition for `\__spintrent_spcoord_mathml:`.)

`\__spintrent_spcoord_exec:nn` Sets the *(keys)* in `#1` and `\l__spintrent_spcoord_raw_arg_tl` to `#2`. Calls `\__spintrent_luafun_spcoord_parse` either way — even with `silent=true`, since `\__spintrent_spcoord_set_wrap_parens_size:` still needs `\l__spintrent_spcoord_luaset_ #1 _has_frac_str` — but with `silent=true`, forces `\l__spintrent_spcoord_silent_bool` back to `false` rather than reporting it: the raw *(argument)* is typeset regardless of whether it parsed.

Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\__spintent_spcoord_mathml:` unless there was an error.

```

2985 \cs_new_protected:Nn \__spintent_spcoord_exec:nn
2986 {
2987   \keys_set:nn { spintent / spcoord } {#1}
2988   \tl_set:Nn \__spintent_spcoord_raw_arg_tl {#2}
2989   \bool_if:NTF \__spintent_spcoord_silent_bool
2990   {
2991     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2992     \str_set:Nn \__spintent_spcoord_luastr_error_str { false }
2993   }
2994   {
2995     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2996     \str_if_eq:VnT \__spintent_spcoord_luastr_error_str { true }
2997     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#2} } }
2998   }
2999   \__spintent_spcoord_set_wrap_parens_size:
3000   \str_if_eq:VnF \__spintent_spcoord_luastr_error_str { true }
3001   { \__spintent_spcoord_mathml: }
3002 }

```

(End of definition for `\__spintent_spcoord_exec:nn`.)

`\spcoord` The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spcoord_exec:nn`.

```

3003 \NewDocumentCommand \spcoord { 0{ } m }
3004 {
3005   \mode_if_math:TF
3006   {
3007     \group_begin:
3008     \attribute \__spintent_scope_attr_int = 1
3009     \__spintent_spcoord_exec:nn {#1} {#2}
3010     \group_end:
3011   }
3012   { \msg_error:nnn { spintent } { need-math-mode } { \spcoord } }
3013 }

```

(End of definition for `\spcoord`. This function is documented on page 19.)

11.35 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command's own `read-cmd/silent` state, looked up by name (#2).

```

3014 \tl_new:N \__spintent_mathmlarg_point_ref_tl
3015 \str_new:N \__spintent_mathmlarg_point_mode_str
3016 \tl_new:N \__spintent_mathmlarg_point_kv_tl
3017 \bool_new:N \__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\__spintent_mathmlarg_point_ref_tl` and others.)

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\__spintent_geo_luastr_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

3018 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
3019 {
3020   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
3021   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
3022   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_bool }
3023   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
3024   \__spintent_invisible_sep:
3025   \str_case:VnF \__spintent_geo_luastr_tipo_str
3026   {
3027     { sub }
3028     {
3029       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3030       {
3031         \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
3032         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua

```

```

3033         \__spintent_luafun_mrow_intent_arg:nn
3034         { } { \__spintent_mathmlarg_point_kv_tl }
3035     {
3036         \__spintent_geo_luaset_letra_str
3037         \c_math_subscript_token
3038         { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
3039     }
3040 }
3041 {
3042     {
3043         \__spintent_inner_sep:
3044         \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
3045         {
3046             {
3047                 \__spintent_geo_luaset_letra_str
3048                 \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3049             }
3050         }
3051     }
3052 }
3053 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

3054     { prima }
3055     {
3056         \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3057         {
3058             \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
3059             { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
3060             \__spintent_luafun_mrow_intent_arg:nn
3061             { } { \__spintent_mathmlarg_point_kv_tl }
3062             { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
3063         }
3064         {
3065             {
3066                 \__spintent_inner_sep:
3067                 \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
3068                 {
3069                     { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
3070                 }
3071             }
3072         }
3073     }
3074 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3075     {
3076         \bool_if:NTF \__spintent_mathmlarg_point_silent_bool
3077         {
3078             \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
3079             { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "- " }
3080             \__spintent_luafun_mrow_intent_arg:nn
3081             { } % debe ir vacío
3082             { \__spintent_mathmlarg_point_kv_tl }
3083             { \__spintent_geo_luaset_print_tl }
3084         }
3085         {
3086             \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3087             {
3088                 \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
3089                 { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
3090                 \__spintent_luafun_mrow_intent_arg:nn
3091                 { } % debe ir vacío
3092                 { \__spintent_mathmlarg_point_kv_tl }
3093                 { \__spintent_geo_luaset_print_tl }
3094             }
3095         }

```

```

3096             \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
3097             { \l__spintent_geo_luaseta_print_tl }
3098         }
3099     }
3100 }
3101 }
3102 \cs_generate_variant:Nn \l__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\l__spintent_mathmlarg_point:nnn`.)

11.36 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.36.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
3103 \str_new:N \l__spintent_spPoint_mode_str
3104 \tl_new:N \l__spintent_spPoint_prefix_tl
3105 \tl_new:N \l__spintent_spPoint_intent_arg_tl

```

(End of definition for `\l__spintent_spPoint_mode_str`, `\l__spintent_spPoint_prefix_tl`, and `\l__spintent_spPoint_intent_arg_tl`.)

11.36.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

3106 \msg_new:nnnn { spintent } { spPoint-invalid-point }
3107 {
3108     Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
3109 }
3110 {
3111     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
3112     subscript~(\_{...})~or~prime~(').\\
3113 }

```

(End of definition for `spPoint-invalid-point`.)

11.36.3 Definition of keys for `\spPoint`

`read-cmd`

We define the *(keys)* `read-cmd`, `prefix`, `silent` and `concept`.

```

prefix
silent
unknown
3114 \keys_define:nn { spintent / spPoint }
3115 {
3116     read-cmd .choices:nn = { school , mathcat }
3117     {
3118         \int_case:nn { \l_keys_choice_int }
3119         {
3120             {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
3121             {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
3122         }
3123     },
3124     read-cmd .initial:n = school,
3125     read-cmd .value_required:n = true,
3126     read-cmd / unknown .code:n =
3127         \msg_error:nneee { spintent } { unknown-choice }
3128         { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3129     prefix .code:n =
3130         { \l__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
3131     prefix .initial:n = ,
3132     prefix .value_required:n = true,
3133     silent .bool_set:N = \l__spintent_spPoint_silent_bool,
3134     silent .initial:n = false,
3135     silent .value_required:n = true,
3136     concept .bool_set:N = \l__spintent_spPoint_concept_bool,
3137     concept .initial:n = true,
3138     concept .value_required:n = true,
3139     unknown .code:n =
3140     {
3141         \str_set:Nn \l__spintent_command_name_str { spPoint }
3142         \l__spintent_unknown_keys_cmd:n {#1}
3143     },
3144 }

```

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for \spPoint

_spinttent_spPoint_print:n

The internal function `\_spinttent_spPoint_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\_spinttent_luafun_geo_point_parse_and_set:n`, reporting `spPoint-invalid-point` on error. If `silent` is `true` or `concept` is `false`, sets `\_spinttent_spPoint_intent_arg_tl` to `\_($pt)` — forced further to `\_` — when `silent` is `true`, whatever `read-cmd` is: a composite point (one with a subscript) is not reliably silenced by the per-point `:silent` alone. Otherwise sets it to `\_punto:prefix($pt)`, with `prefix` placed after the leading `\_` when given. Finally calls `\_spinttent_spPoint_mathml:n`.

```

3145 \cs_new_protected:Nn \_spinttent_spPoint_print:n
3146 {
3147   \_spinttent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
3148   \str_if_eq:VnT \_spinttent_geo_luaset_error_str { true }
3149   {
3150     \msg_error:nne { spinttent } { spPoint-invalid-point } { \exp_not:n {#1} }
3151   }
3152   \bool_if:nTF
3153   {
3154     \bool_lazy_or:p:nn
3155     { \_spinttent_spPoint_silent_bool }
3156     { ! \_spinttent_spPoint_concept_bool }
3157   }
3158   {
3159     \tl_set:Nn \_spinttent_spPoint_intent_arg_tl { \_($pt) }
3160     \bool_if:NT \_spinttent_spPoint_silent_bool
3161     {
3162       \tl_set:Nn \_spinttent_spPoint_intent_arg_tl { \_ - }
3163     }
3164   }
3165   {
3166     \tl_set:Ne \_spinttent_spPoint_intent_arg_tl
3167     {
3168       -
3169       \tl_if_empty:NF \_spinttent_spPoint_prefix_tl
3170       {
3171         \tl_use:N \_spinttent_spPoint_prefix_tl -
3172       }
3173       punto:prefix($pt)
3174     }
3175   }
3176   \_spinttent_spPoint_mathml:n { #1 }
3177 }

```

(End of definition for `\_spinttent_spPoint_print:n`.)

_spinttent_spPoint_mathml:n

The function `\_spinttent_spPoint_mathml:n` calls `\_spinttent_invisible_sep:` first, then wraps `\_spinttent_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\_spinttent_spPoint_intent_arg_tl` as the concept `intent`, with `\_spinttent_inner_sep:` first inside that group.

```

3178 \cs_new_protected:Nn \_spinttent_spPoint_mathml:n
3179 {
3180   \_spinttent_invisible_sep:
3181   \MathMLintent { \_spinttent_spPoint_intent_arg_tl }
3182   {
3183     \_spinttent_inner_sep:
3184     \_spinttent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
3185   }
3186 }

```

(End of definition for `\_spinttent_spPoint_mathml:n`.)

_spinttent_spPoint_exec:nnn

The function `\_spinttent_spPoint_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’ and the point $\langle argument \rangle$ in ‘#2’, delegating to `\_spinttent_spPoint_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\_spinttent_spcoord_silent_bool` to `\_spinttent_spPoint_silent_bool` and calls `\_spinttent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\_spinttent_spcoord_set_wrap_parens_size:` and `\_spinttent_spcoord_mathml:`.

```

3187 \cs_new_protected:Nn \_spinttent_spPoint_exec:nnn
3188 {
3189   \keys_set:nn { spinttent / spPoint } { #1 }
3190   \_spinttent_spPoint_print:n { #2 }
3191   \tl_if_novalue:nF {#3}

```

```

3192     {
3193       \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % silencia el "por"
3194       \bool_set_eq:NN \l__spintent_spcoord_silent_bool \l__spintent_spPoint_silent_bool
3195       \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3196       \bool_if:NTF \l__spintent_spcoord_silent_bool
3197       {
3198         \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3199         \str_set:Nn \l__spintent_spcoord_luaset_error_str { false }
3200       }
3201       {
3202         \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3203         \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
3204         { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3205       }
3206       \__spintent_spcoord_set_wrap_parens_size:
3207       \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
3208       { \__spintent_spcoord_mathml: }
3209     }
3210   }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

3211 \NewDocumentCommand \spPoint { 0 } { m d() }
3212 {
3213   \mode_if_math:TF
3214   {
3215     \group_begin:
3216     \attribute \l__spintent_scope_attr_int = 1
3217     \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
3218     \group_end:
3219   }
3220   { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
3221 }

```

(End of definition for `\spPoint`. This function is documented on page 19.)

11.37 The command `\spvec`

The user command `\spvec` supports two types of notation for the input $\langle \textit{argument} \rangle$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\langle \textit{argument} \rangle$.

11.37.1 Definition of variables for `\spvec`

The variable `\l__spintent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\l__spintent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

3222 \str_new:N \l__spintent_spvec_mode_str
3223 \tl_new:N \l__spintent_spvec_prefix_tl
3224 \tl_new:N \l__spintent_spvec_intent_arg_tl
3225 \tl_new:N \l__spintent_spvec_kv_tl
3226 \seq_new:N \l__spintent_spvec_pt_seq
3227 \tl_new:N \l__spintent_spvec_refs_tl
3228 \int_new:N \l__spintent_spvec_pt_idx_int

```

(End of definition for `\l__spintent_spvec_mode_str` and others.)

11.37.2 Definition of error messages for `\spvec`

`spvec-invalid-arg` The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

3229 \msg_new:nnn { spintent } { spvec-invalid-arg }
3230 {
3231   Invalid~argument~'#1'~for~\token_to_str:N \spvec.
3232 }

```

(End of definition for `spvec-invalid-arg`.)

11.37.3 Definition of keys for \spvec

We define the *(keys)* `silent`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

3233 \keys_define:nn { spintent / spvec }
3234 {
3235   silent      .bool_set:N = \__spintent_spvec_silent_bool,
3236   silent      .initial:n = false,
3237   silent      .value_required:n = true,
3238   concept     .bool_set:N = \__spintent_spvec_concept_bool,
3239   concept     .initial:n = true,
3240   concept     .value_required:n = true,
3241   autofit     .bool_set:N = \__spintent_spvec_autofit_bool,
3242   autofit     .initial:n = false,
3243   autofit     .value_required:n = true,
3244   read-cmd    .choices:nn = { school , mathcat }
3245   {
3246     \int_case:nn { \l_keys_choice_int }
3247     {
3248       {1} { \str_set:Nn \__spintent_spvec_mode_str { school } }
3249       {2} { \str_set:Nn \__spintent_spvec_mode_str { mathcat } }
3250     }
3251   },
3252   read-cmd    .initial:n = school,
3253   read-cmd    .value_required:n = true,
3254   read-cmd / unknown .code:n =
3255     \msg_error:nneee { spintent } { unknown-choice }
3256     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3257   prefix      .code:n =
3258     { \__spintent_sanitizize_affix:Nn \__spintent_spvec_prefix_tl {#1} },
3259   prefix      .initial:n = { },
3260   prefix      .value_required:n = true,
3261   unknown     .code:n =
3262     {
3263       \str_set:Nn \__spintent_command_name_str { spvec }
3264       \__spintent_unknown_keys_cmd:n {#1}
3265     },
3266 }

```

(End of definition for `silent` and others.)

11.37.4 Internal functions for \spvec

`\__spintent_spvec_print:n` The internal function `\__spintent_spvec_print:n` processes the *(argument)* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\__spintent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintent_spvec_mathml:n`.

```

3267 \cs_new_protected:Nn \__spintent_spvec_print:n
3268 {
3269   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3270   \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { true }
3271   {
3272     \str_if_eq:VnT \__spintent_geo_luaset_tipo_str { prima }
3273     { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
3274   }
3275   \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { false }
3276   {
3277     \int_compare:nT { \clist_count:N \__spintent_geo_luaset_points_clist != 2 }
3278     { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
3279   }
3280   \str_if_eq:VnTF \__spintent_geo_luaset_error_str { true }
3281   { \msg_error:nne { spintent } { spvec-invalid-arg } { \exp_not:n {#1} } }
3282   { \__spintent_spvec_mathml:n { #1 } }
3283 }

```

(End of definition for `\__spintent_spvec_print:n`.)

`\__spintent_spvec_mathml:n` The function `\__spintent_spvec_mathml:n` branches on `\__spintent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_( $\$$ pt)` when `silent` is `true` or `concept` is `false` — forced further to `_` when `silent` is `true`, whatever `read-cmd` is; otherwise `_vector:prefix( $\$$ pt)`, with `prefix` placed after the leading `_` when given. With `\__spintent_geo_luaset_tipo` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native

`\MathMLarg` instead, the subscript silenced only under `silent`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same kv logic wraps `\vec{print\_tl}` as a whole.

```

3284 \cs_new_protected:Nn \__spintent_sptvec_mathml:n
3285 {
3286   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
3287   {
3288     \bool_if:nTF
3289     {
3290       \bool_lazy_or_p:nn
3291       { \l__spintent_sptvec_silent_bool }
3292       { ! \l__spintent_sptvec_concept_bool }
3293     }
3294     {
3295       \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { _($pt) }
3296       \bool_if:NT \l__spintent_sptvec_silent_bool
3297       { \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { _- } }
3298     }
3299   }
3300   \tl_set:Ne \l__spintent_sptvec_intent_arg_tl
3301   {
3302     -
3303     \tl_if_empty:NF \l__spintent_sptvec_prefix_tl
3304     { \tl_use:N \l__spintent_sptvec_prefix_tl - }
3305     vector:prefix($pt)
3306   }
3307 }
3308 \str_case:VnF \l__spintent_geo_luaset_tipo_str
3309 {
3310   { sub }
3311   {
3312     \str_if_eq:VnTF \l__spintent_sptvec_mode_str { school }
3313     {
3314       \bool_if:NTF \l__spintent_sptvec_silent_bool
3315       { \tl_set:Nn \l__spintent_sptvec_kv_tl { arg = "pt", intent = "_-" } }
3316       {
3317         \tl_set:Ne \l__spintent_sptvec_kv_tl
3318         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3319       }
3320       \__spintent_invisible_sep:
3321       \MathMLintent { \l__spintent_sptvec_intent_arg_tl }
3322       {
3323         \__spintent_inner_sep:
3324         \__spintent_luafun_mrow_intent_arg:nn
3325         { } { \l__spintent_sptvec_kv_tl }
3326       }
3327       \vec
3328       {
3329         \bool_if:NT \l__spintent_sptvec_autofit_bool
3330         { \vphantom{ b } }
3331         \l__spintent_geo_luaset_letra_str
3332       }
3333       \c_math_subscript_token
3334       { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3335     }
3336   }
3337 }
3338 {
3339   \__spintent_invisible_sep:
3340   \MathMLintent { \l__spintent_sptvec_intent_arg_tl }
3341   {
3342     \__spintent_inner_sep:
3343     \vec
3344     {
3345       \bool_if:NT \l__spintent_sptvec_autofit_bool
3346       { \vphantom{ b } }
3347       \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
3348     }
3349     \bool_if:NT \l__spintent_sptvec_silent_bool
3350     { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3351   }

```

```

3352         \bool_if:NF \l__spintent_sspec_silent_bool
3353         { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3354     }
3355 }
3356 }
3357 {
3358     \bool_if:NTF \l__spintent_sspec_silent_bool
3359     { \tl_set:Nn \l__spintent_sspec_kv_tl { arg = "pt", intent = "-" } }
3360     {
3361         \tl_set:Nn \l__spintent_sspec_kv_tl
3362         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3363     }
3364     \__spintent_invisible_sep:
3365     \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3366     {
3367         \__spintent_inner_sep:
3368         \__spintent_luafun_mrow_intent_arg:nn
3369         { } { \l__spintent_sspec_kv_tl }
3370     }
3371     {
3372         \vec
3373         {
3374             \bool_if:NT \l__spintent_sspec_autofit_bool
3375             { \vphantom{ b } }
3376             \l__spintent_geo_luaset_print_tl
3377         }
3378     }
3379 }
3380 }

```

For a pair of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sspec_pt_seq` and walked to build `\l__spintent_sspec_refs_tl`, the `$pts1`, `$pts2` reference list. The concept `intent` is `_` when `silent` is `true`, `_` (refs) when `concept` is `false` or `_vector:prefix(refs)` otherwise, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3381 {
3382     \seq_set_from_clist:NN \l__spintent_sspec_pt_seq \l__spintent_geo_luaset_points_clist
3383     \tl_clear:N \l__spintent_sspec_refs_tl
3384     \int_zero:N \l__spintent_sspec_pt_idx_int
3385     \seq_map_inline:Nn \l__spintent_sspec_pt_seq
3386     {
3387         \int_incr:N \l__spintent_sspec_pt_idx_int
3388         \tl_if_empty:NF \l__spintent_sspec_refs_tl
3389         { \tl_put_right:Nn \l__spintent_sspec_refs_tl { , } }
3390         \tl_put_right:Nn \l__spintent_sspec_refs_tl { $ pts \int_use:N \l__spintent_sspec_pt_idx_int }
3391     }
3392     \bool_if:NTF
3393     {
3394         \bool_lazy_or_p:nn
3395         { \l__spintent_sspec_silent_bool }
3396         { ! \l__spintent_sspec_concept_bool }
3397     }
3398     {
3399         \tl_set:Nn \l__spintent_sspec_intent_arg_tl { _ ( \l__spintent_sspec_refs_tl ) }
3400         \bool_if:NF \l__spintent_sspec_silent_bool
3401         { \tl_set:Nn \l__spintent_sspec_intent_arg_tl { _- } }
3402     }
3403     {
3404         \tl_set:Nn \l__spintent_sspec_intent_arg_tl
3405         {
3406             -
3407             \tl_if_empty:NF \l__spintent_sspec_prefix_tl
3408             { \tl_use:N \l__spintent_sspec_prefix_tl - }
3409             vector:prefix( \l__spintent_sspec_refs_tl )
3410         }
3411     }
3412     \__spintent_invisible_sep:
3413     \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3414     {
3415         \__spintent_inner_sep:
3416         \overrightarrow

```

```

3417         {
3418             \int_zero:N \__spintent_svec_pt_idx_int
3419             \seq_map_inline:Nn \__spintent_svec_pt_seq
3420             {
3421                 \int_incr:N \__spintent_svec_pt_idx_int
3422                 \__spintent_mathmlarg_point:nnn
3423                 { pts \int_use:N \__spintent_svec_pt_idx_int }
3424                 { svec }
3425                 { ##1 }
3426             }
3427         }
3428     }
3429 }
3430 }

```

(End of definition for `\__spintent_svec_mathml:n`.)

`\__spintent_svec_exec:nnn` The function `\__spintent_svec_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’, delegating to `\__spintent_svec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\__spintent_spcoord_silent_bool` to `\__spintent_svec_silent_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_mathml:` — the same rendering functions `\spcoord` itself uses.

```

3431 \cs_new_protected:Nn \__spintent_svec_exec:nnn
3432 {
3433     \keys_set:nn { spintent / svec } { #1 }
3434     \__spintent_svec_print:n { #2 }
3435     \tl_if_novalue:nF {#3}
3436     {
3437         \bool_if:NTF \__spintent_svec_silent_bool
3438         { \MathMLint { :silent } { \equal } }
3439         { \equal } % notación de vector =( , )
3440         \MathMLint { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
3441         \bool_set_eq:NN \__spintent_spcoord_silent_bool \__spintent_svec_silent_bool
3442         \tl_set:Nn \__spintent_spcoord_raw_arg_tl {#3}
3443         \bool_if:NTF \__spintent_svec_silent_bool
3444         {
3445             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3446             \str_set:Nn \__spintent_spcoord_luaset_error_str { false }
3447         }
3448         {
3449             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3450             \str_if_eq:VnT \__spintent_spcoord_luaset_error_str { true }
3451             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3452         }
3453         \__spintent_spcoord_set_wrap_parens_size:
3454         \str_if_eq:VnF \__spintent_spcoord_luaset_error_str { true }
3455         { \__spintent_spcoord_mathml: }
3456     }
3457 }

```

(End of definition for `\__spintent_svec_exec:nnn`.)

$\backslash$ spvec The public command `\spvec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_svec_exec:nnn`.

```

3458 \NewDocumentCommand \spvec { O{} m d() }
3459 {
3460     \mode_if_math:TF
3461     {
3462         \group_begin:
3463         \attribute \__spintent_scope_attr_int = 1
3464         \__spintent_svec_exec:nnn { #1 } { #2 } { #3 }
3465         \group_end:
3466     }
3467     { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
3468 }

```

(End of definition for `\spvec`. This function is documented on page 20.)

11.38 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\{\langle argument \rangle\}$: when the argument contains two points (separated by an optional comma), `\overlefttrightarrow` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.38.1 Definition of variables for `\sprecta`

The variable `\l__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

3469 \str_new:N \l__spintent_sprecta_mode_str
3470 \tl_new:N \l__spintent_sprecta_prefix_tl
3471 \seq_new:N \l__spintent_sprecta_pt_seq
3472 \tl_new:N \l__spintent_sprecta_refs_tl
3473 \tl_new:N \l__spintent_sprecta_intent_arg_tl
3474 \tl_new:N \l__spintent_sprecta_kv_tl
3475 \int_new:N \l__spintent_sprecta_pt_idx_int

```

(End of definition for `\l__spintent_sprecta_mode_str` and others.)

11.38.2 Definition of error messages for `\sprecta`

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3476 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3477 {
3478   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
3479 }
3480 {
3481   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3482 }

```

(End of definition for `sprecta-invalid-points`.)

11.38.3 Definition of keys for `\sprecta`

We define the $\langle keys \rangle$ `read-cmd`, `prefix` and `silent`.

```

read-cmd 3483 \keys_define:nn { spintent / sprecta }
prefix   3484 {
silent   3485   read-cmd           .choices:nn = { school , mathcat }
unknown  3486   {
3487     \int_case:nn { \l_keys_choice_int }
3488     {
3489       {1} { \str_set:Nn \l__spintent_sprecta_mode_str { school } }
3490       {2} { \str_set:Nn \l__spintent_sprecta_mode_str { mathcat } }
3491     }
3492   },
3493   read-cmd           .initial:n = school,
3494   read-cmd           .value_required:n = true,
3495   read-cmd / unknown .code:n =
3496     \msg_error:nneee { spintent } { unknown-choice }
3497     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3498   prefix             .code:n =
3499     { \__spintent_sanitize_affix:Nn \l__spintent_sprecta_prefix_tl {#1} },
3500   prefix             .initial:n = ,
3501   prefix             .value_required:n = true,
3502   silent             .bool_set:N = \l__spintent_sprecta_silent_bool,
3503   silent             .initial:n = false,
3504   silent             .value_required:n = true,
3505   concept            .bool_set:N = \l__spintent_sprecta_concept_bool,
3506   concept            .initial:n = true,
3507   concept            .value_required:n = true,
3508   unknown            .code:n =
3509     {
3510       \str_set:Nn \l__spintent_command_name_str { sprecta }
3511       \__spintent_unknown_keys_cmd:n {#1}
3512     },
3513   }

```

(End of definition for `read-cmd` and others.)

11.38.4 Internal functions for \sprecta

_spintenz_sprecta_print:n

The internal function `\_spintenz_sprecta_print:n` processes the `{(argument)}` passed in ‘#1’. Calls `\_spintenz_luafun_geo_parse_and_set:n`, reporting sprecta-invalid-points on error, then `\_spintenz_`

```

3514 \cs_new_protected:Nn \_spintenz_sprecta_print:n
3515 {
3516   \_spintenz_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3517   \str_if_eq:VnT \_spintenz_geo_luaset_error_str { true }
3518   {
3519     \msg_error:nne { spintenz } { sprecta-invalid-points } { \exp_not:n {#1} }
3520   }
3521   \_spintenz_sprecta_mathml:n { #1 }
3522 }

```

(End of definition for _spintenz_sprecta_print:n.)

_spintenz_sprecta_mathml:n

The function `\_spintenz_sprecta_mathml:n` branches on `\_spintenz_geo_luaset_is_name_str`. For a proper name (m, l, ...), the concept intent is `_( $pt)` when `silent` is `true` or `concept` is `false` — forced further to `_-` when `silent` is `true` and `read-cmd` is not `school`; otherwise `_recta:prefix($pt)`, with `prefix` placed after the leading `_` when given. With `read-cmd=school`, the sub and prima cases tag the letter with a literal kv that is `_-` when `silent` is `true`.

With `\_spintenz_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the base through `\_spintenz_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags base and subscript together through a native `\MathMLarg` instead.

```

3523 \cs_new_protected:Nn \_spintenz_sprecta_mathml:n
3524 {
3525   \str_if_eq:VnTF \_spintenz_geo_luaset_is_name_str { true }
3526   {
3527     \_spintenz_invisible_sep:
3528     \bool_if:nTF
3529     {
3530       \bool_lazy_or_p:nn
3531       { \_spintenz_sprecta_silent_bool }
3532       { ! \_spintenz_sprecta_concept_bool }
3533     }
3534     {
3535       \tl_set:Nn \_spintenz_sprecta_intent_arg_tl { _($pt) }
3536     }
3537     \bool_if:NT \_spintenz_sprecta_silent_bool
3538     {
3539       \str_if_eq:VnF \_spintenz_sprecta_mode_str { school }
3540       { \tl_set:Nn \_spintenz_sprecta_intent_arg_tl { _- } }
3541     }
3542     {
3543       \tl_set:Ne \_spintenz_sprecta_intent_arg_tl
3544       {
3545         -
3546         \tl_if_empty:NF \_spintenz_sprecta_prefix_tl
3547         { \tl_use:N \_spintenz_sprecta_prefix_tl - }
3548         recta:prefix($pt)
3549       }
3550     }
3551     \str_case:VnF \_spintenz_geo_luaset_tipo_str
3552     {
3553       sub
3554       {
3555         \str_if_eq:VnTF \_spintenz_sprecta_mode_str { school }
3556         {
3557           \bool_if:NTF \_spintenz_sprecta_silent_bool
3558           { \tl_set:Nn \_spintenz_sprecta_kv_tl { arg = "pt", intent = "_-" } }
3559           {
3560             \tl_set:Ne \_spintenz_sprecta_kv_tl
3561             { arg = "pt", intent = "_ \_spintenz_geo_luaset_intent_str -" }
3562           }
3563           \MathMLintent { \_spintenz_sprecta_intent_arg_tl }
3564           {
3565             \_spintenz_inner_sep:
3566             \_spintenz_luafun_mrow_intent_arg:nn
3567             { } { \_spintenz_sprecta_kv_tl }
3568           }
3569           \_spintenz_geo_luaset_letra_str

```

```

3570         \c_math_subscript_token
3571         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3572     }
3573 }
3574 }

```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3575 {
3576     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3577     {
3578         {
3579             \__spintent_inner_sep:
3580             \MathMLarg { pt }
3581             {
3582                 {
3583                     \l__spintent_geo_luaset_letra_str
3584                     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3585                 }
3586             }
3587         }
3588     }
3589 }
3590 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

3591 { prima }
3592 {
3593     \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3594     {
3595         \bool_if:NTF \l__spintent_sprecta_silent_bool
3596         { \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "--" } }
3597         {
3598             \tl_set:Ne \l__spintent_sprecta_kv_tl
3599             { arg = "pt", intent = "- \l__spintent_geo_luaset_intent_str -" }
3600         }
3601         \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3602         {
3603             \__spintent_inner_sep:
3604             \__spintent_luafun_mrow_intent_arg:nn
3605             { } { \l__spintent_sprecta_kv_tl }
3606             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3607         }
3608     }
3609     {
3610         \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3611         {
3612             {
3613                 \__spintent_inner_sep:
3614                 \MathMLarg { pt }
3615                 {
3616                     { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3617                 }
3618             }
3619         }
3620     }
3621 }
3622 }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

3623 {
3624     \bool_if:NTF \l__spintent_sprecta_silent_bool
3625     {
3626         \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "--" }
3627     }
3628     {
3629         \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3630         {

```

```

3631         \tl_set:Nc \l__spintent_sprecta_kv_tl
3632         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3633     }
3634     {
3635         \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt" }
3636     }
3637 }
3638 \__spintent_luafun_mrow_intent_arg:nn
3639 { \l__spintent_sprecta_intent_arg_tl }
3640 { \l__spintent_sprecta_kv_tl }
3641 { \l__spintent_geo_luaset_print_tl }
3642 }
3643 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sprecta_pt_seq` and walked to build `\l__spintent_sprecta_refs_tl`, the $\$pt_1, \$pt_2, \dots$ reference list. The concept `intent` is `_(refs)` or `_recta:prefix(refs)`, same conditions as above, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg` once per point.

```

3644 {
3645     \seq_set_from_clist:NN \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
3646     \tl_clear:N \l__spintent_sprecta_refs_tl
3647     \int_zero:N \l__spintent_sprecta_pt_idx_int
3648     \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3649     {
3650         \int_incr:N \l__spintent_sprecta_pt_idx_int
3651         \tl_if_empty:NF \l__spintent_sprecta_refs_tl
3652         { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
3653         \tl_put_right:Nc \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_
3654     }
3655     \bool_if:nTF
3656     {
3657         \bool_lazy_or:p:nn
3658         { \l__spintent_sprecta_silent_bool }
3659         { ! \l__spintent_sprecta_concept_bool }
3660     }
3661     {
3662         \tl_set:Nc \l__spintent_sprecta_intent_arg_tl { _ ( \l__spintent_sprecta_refs_tl ) }
3663     }
3664     {
3665         \tl_set:Nc \l__spintent_sprecta_intent_arg_tl
3666         {
3667             -
3668             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3669             { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3670             recta:prefix( \l__spintent_sprecta_refs_tl )
3671         }
3672     }
3673     \__spintent_invisible_sep:
3674     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3675     {
3676         \__spintent_inner_sep:
3677         \overleftrightharrow
3678         {
3679             \int_zero:N \l__spintent_sprecta_pt_idx_int
3680             \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3681             {
3682                 \int_incr:N \l__spintent_sprecta_pt_idx_int
3683                 \__spintent_mathmlarg_point:nnn
3684                 { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
3685                 { sprecta }
3686                 { ##1 }
3687             }
3688         }
3689     }
3690 }
3691 }

```

(End of definition for `\__spintent_sprecta_mathml:n`)

`\__spintent_sprecta_exec:nn` The function `\__spintent_sprecta_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{ \langle argu-$

`ment\}` passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

3692 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
3693 {
3694     \keys_set:nn { spintent / sprecta } { #1 }
3695     \__spintent_sprecta_print:n { #2 }
3696 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta` The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

3697 \NewDocumentCommand \sprecta { 0{} m }
3698 {
3699     \mode_if_math:TF
3700     {
3701         \group_begin:
3702         \attribute \__spintent_scope_attr_int = 1
3703         \__spintent_sprecta_exec:nn { #1 } { #2 }
3704         \group_end:
3705     }
3706     { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3707 }

```

(End of definition for `\sprecta`. This function is documented on page 20.)

11.39 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

11.39.1 Definition of variables for `\sprayo`

The variable `\l__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\l__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

🍷 The variables defined here are assigned using `.choices:n` or `.code:n` within `\keys_define:nn`.

```

3708 \str_new:N \l__spintent_sprayo_mode_str
3709 \tl_new:N \l__spintent_sprayo_prefix_tl
3710 \str_new:N \l__spintent_sprayo_read_sty_str

```

(End of definition for `\l__spintent_sprayo_mode_str`, `\l__spintent_sprayo_prefix_tl`, and `\l__spintent_sprayo_read_sty_str`.)

11.39.2 Definition of error messages for `\sprayo`

`sprayo-invalid-points` Raised when the argument is not a comma-separated pair of uppercase letters.

```

3711 \msg_new:nnnn { spintent } { sprayo-invalid-points }
3712 {
3713     Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3714 }
3715 {
3716     See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3717 }

```

(End of definition for `sprayo-invalid-points`.)

11.39.3 Definition of keys for `\sprayo`

`read-cmd` We define the `(keys)` `read-cmd`, `prefix`, `silent`, `concept` and `read-sty`.

```

prefix
silent
read-sty
unknown
3718 \keys_define:nn { spintent / sprayo }
3719 {
3720     read-cmd .choices:nn = { school , mathcat }
3721     {
3722         \int_case:nn { \l_keys_choice_int }
3723         {
3724             {1} { \str_set:Nn \l__spintent_sprayo_mode_str { school } }
3725             {2} { \str_set:Nn \l__spintent_sprayo_mode_str { mathcat } }
3726         }
3727     },
3728     read-cmd .initial:n = school,
3729     read-cmd .value_required:n = true,
3730     read-cmd / unknown .code:n =
3731     { \msg_error:nneee { spintent } { unknown-choice }
3732       { read-cmd } { school,~mathcat } { \exp_not:n {#1} } },
3733     prefix .code:n =

```

```

3734     { \__spintent_sanitizе_affix:Nn \__spintent_sprayo_prefix_tl {#1} },
3735     prefix                .initial:n = ,
3736     prefix                .value_required:n = true,
3737     silent                .bool_set:N = \__spintent_sprayo_silent_bool,
3738     silent                .initial:n = false,
3739     silent                .value_required:n = true,
3740     concept               .bool_set:N = \__spintent_sprayo_concept_bool,
3741     concept               .initial:n = true,
3742     concept               .value_required:n = true,
3743     read-sty              .choices:nn = { rayo , semirrecta }
3744     {
3745       \int_case:nn { \l_keys_choice_int }
3746       {
3747         {1} { \str_set:Nn \__spintent_sprayo_read_sty_str { rayo } }
3748         {2} { \str_set:Nn \__spintent_sprayo_read_sty_str { semirrecta } }
3749       }
3750     },
3751     read-sty              .initial:n = rayo,
3752     read-sty              .value_required:n = true,
3753     read-sty / unknown    .code:n =
3754       \msg_error:nneee { spintent } { unknown-choice }
3755       { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
3756     unknown               .code:n =
3757     {
3758       \str_set:Nn \__spintent_command_name_str { sprayo }
3759       \__spintent_unknown_keys_cmd:n {#1}
3760     },
3761   }

```

(End of definition for read-cmd and others.)

11.39.4 Internal functions for \sprayo

```

\__spintent_sprayo_pt_seq
\__spintent_sprayo_refs_tl
\__spintent_sprayo_intent_arg_tl
\__spintent_sprayo_pt_idx_int

```

Auxiliary variables used by `\__spintent_sprayo_mathml:` to split `\__spintent_geo_luaset_points_clist` and build the `$pts1`, `$pts2`, ... reference list.

```

3762 \seq_new:N \__spintent_sprayo_pt_seq
3763 \tl_new:N \__spintent_sprayo_refs_tl
3764 \tl_new:N \__spintent_sprayo_intent_arg_tl
3765 \int_new:N \__spintent_sprayo_pt_idx_int

```

(End of definition for __spintent_sprayo_pt_seq and others.)

```
\__spintent_sprayo_print:n
```

The internal function `\__spintent_sprayo_print:n` process the `{(argument)}` passed in ‘#1’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process the `{(argument)}`, returning an “error” if the input is invalid, then calls `\__spintent_sprayo_mathml:`. Unlike `\sprecta`, `\sprayo` has no proper-name argument: it always takes a list of points, so no `intent` is built here.

```

3766 \cs_new_protected:Nn \__spintent_sprayo_print:n
3767 {
3768   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3769   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3770   {
3771     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3772   }
3773   \__spintent_sprayo_mathml:
3774 }

```

(End of definition for __spintent_sprayo_print:n.)

```
\__spintent_sprayo_mathml:
```

The function `\__spintent_sprayo_mathml:` splits `\__spintent_geo_luaset_points_clist` (set by Lua, one raw point per element) into `\__spintent_sprayo_pt_seq` and walks it once to build `\__spintent_sprayo_refs_tl`, the comma-separated `$pts1`, `$pts2`, ... reference list, naming each point by position.

The concept `intent` is `_` when `silent` is `true`, `_(refs)` when `concept` is `false`; otherwise it is `_\__spintent_sprayo_read_sty_str:prefix(refs)` — the concept word itself (`rayo` or `semirrecta`) comes from the `read-sty` key, not a literal, since `\sprayo` verbalizes under two names — prefixed with `prefix` when given. It is applied with a native `\MathMLintent` wrapping `\overrightarrow`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sprayo:` it alone decides `arg`/literal `intent` per point, so `concept` plays no role at that level.

```
3775 \cs_new_protected:Nn \__spintent_sprayo_mathml:
```

```

3776 {
3777   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3778   \tl_clear:N \l__spintent_sprayo_refs_tl
3779   \int_zero:N \l__spintent_sprayo_pt_idx_int
3780   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3781     {
3782       \int_incr:N \l__spintent_sprayo_pt_idx_int
3783       \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3784       { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
3785       \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3786     }
3787   \bool_if:nTF
3788     {
3789       \bool_lazy_or:p:nn
3790       { \l__spintent_sprayo_silent_bool }
3791       { ! \l__spintent_sprayo_concept_bool }
3792     }
3793     {
3794       \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _ ( \l__spintent_sprayo_refs_tl ) }
3795       \bool_if:NT \l__spintent_sprayo_silent_bool
3796       { \tl_set:Nn \l__spintent_sprayo_intent_arg_tl { _- } }
3797     }
3798     {
3799       \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
3800       {
3801         -
3802         \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3803         { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3804         \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3805       }
3806     }
3807   \__spintent_invisible_sep:
3808   \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
3809   {
3810     \__spintent_inner_sep:
3811     \overrightarrow
3812     {
3813       \int_zero:N \l__spintent_sprayo_pt_idx_int
3814       \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3815       {
3816         \int_incr:N \l__spintent_sprayo_pt_idx_int
3817         \__spintent_mathmlarg_point:nnn
3818         { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3819         { sprayo }
3820         { ##1 }
3821       }
3822     }
3823   }
3824 }

```

(End of definition for `\__spintent_sprayo_mathml:`.)

`\__spintent_sprayo_exec:nn` The function `\__spintent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{\langle argument \rangle\}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```

3825 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
3826 {
3827   \keys_set:nn { spintent / sprayo } { #1 }
3828   \__spintent_sprayo_print:n { #2 }
3829 }

```

(End of definition for `\__spintent_sprayo_exec:nn`.)

`\sprayo` The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

3830 \NewDocumentCommand \sprayo { 0{} m }
3831 {
3832   \mode_if_math:TF
3833   {
3834     \group_begin:
3835     \attribute \l__spintent_scope_attr_int = 1
3836     \__spintent_sprayo_exec:nn { #1 } { #2 }
3837     \group_end:

```

```

3838     }
3839     { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
3840 }

```

(End of definition for `\sprayo`. This function is documented on page 21.)

11.40 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.40.1 Definition of variables for `\spside` and `\spmside`

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\__spintent_spside_mode_str` and `\__spintent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spside_prefix_tl` and `\__spintent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\__spintent_spside_read_sty_str` and `\__spintent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\__spintent_spside_read_arg_tl` and `\__spintent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```

3841 \clist_map_inline:nn { side , mside }
3842 {
3843   \str_new:c { \__spintent_sp #1 _mode_str }
3844   \tl_new:c { \__spintent_sp #1 _prefix_tl }
3845   \str_new:c { \__spintent_sp #1 _read_sty_str }
3846   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
3847 }

```

(End of definition for `\__spintent_spside_mode_str` and others.)

11.40.2 Definition of error messages for `\spside` and `\spmside`

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the $\langle argument \rangle$ passed in ‘#1’ to the command.

```

3848 \clist_map_inline:nn { side , mside }
3849 {
3850   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3851   {
3852     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3853   }
3854   {
3855     See~the~documentation~of~##1~for~the~accepted~point~formats.
3856   }
3857 }

```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.40.3 Definition of keys for `\spside` and `\spmside`

We define the shared $\langle keys \rangle$ `read-cmd`, `prefix`, `silent`, `concept`, `read-sty`, `read-arg` and `print-m`.

```

read-cmd 3858 \clist_map_inline:nn { side , mside }
prefix   3859 {
silent   3860   \keys_define:nn { spintent / sp #1 }
concept  3861   {
read-sty 3862     read-cmd .choices:nn = { school , mathcat }
read-arg 3863     {
print-m  3864       \int_case:nn { \_keys_choice_int }
unknown  3865       {
3866         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
3867         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
3868       }
3869     },
3870     read-cmd .initial:n = school,
3871     read-cmd .value_required:n = true,
3872     read-cmd / unknown .code:n =
3873     \msg_error:nneee { spintent } { unknown-choice }
3874     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3875     prefix .code:n =

```

```

3876     { \__spintent_sanitizé_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
3877     prefix                .initial:n = ,
3878     prefix                .value_required:n = true,
3879     silent                .bool_set:c = { \__spintent_sp #1 _silent_bool },
3880     silent                .initial:n = false,
3881     silent                .value_required:n = true,
3882     concept               .bool_set:c = { \__spintent_sp #1 _concept_bool },
3883     concept               .initial:n = true,
3884     concept               .value_required:n = true,
3885     read-sty              .choices:nn = { lado , segmento , trazo }
3886     {
3887         \int_case:nn { \l_keys_choice_int }
3888         {
3889             {1} { \str_set:cn { \__spintent_sp #1 _read_sty_str } { lado } }
3890             {2} { \str_set:cn { \__spintent_sp #1 _read_sty_str } { segmento } }
3891             {3} { \str_set:cn { \__spintent_sp #1 _read_sty_str } { trazo } }
3892         }
3893     },
3894     read-sty              .initial:n = lado,
3895     read-sty              .value_required:n = true,
3896     read-sty / unknown    .code:n =
3897     \msg_error:nneee { spintent } { unknown-choice }
3898     { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3899     read-arg              .code:n =
3900     {
3901         \__spintent_sanitizé_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1}
3902     },
3903     read-arg              .initial:n = ,
3904     read-arg              .value_required:n = true,
3905     print-m               .bool_set:c = { \__spintent_sp #1 _print_m_bool },
3906     print-m               .initial:n = false,
3907     print-m               .value_required:n = true,
3908     unknown               .code:n =
3909     {
3910         \str_set:Nn \__spintent_command_name_str { sp #1 }
3911         \__spintent_unknown_keys_cmd:n {##1}
3912     },
3913 }
3914 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

`\__spintent_spside_intent_tl`
`\__spintent_spside_pt_seq`
`\__spintent_spside_refs_tl`
`\__spintent_spside_intent_arg_tl`
`\__spintent_spside_pt_idx_int`

The variable `\__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\__spintent_geo_luaset_point` and build the `$pts1, $pts2, ...` reference list.

```

3915 \tl_new:N \__spintent_spside_intent_tl
3916 \seq_new:N \__spintent_spside_pt_seq
3917 \tl_new:N \__spintent_spside_refs_tl
3918 \tl_new:N \__spintent_spside_intent_arg_tl
3919 \int_new:N \__spintent_spside_pt_idx_int

```

(End of definition for `\__spintent_spside_intent_tl` and others.)

`\__spintent_spside_process:nn`

The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘#1’ and the `{⟨argument⟩}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then builds `\__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_math` builds after splitting the points.

```

3920 \cs_new_protected:Nn \__spintent_spside_process:nn
3921 {
3922     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3923     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3924     {
3925         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3926         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3927     }
3928     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }

```

```

3929     {
3930         \tl_set:Ne \l__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str } }
3931         \str_if_eq:eeT {#1} { mside }
3932         {
3933             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3934         }
3935     }
3936     {
3937         \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3938         \bool_if:cT { l__spintent_sp #1 _concept_bool }
3939         {
3940             \str_if_eq:eeT {#1} { mside }
3941             {
3942                 \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3943             }
3944         }
3945     }
3946     \__spintent_spside_mathml:n { #1 }
3947 }

```

(End of definition for __spintent_spside_process:nn.)

__spintent_spside_mathml:n The function __spintent_spside_mathml:n receives the short command name in ‘#1’. It splits \l__spintent_geo_luaset_points_clist into \l__spintent_spside_pt_seq and walks it once to build \l__spintent_spside_refs_tl, the comma-separated \$pts1,\$pts2,... reference list, naming each point by position.

```

3948 \cs_new_protected:Nn \__spintent_spside_mathml:n
3949 {
3950     \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
3951     \tl_clear:N \l__spintent_spside_refs_tl
3952     \int_zero:N \l__spintent_spside_pt_idx_int
3953     \seq_map_inline:Nn \l__spintent_spside_pt_seq
3954     {
3955         \int_incr:N \l__spintent_spside_pt_idx_int
3956         \tl_if_empty:NF \l__spintent_spside_refs_tl
3957         { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3958         \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int }
3959     }

```

The concept `intent` is `_-` when `silent` is `true`; otherwise, if `read-arg` is empty, it is `_ (refs)` when `concept` is `false` or `_ \l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `_ \l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see __spintent_spside_process:nn). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3960     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
3961     {
3962         \tl_set:Nn \l__spintent_spside_intent_arg_tl { _- }
3963     }
3964     {
3965         \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3966         {
3967             \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3968             {
3969                 \tl_set:Ne \l__spintent_spside_intent_arg_tl
3970                 {
3971                     -
3972                     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3973                     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3974                     \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3975                 }
3976             }
3977             {
3978                 \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3979             }
3980         }
3981         {
3982             \tl_set:Ne \l__spintent_spside_intent_arg_tl
3983             {
3984                 -
3985                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3986                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }

```

```

3987         \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3988     }
3989 }
3990 }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_inner_sep`: (a second sibling so the sole `<mrow>` does not collapse into `<math>`. Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `pts`/`n` reference name and `silent/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3991     \__spintent_invisible_sep: % (luamml issues #26)
3992     \MathMLintent { \l__spintent_spside_intent_arg_tl }
3993     {
3994         \__spintent_inner_sep:
3995         \str_if_eq:eeTF {#1} { side }
3996         {
3997             \overline { \l__spintent_spside_points_loop:n {#1} }
3998         }
3999         {
4000             \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
4001             {
4002                 \__spintent_luafun_mrow_intent_arg:nn
4003                 { } { intent = ":silent" }
4004                 { \operatorname { m } }
4005                 \overline { \l__spintent_spside_points_loop:n {#1} }
4006             }
4007             {
4008                 \l__spintent_spside_points_loop:n {#1}
4009             }
4010         }
4011     }
4012 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n`

Walks `\l__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in `#1` again since a separate macro does not inherit its caller's own `#1`.

```

4013 \cs_new_protected:Nn \__spintent_spside_points_loop:n
4014 {
4015     \int_zero:N \l__spintent_spside_pt_idx_int
4016     \seq_map_inline:Nn \l__spintent_spside_pt_seq
4017     {
4018         \int_incr:N \l__spintent_spside_pt_idx_int
4019         \__spintent_mathmlarg_point:nnn
4020         { pts \int_use:N \l__spintent_spside_pt_idx_int }
4021         { sp #1 }
4022         { ##1 }
4023     }
4024 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn`

The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in `#1`, the optional `<keys>` passed in `#2` and the `<argument>` passed in `#3`, delegating the rendering to `\__spintent_spside_process:nn`.

```

4025 \cs_new_protected:Nn \__spintent_spside_exec:nnn
4026 {
4027     \keys_set:nn { spintent / sp #1 } { #2 }
4028     \__spintent_spside_process:nn { #1 } { #3 }
4029 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside`
`\spmside`

Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

4030 \NewDocumentCommand \spside { 0{} m }

```

```

4031 {
4032   \mode_if_math:TF
4033   {
4034     \group_begin:
4035       \attribute \l__spintent_scope_attr_int = 1
4036       \__spintent_spside_exec:nnn { side } { #1 } { #2 }
4037     \group_end:
4038   }
4039   { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
4040 }
4041 \NewDocumentCommand \spmside { 0{} m }
4042 {
4043   \mode_if_math:TF
4044   {
4045     \group_begin:
4046       \attribute \l__spintent_scope_attr_int = 1
4047       \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
4048     \group_end:
4049   }
4050   { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
4051 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 21.)

11.41 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.41.1 Definition of variables for `\spangle` and `\spmangle`

```

\l__spintent_spangle_mode_str
\l__spintent_spmangle_mode_str
\l__spintent_spangle_prefix_tl
\l__spintent_spmangle_prefix_tl

```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\l__spintent_spangle_mode_str` and `\l__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spangle_prefix_tl` and `\l__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

4052 \clist_map_inline:nn { angle , mangle }
4053 {
4054   \str_new:c { l__spintent_sp #1 _mode_str }
4055   \tl_new:c { l__spintent_sp #1 _prefix_tl }
4056 }

```

(End of definition for `\l__spintent_spangle_mode_str` and others.)

11.41.2 Definition of error messages for `\spangle` and `\spmangle`

```

spangle-invalid-arg
spmangle-invalid-arg

```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```

4057 \clist_map_inline:nn { angle , mangle }
4058 {
4059   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
4060   {
4061     Invalid~format~in~##1:~'##2'~\msg_line_context:.
4062   }
4063   {
4064     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4065   }
4066 }

```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.41.3 Definition of keys for `\spangle` and `\spmangle`

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `recto`, `print-m`, `silent` and `concept`,

```

4067 \clist_map_inline:nn { angle , mangle }
4068 {
4069   \keys_define:nn { spintent / sp #1 }
4070   {
4071     read-cmd      .choices:nn = { school , mathcat }
4072     {
4073       \int_case:nn { \l_keys_choice_int }

```

```

4074         {
4075             {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4076             {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4077         }
4078     },
4079     read-cmd          .initial:n = school,
4080     read-cmd          .value_required:n = true,
4081     read-cmd / unknown .code:n =
4082         \msg_error:nneee { spintent } { unknown-choice }
4083         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4084     prefix            .code:n =
4085         { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4086     prefix            .initial:n = ,
4087     prefix            .value_required:n = true,
4088     recto              .bool_set:c = { l__spintent_sp #1 _recto_bool },
4089     recto              .initial:n = false,
4090     recto              .value_required:n = true,
4091     print-m            .bool_set:c = { l__spintent_sp #1 _print_m_bool },
4092     print-m            .initial:n = false,
4093     print-m            .value_required:n = true,
4094     silent             .bool_set:c = { l__spintent_sp #1 _silent_bool },
4095     silent             .initial:n = false,
4096     silent             .value_required:n = true,
4097     concept            .bool_set:c = { l__spintent_sp #1 _concept_bool },
4098     concept            .initial:n = true,
4099     concept            .value_required:n = true,
4100     unknown            .code:n =
4101         {
4102             \str_set:Nn \l__spintent_command_name_str { sp #1 }
4103             \__spintent_unknown_keys_cmd:n {##1}
4104         },
4105     }
4106 }

```

🔗 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.41.4 Internal functions for `\spangle` and `\spmangle`

`\__spintent_spangle_sym:n` The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

4107 \cs_new_protected:Nn \__spintent_spangle_sym:n
4108 {
4109     \str_if_eq:eeTF {#1} { mangle }
4110     {
4111         \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
4112         {
4113             \bool_if:cTF { l__spintent_sp #1 _recto_bool }
4114             { \__spintent_right_angle_sqr: } { \angle }
4115         }
4116         {
4117             \bool_if:cTF { l__spintent_sp #1 _recto_bool }
4118             { \measuredrightangle } { \measuredangle }
4119         }
4120     }
4121     {
4122         \bool_if:cTF { l__spintent_sp #1 _recto_bool }
4123         { \__spintent_right_angle_sqr: } { \angle }
4124     }
4125 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\l__spintent_spangle_intent_tl` The variable `\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_luaset_points_clist` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:n`).

```

4126 \tl_new:N \l__spintent_spangle_intent_tl
4127 \seq_new:N \l__spintent_spangle_pt_seq
4128 \tl_new:N \l__spintent_spangle_refs_tl
4129 \tl_new:N \l__spintent_spangle_intent_arg_tl

```

```

4130 \int_new:N \l__spintent_spangle_pt_idx_int
4131 \str_new:N \l__spintent_spangle_kv_tl
4132 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\l__spintent_spangle_intent_tl` and others.)

`\__spintent_spangle_process:nn`

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘#1’ and the `{⟨argument⟩}` in ‘#2’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\l__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is `true`, and calls `\__spintent_spangle_mathml:n`.

```

4133 \cs_new_protected:Nn \__spintent_spangle_process:nn
4134 {
4135   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
4136   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4137   {
4138     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4139     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4140   }
4141   \str_if_eq:VnT \l__spintent_geo_luaset_angulo_case_str { nombre }
4142   {
4143     \str_if_eq:VnF \l__spintent_geo_luaset_nombre_es_num_str { true }
4144     {
4145       \int_compare:nT { \str_count:N \l__spintent_geo_luaset_letra_str = 1 }
4146       {
4147         \str_if_eq:eeF { \l__spintent_geo_luaset_letra_str } { \str_uppercase:n { \l__spint
4148         {
4149           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4150           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4151         }
4152       }
4153     }
4154   }
4155   \str_if_eq:eeTF {#1} { mangle }
4156   {
4157     \tl_set:Nn \l__spintent_spangle_intent_tl { medida-del-ángulo }
4158   }
4159   {
4160     \tl_set:Nn \l__spintent_spangle_intent_tl { ángulo }
4161   }
4162   \bool_if:cT { \l__spintent_sp #1 _recto_bool }
4163   {
4164     \tl_put_right:Nn \l__spintent_spangle_intent_tl { -recto }
4165   }
4166   \__spintent_spangle_mathml:n {#1}
4167 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n`

The function `\__spintent_spangle_mathml:n` branches on `\l__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\__spintent_spangle_sym:n`) is tagged through `\__spintent_luafun_mrow_intent_ar` on its own, with `\l__spintent_spangle_intent_tl` alone (bare, no `$`-reference: `nombre` never refers to more than one thing) — `-` when `silent` is `true` or `concept` is `false`. With `silent` `true`, the name that follows is silenced as a whole.

```

4168 \cs_new_protected:Nn \__spintent_spangle_mathml:n
4169 {
4170   \__spintent_invisible_sep: % (luamml issues #26)
4171   \str_if_eq:VnTF \l__spintent_geo_luaset_angulo_case_str { nombre }
4172   {
4173     \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4174     {
4175       \tl_set:Nn \l__spintent_spangle_intent_arg_tl { - }
4176     }
4177     {
4178       \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4179       {
4180         \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4181         {
4182           -

```

```

4183         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4184         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4185         \l__spintent_spangle_intent_tl
4186     }
4187 }
4188 {
4189     \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4190 }
4191 }
4192 \tl_set:Ne \l__spintent_spangle_kv_tl
4193 { intent = "\l__spintent_spangle_intent_arg_tl" }
4194 \__spintent_luafun_mrow_intent_arg:nn
4195 { } { \l__spintent_spangle_kv_tl }
4196 { \__spintent_spangle_sym:n {#1} }
4197 \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4198 {
4199     \__spintent_luafun_mrow_intent_arg:nn { } { intent = "_-" }
4200     { \l__spintent_geo_luaset_print_tl }
4201 }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

4202 {
4203     \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
4204     {
4205         \tl_set:Ne \l__spintent_spangle_kv_tl
4206         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4207         \__spintent_luafun_mrow_intent_arg:nn
4208         { } { \l__spintent_spangle_kv_tl }
4209         { \l__spintent_geo_luaset_print_tl }
4210     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

4211 {
4212     \str_case:VnF \l__spintent_geo_luaset_tipo_str
4213     {
4214         { sub }
4215         {
4216             \str_if_eq:eeTF { \use:c { l__spintent_sp #1 _mode_str } } { school }
4217             {
4218                 \tl_set:Ne \l__spintent_spangle_kv_tl
4219                 { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4220                 \__spintent_luafun_mrow_intent_arg:nn
4221                 { } { \l__spintent_spangle_kv_tl }
4222                 {
4223                     \l__spintent_geo_luaset_letra_str
4224                     \c_math_subscript_token
4225                     { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4226                 }
4227             }
4228             {
4229                 \l__spintent_geo_luaset_letra_str
4230                 \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4231             }
4232         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

4233 { prima }
4234 {
4235     \str_if_eq:eeTF { \use:c { l__spintent_sp #1 _mode_str } } { school }
4236     {
4237         \tl_set:Ne \l__spintent_spangle_kv_tl
4238         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4239         \__spintent_luafun_mrow_intent_arg:nn
4240         { } { \l__spintent_spangle_kv_tl }
4241         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4242     }

```

```

4243             {
4244             \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
4245             }
4246         }
4247     }
4248     { \l__spintent_geo_luaset_print_tl }
4249 }
4250 }
4251 }

```

For tres-puntos/un-punto, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt_seq` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(ref - _ -` when `silent` is `true`, `_(refs)` when `concept` is `false` alone. For `\spmangle` with `print-m true`, a silent `\operatorname{m}` precedes `\l__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\l__spintent_mathmlarg_point:nnn` once per point.

```

4252     {
4253     \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
4254     \tl_clear:N \l__spintent_spangle_refs_tl
4255     \int_zero:N \l__spintent_spangle_pt_idx_int
4256     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4257     {
4258     \int_incr:N \l__spintent_spangle_pt_idx_int
4259     \tl_if_empty:NF \l__spintent_spangle_refs_tl
4260     { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
4261     \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_pt_idx_int }
4262     }
4263     \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4264     {
4265     \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4266     }
4267     {
4268     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4269     {
4270     \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4271     {
4272     -
4273     \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4274     { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4275     \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
4276     }
4277     }
4278     {
4279     \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _ ( \l__spintent_spangle_refs_tl )
4280     }
4281     }
4282     \bool_set_false:N \l__spintent_spangle_print_m_now_bool
4283     \str_if_eq:eeT {#1} { mangle }
4284     {
4285     \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
4286     { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
4287     }
4288     \MathMLintent { \l__spintent_spangle_intent_arg_tl }
4289     {
4290     \__spintent_inner_sep:
4291     \bool_if:NT \l__spintent_spangle_print_m_now_bool
4292     {
4293     \__spintent_luafun_mrow_intent_arg:nn
4294     { } { intent = ":silent" }
4295     { \operatorname { m } } }
4296     }
4297     \__spintent_luafun_mrow_intent_arg:nn
4298     { } { intent = ":silent" }
4299     { \__spintent_spangle_sym:n {#1} }
4300     \int_zero:N \l__spintent_spangle_pt_idx_int
4301     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4302     {
4303     \int_incr:N \l__spintent_spangle_pt_idx_int
4304     \__spintent_mathmlarg_point:nnn
4305     { pt \int_use:N \l__spintent_spangle_pt_idx_int }
4306     { sp#1 }

```

```

4307         { ##1 }
4308     }
4309 }
4310 }
4311 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn` The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

4312 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
4313 {
4314     \keys_set:nn { spintent / sp #1 } { #2 }
4315     \__spintent_spangle_process:nn { #1 } { #3 }
4316 }

```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle` Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

`\spmangle`

```

4317 \NewDocumentCommand \spangle { O{} m }
4318 {
4319     \mode_if_math:TF
4320     {
4321         \group_begin:
4322         \attribute \__spintent_scope_attr_int = 1
4323         \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4324         \group_end:
4325     }
4326     { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
4327 }
4328 \NewDocumentCommand \spmangle { O{} m }
4329 {
4330     \mode_if_math:TF
4331     {
4332         \group_begin:
4333         \attribute \__spintent_scope_attr_int = 1
4334         \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
4335         \group_end:
4336     }
4337     { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4338 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 23.)

11.42 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.42.1 Definition of variables for `\sparc` and `\spmarc`

`\__spintent_sparc_mode_str` Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```

4339 \clist_map_inline:nn { arc , marc }
4340 {
4341     \str_new:c { \__spintent_sp #1 _mode_str }
4342     \tl_new:c { \__spintent_sp #1 _prefix_tl }
4343 }

```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.42.2 Definition of error messages for `\sparc` and `\spmarc`

`sparc-invalid-points`
`spmarc-invalid-points`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid $\langle argument \rangle$ passed in ‘#1’ to the command.

```

4344 \clist_map_inline:nn { arc , marc }
4345 {
4346   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4347   {
4348     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4349   }
4350   {
4351     See~the~documentation~of~##1~for~the~accepted~point~formats.
4352   }
4353 }
```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.42.3 Definition of keys for `\sparc` and `\spmarc`

`read-cmd`
`prefix`
`silent`
`concept`
`unknown`

We define the shared $\langle keys \rangle$ `read-cmd`, `prefix`, `silent` and `concept`.

```

4354 \clist_map_inline:nn { arc , marc }
4355 {
4356   \keys_define:nn { spintent / sp #1 }
4357   {
4358     read-cmd .choices:nn = { school , mathcat }
4359     {
4360       \int_case:nn { \l_keys_choice_int }
4361       {
4362         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4363         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4364       }
4365     },
4366     read-cmd .initial:n = school,
4367     read-cmd .value_required:n = true,
4368     read-cmd / unknown .code:n =
4369       \msg_error:nneee { spintent } { unknown-choice }
4370       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4371     prefix .code:n =
4372       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4373     prefix .initial:n = ,
4374     prefix .value_required:n = true,
4375     silent .bool_set:c = { l__spintent_sp #1 _silent_bool },
4376     silent .initial:n = false,
4377     silent .value_required:n = true,
4378     concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
4379     concept .initial:n = true,
4380     concept .value_required:n = true,
4381     unknown .code:n =
4382     {
4383       \str_set:Nn \l__spintent_command_name_str { sp #1 }
4384       \__spintent_unknown_keys_cmd:n {##1}
4385     },
4386   }
4387 }
```

(End of definition for `read-cmd` and others.)

11.42.4 Internal functions for `\sparc` and `\spmarc`

`\__spintent_sparc_wide_arc:n`

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```

4388 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4389 {
4390   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4391 }
```

(End of definition for `\__spintent_sparc_wide_arc:n`.)

`\__spintent_sparc_print_arc:n`

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4392 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4393 {
```

```

4394     \cs_if_exist:NTF \widearc
4395     { \widearc {#1} }
4396     { \__spintent_sparc_wide_arc:n {#1} }
4397 }

```

(End of definition for __spintent_sparc_print_arc:n.)

```

\__spintent_sparc_intent_tl
\__spintent_sparc_pt_seq
\__spintent_sparc_refs_tl
\__spintent_sparc_intent_arg_tl
\__spintent_sparc_pt_idx_int

```

The variable __spintent_sparc_intent_tl holds the concept word (arco or medida-del-arco). The rest are scratch variables to split __spintent_geo_luaset_points_clist and build the \$pt1, \$pt2, ... reference list used in the concept intent.

```

4398 \tl_new:N \__spintent_sparc_intent_tl
4399 \seq_new:N \__spintent_sparc_pt_seq
4400 \tl_new:N \__spintent_sparc_refs_tl
4401 \tl_new:N \__spintent_sparc_intent_arg_tl
4402 \int_new:N \__spintent_sparc_pt_idx_int

```

(End of definition for __spintent_sparc_intent_tl and others.)

```
\__spintent_sparc_process:nn
```

The internal function __spintent_sparc_process:nn receives the short name of the command (arc or marc) in ‘#1’ and the {<argument>} in ‘#2’. It calls __spintent_luafun_geo_parse_and_set:n to process ‘#2’, returning an “error” if invalid, then sets __spintent_sparc_intent_tl to arco (or medida-del-arco for \spmarc). No intent is built here: it needs the reference list, which __spintent_sparc_mathml builds after splitting the points.

```

4403 \cs_new_protected:Nn \__spintent_sparc_process:nn
4404 {
4405     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4406     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4407     {
4408         \msg_error:nnee { spintent } { sp #1 -invalid-points }
4409         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4410     }
4411     \str_if_eq:eeTF {#1} { marc }
4412     { \tl_set:Nn \__spintent_sparc_intent_tl { medida-del-arco } }
4413     { \tl_set:Nn \__spintent_sparc_intent_tl { arco } }
4414     \__spintent_sparc_mathml:n {#1}
4415 }

```

(End of definition for __spintent_sparc_process:nn.)

```
\__spintent_sparc_mathml:n
```

The function __spintent_sparc_mathml:n receives the short command name in ‘#1’. It splits __spintent_geo_luaset_points_clist into __spintent_sparc_pt_seq and walks it once to build __spintent_sparc_refs_tl, the comma-separated \$pt1, \$pt2, ... reference list, naming each point by position.

```

4416 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4417 {
4418     \seq_set_from_clist:NN \__spintent_sparc_pt_seq \__spintent_geo_luaset_points_clist
4419
4420     \tl_clear:N \__spintent_sparc_refs_tl
4421     \int_zero:N \__spintent_sparc_pt_idx_int
4422     \seq_map_inline:Nn \__spintent_sparc_pt_seq
4423     {
4424         \int_incr:N \__spintent_sparc_pt_idx_int
4425         \tl_if_empty:NF \__spintent_sparc_refs_tl
4426         { \tl_put_right:Nn \__spintent_sparc_refs_tl { , } }
4427         \tl_put_right:Ne \__spintent_sparc_refs_tl { $ pt \int_use:N \__spintent_sparc_pt_idx_int }
4428     }

```

The concept intent is _- when silent is true, _ (refs) when concept is false; otherwise ___spintent_sparc_ with prefix placed after the leading _ when given.

```

4429     \bool_if:cTF { \__spintent_sp #1 _silent_bool }
4430     {
4431         \tl_set:Nn \__spintent_sparc_intent_arg_tl { _- }
4432     }
4433     {
4434         \bool_if:cTF { \__spintent_sp #1 _concept_bool }
4435         {
4436             \tl_set:Ne \__spintent_sparc_intent_arg_tl
4437             {
4438                 -
4439                 \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
4440                 { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
4441                 \__spintent_sparc_intent_tl :prefix( \__spintent_sparc_refs_tl )

```

```

4442         }
4443     }
4444     {
4445         \tl_set:Nx \l__spintent_sparc_intent_arg_tl { _( \l__spintent_sparc_refs_tl ) }
4446     }
4447 }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, luamml issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4448     \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4449     \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4450     {
4451         \__spintent_inner_sep:
4452         \str_if_eq:eeT {#1} { marc }
4453         {
4454             \__spintent_luafun_mrow_intent_arg:nn
4455             { } { intent = ":silent" }
4456             { \operatorname { m } } }
4457     }
4458     \__spintent_sparc_print_arc:n
4459     {
4460         \int_zero:N \l__spintent_sparc_pt_idx_int
4461         \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4462         {
4463             \int_incr:N \l__spintent_sparc_pt_idx_int
4464             \__spintent_mathmlarg_point:nnn
4465             { pt \int_use:N \l__spintent_sparc_pt_idx_int }
4466             { sp #1 }
4467             { ##1 }
4468         }
4469     }
4470 }
4471 }

```

(End of definition for `\__spintent_sparc_mathml:n`.)

`\__spintent_sparc_exec:nnn`

The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *<keys>* passed in ‘#2’ and the *{<argument>}* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nnn`.

```

4472 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4473 {
4474     \keys_set:nn { spintent / sp #1 } { #2 }
4475     \__spintent_sparc_process:nn { #1 } { #3 }
4476 }

```

(End of definition for `\__spintent_sparc_exec:nnn`.)

`\sparc`
`\spmarc`

Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

4477 \NewDocumentCommand \sparc { O{} m }
4478 {
4479     \mode_if_math:TF
4480     {
4481         \group_begin:
4482         \attribute \l__spintent_scope_attr_int = 1
4483         \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4484         \group_end:
4485     }
4486     { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4487 }
4488 \NewDocumentCommand \spmarc { O{} m }
4489 {
4490     \mode_if_math:TF
4491     {
4492         \group_begin:
4493         \attribute \l__spintent_scope_attr_int = 1
4494         \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4495         \group_end:

```

```

4496     }
4497     { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4498   }

```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 25.)

11.43 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\l__spintent_geo_luaset_intent_str` (the full `intent`, with the figure name).

11.43.1 Definition of variables for `\spPoly`

The variable `\l__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

4499 \str_new:N \l__spintent_spPoly_mode_str
4500 \tl_new:N \l__spintent_spPoly_prefix_tl

```

(End of definition for `\l__spintent_spPoly_mode_str` and `\l__spintent_spPoly_prefix_tl`.)

11.43.2 Definition of error messages for `\spPoly`

`spPoly-invalid-points` Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

4501 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4502 {
4503   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
4504 }
4505 {
4506   See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4507 }

```

(End of definition for `spPoly-invalid-points`.)

11.43.3 Definition of keys for `\spPoly`

We define the *(keys)* `read-cmd`, `prefix`, `print-sym`, `silent` and `concept`.

```

4508 \keys_define:nn { spintent / spPoly }
4509 {
4510   read-cmd .choices:nn = { school , mathcat }
4511   {
4512     \int_case:nn { \l_keys_choice_int }
4513     {
4514       {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
4515       {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
4516     }
4517   },
4518   read-cmd .initial:n = school,
4519   read-cmd .value_required:n = true,
4520   read-cmd / unknown .code:n =
4521     \msg_error:nnee { spintent } { unknown-choice }
4522     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4523   prefix .code:n =
4524     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4525   prefix .initial:n = ,
4526   prefix .value_required:n = true,
4527   print-sym .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4528   print-sym .initial:n = true,
4529   print-sym .value_required:n = true,
4530   silent .bool_set:N = \l__spintent_spPoly_silent_bool,
4531   silent .initial:n = false,
4532   silent .value_required:n = true,
4533   concept .bool_set:N = \l__spintent_spPoly_concept_bool,
4534   concept .initial:n = true,
4535   concept .value_required:n = true,
4536   unknown .code:n =
4537     {
4538       \str_set:Nn \l__spintent_command_name_str { spPoly }
4539       \__spintent_unknown_keys_cmd:n {#1}
4540     },
4541 }

```

(End of definition for `read-cmd` and others.)

11.43.4 Internal functions for \spPoly

Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1,\$pt2,... reference list, and to hold the final concept intent once built.

```

4542 \seq_new:N \l__spintent_spPoly_pt_seq
4543 \tl_new:N \l__spintent_spPoly_refs_tl
4544 \tl_new:N \l__spintent_spPoly_intent_arg_tl
4545 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym: Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

4546 \cs_new_protected:Nn \__spintent_spPoly_sym:
4547 {
4548   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4549   {
4550     { triangle } { \triangle }
4551     { square } { \mdlgwhtsquare }
4552   }
4553 }

```

(End of definition for __spintent_spPoly_sym:.)

__spintent_spPoly_build_refs: Splits \l__spintent_geo_luaset_points_clist into \l__spintent_spPoly_pt_seq and walks it once to build \l__spintent_spPoly_refs_tl, the comma-separated \$pt1,\$pt2,... reference list, naming each vertex by position.

```

4554 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4555 {
4556   \seq_set_from_clist:NN \l__spintent_spPoly_pt_seq \l__spintent_geo_luaset_points_clist
4557   \tl_clear:N \l__spintent_spPoly_refs_tl
4558   \int_zero:N \l__spintent_spPoly_pt_idx_int
4559   \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
4560   {
4561     \int_incr:N \l__spintent_spPoly_pt_idx_int
4562     \tl_if_empty:NF \l__spintent_spPoly_refs_tl
4563     { \tl_put_right:Nn \l__spintent_spPoly_refs_tl { , } }
4564     \tl_put_right:Ne \l__spintent_spPoly_refs_tl { $ pt \int_use:N \l__spintent_spPoly_pt_idx.
4565   }
4566 }

```

(End of definition for __spintent_spPoly_build_refs:.)

__spintent_spPoly_build_concept: Builds \l__spintent_spPoly_intent_arg_tl, independently of print-sym: _- when silent is true, _ (refs) when concept is false; otherwise \l__spintent_geo_luaset_concept_str:prefix(refs), with prefix placed after the leading _ when given. Must be called after __spintent_spPoly_build_refs:.

```

4567 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
4568 {
4569   \bool_if:NTF \l__spintent_spPoly_silent_bool
4570   {
4571     \tl_set:Nn \l__spintent_spPoly_intent_arg_tl { _- }
4572   }
4573   {
4574     \bool_if:NTF \l__spintent_spPoly_concept_bool
4575     {
4576       \tl_set:Ne \l__spintent_spPoly_intent_arg_tl
4577       {
4578         -
4579         \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
4580         { \tl_use:N \l__spintent_spPoly_prefix_tl - }
4581         \l__spintent_geo_luaset_concept_str :prefix( \l__spintent_spPoly_refs_tl )
4582       }
4583     }
4584     {
4585       \tl_set:Ne \l__spintent_spPoly_intent_arg_tl { _( \l__spintent_spPoly_refs_tl ) }
4586     }
4587   }
4588 }

```

(End of definition for __spintent_spPoly_build_concept:.)

`\__spintent_spPoly_points_loop:` Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `pt` *n* reference name, looked up under `spPoly`: it alone decides `arg`/literal `intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

4589 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
4590 {
4591   \int_zero:N \__spintent_spPoly_pt_idx_int
4592   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4593   {
4594     \int_incr:N \__spintent_spPoly_pt_idx_int
4595     \__spintent_mathmlarg_point:nnn
4596     { pt \int_use:N \__spintent_spPoly_pt_idx_int }
4597     { spPoly }
4598     { ##1 }
4599   }
4600 }
```

(End of definition for `\__spintent_spPoly_points_loop:.`)

`\__spintent_spPoly_process:n` Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:.` No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4601 \cs_new_protected:Nn \__spintent_spPoly_process:n
4602 {
4603   \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {##1} }
4604   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4605   {
4606     \msg_error:nne { spintent } { spPoly-invalid-points }
4607     { \exp_not:n {##1} }
4608   }
4609   \__spintent_spPoly_mathml:
4610 }
```

(End of definition for `\__spintent_spPoly_process:n`.)

`\__spintent_spPoly_mathml:` Builds the reference list and the concept `intent` (`\__spintent_spPoly_build_refs:`, `\__spintent_spPoly_build_refs:` then wraps everything in a single native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).

The keys `print-sym` and `concept/silent` are independent: the former only decides whether the glyph (`\__spintent_spPoly_sym:`, silenced with `\__spintent_luafun_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\__spintent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4611 \cs_new_protected:Nn \__spintent_spPoly_mathml:
4612 {
4613   \__spintent_invisible_sep: % (luamml issues #26)
4614   \__spintent_spPoly_build_refs:
4615   \__spintent_spPoly_build_concept:
4616   \MathMLintent { \__spintent_spPoly_intent_arg_tl }
4617   {
4618     \__spintent_inner_sep:
4619     \bool_if:NT \__spintent_spPoly_print_sym_bool
4620     {
4621       \str_if_empty:NF \__spintent_geo_luaset_poly_sym_str
4622       {
4623         \__spintent_luafun_mrow_intent_arg:nn
4624         { } { intent = ":silent" }
4625         { \__spintent_spPoly_sym: }
4626       }
4627     }
4628     \__spintent_spPoly_points_loop:
4629   }
4630 }
```

(End of definition for `\__spintent_spPoly_mathml:.`)

`\__spintent_spPoly_exec:nn` The function `\__spintent_spPoly_exec:nn` receives the optional keys in `#1` and the argument in `#2`, delegating the rendering to `\__spintent_spPoly_process:n`.

```

4631 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
4632 {
4633   \keys_set:nn { spintent / spPoly } { #1 }
4634   \__spintent_spPoly_process:n { #2 }
4635 }
```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly` The public command `\spPoly` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoly_exec:nn`.

```

4636 \NewDocumentCommand \spPoly { O{} m }
4637 {
4638   \mode_if_math:TF
4639   {
4640     \group_begin:
4641       \attribute \__spintent_scope_attr_int = 1
4642       \__spintent_spPoly_exec:nn { #1 } { #2 }
4643     \group_end:
4644   }
4645   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
4646 }

```

(End of definition for `\spPoly`. This function is documented on page 26.)

11.44 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{[:silent]}`.

11.44.1 Definition of variables for `\spAfig` and `\spPfig`

`\__spintent_geo_final_print_tl`

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. The variable `\__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```

4647 \tl_new:N \__spintent_geo_final_print_tl

```

(End of definition for `\__spintent_geo_final_print_tl`.)

`\__spintent_spAfig_mode_str`

`\__spintent_spPfig_mode_str`

`\__spintent_spAfig_print_sym_str`

`\__spintent_spPfig_print_sym_str`

`\__spintent_spAfig_prefix_tl`

`\__spintent_spPfig_prefix_tl`

`\__spintent_spAfig_read_arg_tl`

`\__spintent_spPfig_read_arg_tl`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```

4648 \clist_map_inline:nn { Afig , Pfig }
4649 {
4650   \str_new:c { \__spintent_sp #1 _mode_str }
4651   \str_new:c { \__spintent_sp #1 _print_sym_str }
4652   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4653   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
4654 }

```

(End of definition for `\__spintent_spAfig_mode_str` and others.)

11.44.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```

4655 \clist_map_inline:nn { Afig , Pfig }
4656 {
4657   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4658   {
4659     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4660   }
4661   {
4662     See~the~documentation~of~##1~for~the~accepted~point~formats.
4663   }
4664   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
4665   {
4666     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
4667   }
4668   {
4669     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4670   }
4671   \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
4672   {
4673     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.

```

```

4674     }
4675     {
4676         See~the~documentation~of~##1~for~details~on~print-sym.
4677     }
4678     \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
4679     {
4680         print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
4681     }
4682     {
4683         See~the~documentation~of~##1~for~details~on~print-sym.
4684     }
4685     \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
4686     {
4687         read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
4688     }
4689     {
4690         See~the~documentation~of~##1~for~details~on~read-arg.
4691     }
4692 }

```

(End of definition for spAfig-invalid-points and others.)

11.44.3 Definition of keys for \spAfig and \spPfig

read-cmd The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle`/square force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle`/square nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

4693 \clist_map_inline:nn { Afig , Pfig }
4694 {
4695     \keys_define:nn { spintent / sp #1 }
4696     {
4697         read-cmd .choices:nn = { school , mathcat }
4698         {
4699             \int_case:nn { \l_keys_choice_int }
4700             {
4701                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4702                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4703             }
4704         },
4705         read-cmd .initial:n = school,
4706         read-cmd .value_required:n = true,
4707         read-cmd / unknown .code:n =
4708             \msg_error:nneee { spintent } { unknown-choice }
4709             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4710         prefix .code:n =
4711             { \__spintent_sanitizize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4712         prefix .initial:n = ,
4713         prefix .value_required:n = true,
4714         print-sym .choices:nn = { auto , triangle , square , none }
4715         {
4716             \int_case:nn { \l_keys_choice_int }
4717             {
4718                 {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
4719                 {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
4720                 {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
4721                 {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
4722             }
4723         },
4724         print-sym .initial:n = auto,
4725         print-sym .value_required:n = true,
4726         print-sym / unknown .code:n =
4727             \msg_error:nneee { spintent } { unknown-choice }
4728             { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
4729         read-arg .code:n =
4730             { \__spintent_sanitizize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4731         read-arg .initial:n = ,
4732         read-arg .value_required:n = true,
4733         read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },

```

```

4734     read-sub          .initial:n = false,
4735     read-sub          .value_required:n = true,
4736     silent            .bool_set:c = { l__spintent_sp #1 _silent_bool },
4737     silent            .initial:n = false,
4738     silent            .value_required:n = true,
4739     concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
4740     concept           .initial:n = true,
4741     concept           .value_required:n = true,
4742     concept*          .meta:n =
4743     {
4744         concept = false,
4745         read-arg = { \str_if_eq:eeTF {#1} { Afig } { _área } { _perímetro } },
4746     },
4747     concept*          .value_forbidden:n = true,
4748     unknown           .code:n =
4749     {
4750         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4751         \__spintent_unknown_keys_cmd:n {##1}
4752     },
4753 }
4754 }

```

(End of definition for read-cmd and others.)

11.44.4 Internal functions for \spAfig and \spPfig

`\__spintent_spfig_sym:` The function `\__spintent_spfig_sym:` reads `\l__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

4755 \cs_new_protected:Nn \__spintent_spfig_sym:
4756 {
4757     \str_case:Vn \l__spintent_geo_print_sym_str
4758     {
4759         { triangle } { \triangle }
4760         { square } { \mdlgwhtsquare }
4761     }
4762 }

```

(End of definition for `\__spintent_spfig_sym:`.)

`\l__spintent_spfig_op_tl` The variable `\l__spintent_spfig_op_tl` holds `área` or `perímetro`. The variable `\l__spintent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list.

```

\l__spintent_spfig_op_tl
\l__spintent_spfig_intent_tl
\l__spintent_spfig_symbol_tl
\l__spintent_spfig_pt_seq
\l__spintent_spfig_refs_tl
\l__spintent_spfig_intent_arg_tl
\l__spintent_spfig_pt_idx_int
\l__spintent_geo_print_sym_str
4763 \tl_new:N \l__spintent_spfig_op_tl
4764 \tl_new:N \l__spintent_spfig_intent_tl
4765 \tl_new:N \l__spintent_spfig_desc_tl
4766 \tl_new:N \l__spintent_spfig_symbol_tl
4767 \seq_new:N \l__spintent_spfig_pt_seq
4768 \tl_new:N \l__spintent_spfig_refs_tl
4769 \tl_new:N \l__spintent_spfig_intent_arg_tl
4770 \int_new:N \l__spintent_spfig_pt_idx_int
4771 \str_new:N \l__spintent_geo_print_sym_str

```

(End of definition for `\l__spintent_spfig_op_tl` and others.)

`\__spintent_spfig_process:nn` Receives the short command name (`Afig` or `Pfig`) in ‘`#1`’ and the `{\argument}` in ‘`#2`’. Calls `\__spintent_luafun_geo_pfig_parse_and_set:n` or `\__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\l__spintent_spfig_op_tl` to `área/perímetro`, then delegates to `\l__spintent_spfig_mathml:n`.

```

4772 \cs_new_protected:Nn \__spintent_spfig_process:nn
4773 {
4774     \str_if_eq:eeTF {#1} { Afig }
4775     { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4776     { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4777     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4778     {
4779         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4780         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4781     }
4782     \clist_if_empty:NT \l__spintent_geo_luaset_points_clist
4783     {
4784         \tl_if_empty:cF { \l__spintent_sp #1 _read_arg_tl }
4785     }

```

```

4786         \msg_error:nne { spintent } { sp #1 -read-arg-key }
4787         { \c_backslash_str sp #1 }
4788     }
4789 }
4790 \str_if_eq:eeTF {#1} { Afig }
4791 { \tl_set:Nn \l__spintent_spfing_op_tl { área } }
4792 { \tl_set:Nn \l__spintent_spfing_op_tl { perímetro } }
4793 \str_set:Ne \l__spintent_geo_print_sym_str { \str_use:c { l__spintent_sp #1 _print_sym_str } }
4794 \__spintent_spfing_mathml:n {#1}
4795 }

```

(End of definition for `\__spintent_spfing_process:nn`)

`\__spintent_spfing_build_simple:n`

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\l__spintent_geo_final_print_tl` (the visual placed under `\mathcal`'s subscript) and `\l__spintent_spfing_desc_tl` (the descriptive part, without *área*/*perímetro*). Receives the short command name in `'#1'`.

With an empty argument, both are cleared for `print-sym auto/none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

4796 \cs_new_protected:Nn \__spintent_spfing_build_simple:n
4797 {
4798     \tl_if_empty:nTF { \l__spintent_geo_luaset_print_tl }
4799     {
4800         \str_case:VnF \l__spintent_geo_print_sym_str
4801         {
4802             { auto }
4803             {
4804                 \tl_clear:N \l__spintent_geo_final_print_tl
4805                 \tl_clear:N \l__spintent_spfing_desc_tl
4806             }
4807             { none }
4808             {
4809                 \tl_clear:N \l__spintent_geo_final_print_tl
4810                 \tl_clear:N \l__spintent_spfing_desc_tl
4811             }
4812         }
4813         {
4814             \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_spfing_sym: }
4815             \tl_set:Ne \l__spintent_spfing_desc_tl
4816             {
4817                 \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4818                 { triángulo } { cuadrado }
4819             }
4820         }
4821     }

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4822     {
4823         \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
4824         {
4825             \str_case:VnF \l__spintent_geo_print_sym_str
4826             {
4827                 { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl } }
4828                 { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl } }
4829             }
4830             {
4831                 \tl_set:Nn \l__spintent_geo_final_print_tl
4832                 {
4833                     \__spintent_spfing_sym:
4834                     \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4835                 }
4836             }
4837             \str_case:VnF \l__spintent_geo_print_sym_str
4838             {
4839                 { auto }
4840                 {
4841                     \tl_set:Ne \l__spintent_spfing_desc_tl
4842                     {
4843                         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }

```

```

4844         \l__spintent_geo_luaset_sub_spoken_str
4845     }
4846 }
4847 { none }
4848 {
4849     \tl_set:Ne \l__spintent_spfig_desc_tl
4850     {
4851         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4852         \l__spintent_geo_luaset_sub_spoken_str
4853     }
4854 }
4855 }
4856 {
4857     \tl_set:Ne \l__spintent_spfig_desc_tl
4858     {
4859         del-
4860         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4861         { triángulo } { cuadrado }
4862         -
4863         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4864         \l__spintent_geo_luaset_sub_spoken_str
4865     }
4866 }
4867 }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4868     {
4869         \str_case:VnF \l__spintent_geo_print_sym_str
4870         {
4871             { auto } { }
4872             { none } { }
4873         }
4874         {
4875             \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4876             { \c_backslash_str sp #1 }
4877         }
4878         \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
4879         \tl_set:Ne \l__spintent_spfig_desc_tl
4880         {
4881             \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4882             { \l__spintent_geo_luaset_letra_conector_str - }
4883             \l__spintent_geo_luaset_letra_word_str
4884             \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
4885             {
4886                 -
4887                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4888                 \l__spintent_geo_luaset_sub_spoken_str
4889             }
4890         }
4891     }
4892 }

```

The variable `\l__spintent_spfig_intent_tl` is always the word- shape, without the leading `_`: `\l__spintent_spfig_intent_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent` overrides `\l__spintent_spfig_intent_arg_tl` with `_`; with `silent=false` and `concept false`, it becomes `_\l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro*); with `concept true` it is `_`, then `prefix` when given, and then `\l__spintent_spfig_intent_tl`.

```

4893     \tl_set:Ne \l__spintent_spfig_intent_tl
4894     {
4895         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
4896         { \l__spintent_spfig_op_tl - }
4897         { \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
4898     }
4899     \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4900     { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
4901     {
4902         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4903         {

```

```

4904         \tl_set:Nc \l__spintent_spfig_intent_arg_tl
4905         {
4906             -
4907             \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4908             { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4909             \l__spintent_spfig_intent_tl
4910         }
4911     }
4912     { \tl_set:Nc \l__spintent_spfig_intent_arg_tl { _ \l__spintent_spfig_desc_tl - } }
4913 }
4914 }

```

(End of definition for `\l__spintent_spfig_build_simple:n`.)

`\l__spintent_spfig_build_refs:` Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each vertex by position.

```

4915 \cs_new_protected:Nn \l__spintent_spfig_build_refs:
4916 {
4917     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
4918     \tl_clear:N \l__spintent_spfig_refs_tl
4919     \int_zero:N \l__spintent_spfig_pt_idx_int
4920     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4921     {
4922         \int_incr:N \l__spintent_spfig_pt_idx_int
4923         \tl_if_empty:NF \l__spintent_spfig_refs_tl
4924         { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
4925         \tl_put_right:Nc \l__spintent_spfig_refs_tl { $ \pt \int_use:N \l__spintent_spfig_pt_idx_int
4926     }
4927 }

```

(End of definition for `\l__spintent_spfig_build_refs:`.)

`\l__spintent_spfig_build_concept_vertices:n` Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\l__spintent_spfig_build_refs:n`. The key `read-arg`, when given, overrides the auto-detected figure name in `\l__spintent_spfig_intent_tl`: with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim. With `read-arg` empty, `\l__spintent_spfig_intent_tl` is simply `\l__spintent_geo_luaset_concept_`.

```

4928 \cs_new_protected:Nn \l__spintent_spfig_build_concept_vertices:n
4929 {
4930     \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4931     { \tl_set:Nc \l__spintent_spfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
4932     {
4933         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4934         {
4935             \tl_set:Nc \l__spintent_spfig_intent_tl
4936             {
4937                 \l__spintent_spfig_op_tl -del-
4938                 \tl_use:c { \l__spintent_sp #1 _read_arg_tl }
4939             }
4940         }
4941         {
4942             \tl_set:Nc \l__spintent_spfig_intent_tl { \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
4943         }
4944     }

```

Determines `\l__spintent_spfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\l__spintent_geo_luaset_poly_sym_str` (`\triangle/ \mdlgwhtsquare`), or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4945     \str_case:VnF \l__spintent_geo_print_sym_str
4946     {
4947         { auto }
4948         {
4949             \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4950             {
4951                 { triangle } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \triangle } }
4952                 { square } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \mdlgwhtsquare } }
4953             }
4954             \str_if_empty:NT \l__spintent_geo_luaset_poly_sym_str
4955             { \tl_clear:N \l__spintent_spfig_symbol_tl }
4956         }

```

```

4957     { none }
4958     { \tl_clear:N \l__spintent_spfig_symbol_tl }
4959   }
4960   {
4961     \str_if_eq:VVTf \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_str
4962     { \tl_set:Nn \l__spintent_spfig_symbol_tl { \l__spintent_spfig_sym: } }
4963     {
4964       \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
4965       { \c_backslash_str sp #1 }
4966     }
4967   }

```

The key `silent` sets `\l__spintent_spfig_intent_arg_tl` to `_-`. Otherwise, with `read-arg` empty, `concept` chooses between `\l__spintent_spfig_intent_tl:prefix(refs)` (with `prefix` placed after the leading `_` when given) and `_(refs)` alone; with `read-arg` given, it is always `\l__spintent_spfig_intent_tl:— read-arg` is never silenced by `concept` alone, only by `silent`.

```

4968   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4969   {
4970     \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- }
4971   }
4972   {
4973     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
4974     {
4975       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4976       {
4977         \tl_set:Ne \l__spintent_spfig_intent_arg_tl
4978         {
4979           -
4980           \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4981           { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4982           \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4983         }
4984       }
4985       {
4986         \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ ( \l__spintent_spfig_refs_tl ) }
4987       }
4988     }
4989     {
4990       \tl_set:Ne \l__spintent_spfig_intent_arg_tl
4991       {
4992         -
4993         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4994         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4995         \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4996       }
4997     }
4998   }
4999 }

```

(End of definition for `\l__spintent_spfig_build_concept_vertices:n`.)

`\l__spintent_spfig_points_loop:n` Walks `\l__spintent_spfig_pt_seq` a second time, calling `\l__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in `'#1'`.

```

5000 \cs_new_protected:Nn \l__spintent_spfig_points_loop:n
5001 {
5002   \int_zero:N \l__spintent_spfig_pt_idx_int
5003   \seq_map_inline:Nn \l__spintent_spfig_pt_seq
5004   {
5005     \int_incr:N \l__spintent_spfig_pt_idx_int
5006     \l__spintent_mathmlarg_point:nnn
5007     { pt \int_use:N \l__spintent_spfig_pt_idx_int }
5008     { sp #1 }
5009     { ##1 }
5010   }
5011 }

```

(End of definition for `\l__spintent_spfig_points_loop:n`.)

`\l__spintent_spfig_mathml:n` Receives the short command name in `'#1'`. Branches on whether `\l__spintent_geo_luaset_points_clist` is empty.

Without vertices, `\l__spintent_spfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no `intent` at all.

With vertices, `\__spintent_spfig_build_refs:` then `\__spintent_spfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside, `\__spintent_mathcal:n{⟨A | P⟩}` is silenced with `\__spintent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`'s own symbol); `\__spintent_spfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\__spintent_spfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback's recursive lookup through a noad's sub field, not just its nucleus.

The function `\__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

5012 \cs_new_protected:Nn \__spintent_spfig_mathml:n
5013 {
5014   \__spintent_invisible_sep: % (luamml issues #26)
5015   \clist_if_empty:NTF \__spintent_geo_luaset_points_clist
5016   {
5017     \__spintent_spfig_build_simple:n {#1}
5018     \MathMLintent { \__spintent_spfig_intent_arg_tl }
5019     {
5020       \__spintent_inner_sep:
5021       \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5022         \tl_if_empty:NF \__spintent_geo_final_print_tl
5023         {
5024           \c_math_subscript_token { \__spintent_geo_final_print_tl }
5025         }
5026       }
5027     }
5028   }
5029   \__spintent_spfig_build_refs:
5030   \__spintent_spfig_build_concept_vertices:n {#1}
5031   \MathMLintent { \__spintent_spfig_intent_arg_tl }
5032   {
5033     \__spintent_inner_sep:
5034     \__spintent_luafun_mrow_intent_arg:nn
5035     { } { intent = ":silent" }
5036     {
5037       \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:
5038     }
5039     \c_math_subscript_token
5040     {
5041       \tl_if_empty:NF \__spintent_spfig_symbol_tl
5042       {
5043         \__spintent_luafun_mrow_intent_arg:nn
5044         { } { intent = ":silent" }
5045         { \tl_use:N \__spintent_spfig_symbol_tl }
5046       }
5047       \__spintent_spfig_points_loop:n {#1}
5048     }
5049   }
5050 }
5051 }

```

(End of definition for `\__spintent_spfig_mathml:n`.)

`\__spintent_spfig_exec:nnn`

The function `\__spintent_spfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spfig_process:nn`.

```

5052 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
5053 {
5054   \keys_set:nn { spintent / sp #1 } { #2 }
5055   \__spintent_spfig_process:nn { #1 } { #3 }
5056 }

```

(End of definition for `\__spintent_spfig_exec:nnn`.)

`\spAfig`

`\spPfig`

Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\__spintent_spfig_exec:nnn`.

```

5057 \NewDocumentCommand \spAfig { 0{} d() }
5058 {
5059   \mode_if_math:TF
5060   {
5061     \group_begin:

```

```

5062         \attribute \l__spintent_scope_attr_int = 1
5063         \tl_if_novalue:nTF {#2}
5064         { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
5065         { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
5066     \group_end:
5067 }
5068 { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
5069 }
5070 \NewDocumentCommand \spPfig { 0{ } d{ } }
5071 {
5072     \mode_if_math:TF
5073     {
5074         \group_begin:
5075         \attribute \l__spintent_scope_attr_int = 1
5076         \tl_if_novalue:nTF {#2}
5077         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
5078         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
5079     \group_end:
5080     }
5081     { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
5082 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 26.)

11.45 The commands `\sprad` and `\spdiam`

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle.

11.45.1 Definition of variables for `\sprad` and `\spdiam`

The variables `\l__spintent_sp_rad_diam_letra_tl` and `\l__spintent_sp_rad_diam_word_tl` are scratch variables used by `\__spintent_sp_rad_diam_print:n`. `\l__spintent_sp_rad_diam_read_arg_full_tl` is used by `\__spintent_sp_rad_diam_read_arg_print:n` to hold the text to voice when `read-arg` is given.

```

5083 \tl_new:N \l__spintent_sp_rad_diam_letra_tl
5084 \tl_new:N \l__spintent_sp_rad_diam_word_tl
5085 \tl_new:N \l__spintent_sp_rad_diam_read_arg_full_tl

```

(End of definition for `\l__spintent_sp_rad_diam_letra_tl`, `\l__spintent_sp_rad_diam_word_tl`, and `\l__spintent_sp_rad_diam_read_arg_full_tl`.)

11.45.2 Definition of keys for `\sprad` and `\spdiam`

We define the shared *(keys)* `read-cmd`, `upper-case`, `sub-index`, `read-sub`, `read-arg`, `concept`, `silent` and `print-sym`.

```

read-cmd      5086 \clist_map_inline:nn { rad , diam }
upper-case    5087 {
sub-index     5088 { \keys_define:nn { spintent / sp #1 }
read-sub      5089 {
concept       5090     read-cmd      .choices:nn = { school , mathcat }
silent        5091     {
print-sym     5092         \int_case:nn { \l_keys_choice_int }
unknown       5093         {
5094             {1} { \str_set:cn { \l__spintent_sp #1 _mode_str } { school } }
5095             {2} { \str_set:cn { \l__spintent_sp #1 _mode_str } { mathcat } }
5096         }
5097     },
5098     read-cmd      .initial:n = school,
5099     read-cmd      .value_required:n = true,
5100     read-cmd / unknown .code:n =
5101     \msg_error:nneee { spintent } { unknown-choice }
5102     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5103     upper-case    .bool_set:c = { \l__spintent_sp #1 _upper_case_bool },
5104     upper-case    .initial:n = false,
5105     upper-case    .value_required:n = true,
5106     sub-index     .code:n =
5107     { \str_set:cn { \l__spintent_sp #1 _sub_index_str } {##1} },
5108     sub-index     .initial:n = ,
5109     sub-index     .value_required:n = true,
5110     read-sub      .bool_set:c = { \l__spintent_sp #1 _read_sub_bool },
5111     read-sub      .initial:n = false,
5112     read-sub      .value_required:n = true,
5113     read-arg      .code:n =

```

```

5114     { \__spintent_sanitizet_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1} },
5115     read-arg .initial:n = ,
5116     read-arg .value_required:n = true,
5117     concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
5118     concept .initial:n = true,
5119     concept .value_required:n = true,
5120     silent .bool_set:c = { \__spintent_sp #1 _silent_bool },
5121     silent .initial:n = false,
5122     silent .value_required:n = true,
5123     print-sym .bool_set:c = { \__spintent_sp #1 _print_sym_bool },
5124     print-sym .initial:n = false,
5125     print-sym .value_required:n = true,
5126     unknown .code:n =
5127     {
5128         \str_set:Nn \__spintent_command_name_str { sp #1 }
5129         \__spintent_unknown_keys_cmd:n {##1}
5130     },
5131 }
5132 }

```

🔗 The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

11.45.3 Internal functions for `\sprad` and `\spdiam`

`\__spintent_sp_rad_diam_print:n`

The function `\__spintent_sp_rad_diam_print:n` sets `\__spintent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per upper-case, and, for `\spdiam` with `print-sym` true, silently typesets `\diameter` first. Receives the short command name in ‘`#1`’ (rad/diam).

```

5133 \cs_new_protected:Nn \__spintent_sp_rad_diam_print:n
5134 {
5135     \bool_if:cTF { \__spintent_sp #1 _upper_case_bool }
5136     {
5137         \str_if_eq:eeTF {#1} { rad }
5138         { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { R } }
5139         { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { D } }
5140     }
5141     {
5142         \str_if_eq:eeTF {#1} { rad }
5143         { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { r } }
5144         { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { d } }
5145     }
5146     \__spintent_invisible_sep:
5147     \str_if_eq:eeT {#1} { diam }
5148     {
5149         \bool_if:NT \__spintent_spdiam_print_sym_bool
5150         { \MathMLintent { :silent } { \diameter } }
5151     }

```

With `silent` true, the intent is `_-` — the letter alone through the shared queue when there is no sub-index, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

5152     \bool_if:cTF { \__spintent_sp #1 _silent_bool }
5153     {
5154         \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
5155         {
5156             \__spintent_luafun_mrow_intent_arg:nn
5157             { } { intent = "_-" }
5158             { \__spintent_sp_rad_diam_letra_tl }
5159         }
5160         {
5161             \MathMLintent { _- }
5162             {
5163                 \__spintent_inner_sep:
5164                 \__spintent_sp_rad_diam_letra_tl
5165                 \c_math_subscript_token
5166                 {
5167                     \MathMLintent { :silent }
5168                     { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5169                 }
5170             }
5171         }
5172     }

```

Otherwise, with `concept true` (the default), it verbalizes `radio/diámetro` (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing `sub-⟨value⟩` or `⟨value⟩` alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

5173     {
5174       \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5175       {
5176         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5177         {
5178           \str_if_eq:eeTF {#1} { rad }
5179           { \tl_set:Nn \l__spintent_sp_rad_diam_word_tl { radio } }
5180           { \tl_set:Nn \l__spintent_sp_rad_diam_word_tl { diámetro } }
5181         \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
5182         {
5183           \__spintent_luafun_mrow_intent_arg:nn
5184           { } { intent = "_ \l__spintent_sp_rad_diam_word_tl :nofix" }
5185           { \l__spintent_sp_rad_diam_letra_tl }
5186         }
5187         {
5188           \str_if_eq:eeTF
5189           { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
5190           {
5191             \MathMLintent
5192             {
5193               _ \l__spintent_sp_rad_diam_word_tl -
5194               \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5195               \str_use:c { l__spintent_sp #1 _sub_index_str }
5196             }
5197             {
5198               \__spintent_inner_sep:
5199               \l__spintent_sp_rad_diam_letra_tl
5200               \c_math_subscript_token
5201               {
5202                 \MathMLintent { :silent }
5203                 { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5204               }
5205             }
5206           }
5207           {
5208             \__spintent_luafun_mrow_intent_arg:nn
5209             { } { intent = "_ \l__spintent_sp_rad_diam_word_tl :nofix" }
5210             { \l__spintent_sp_rad_diam_letra_tl }
5211             \c_math_subscript_token
5212             { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5213           }
5214         }
5215       }

```

With `concept=false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd=mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by us.

```

5216     {
5217       \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
5218       {
5219         \str_if_eq:eeTF
5220         { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
5221         {
5222           \__spintent_luafun_mrow_intent_arg:nn
5223           { } { intent = "_ \l__spintent_sp_rad_diam_letra_tl -" }
5224           { \l__spintent_sp_rad_diam_letra_tl }
5225         }
5226         { \l__spintent_sp_rad_diam_letra_tl }
5227       }
5228       {
5229         \str_if_eq:eeTF
5230         { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
5231         {
5232           \MathMLintent

```

```

5233         {
5234             _ \l__spintent_sp_rad_diam_letra_tl -
5235             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5236             \str_use:c { l__spintent_sp #1 _sub_index_str }
5237         }
5238         {
5239             \__spintent_inner_sep:
5240             \l__spintent_sp_rad_diam_letra_tl
5241             \c_math_subscript_token
5242             {
5243                 \MathMLintent { :silent }
5244                 { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5245             }
5246         }
5247     }
5248     {
5249         \l__spintent_sp_rad_diam_letra_tl
5250         \c_math_subscript_token
5251         { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5252     }
5253 }
5254 }
5255 }
5256 { \__spintent_sp_rad_diam_read_arg_print:n {#1} }
5257 }
5258 }

```

(End of definition for __spintent_sp_rad_diam_print:n.)

__spintent_sp_rad_diam_read_arg_print:n

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true` (the default), `\l__spintent_sp_rad_diam_read_arg_full_tl` becomes `radio/diámetro` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively, regardless of `read-cmd`: with no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent`.

```

5259 \cs_new_protected:Nn \__spintent_sp_rad_diam_read_arg_print:n
5260 {
5261     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5262     {
5263         \str_if_eq:eeTF {#1} { rad }
5264         {
5265             \tl_set:Nc \l__spintent_sp_rad_diam_read_arg_full_tl
5266             { radio- \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
5267         }
5268         {
5269             \tl_set:Nc \l__spintent_sp_rad_diam_read_arg_full_tl
5270             { diámetro- \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
5271         }
5272     }
5273     {
5274         \tl_set_eq:Nc \l__spintent_sp_rad_diam_read_arg_full_tl
5275         { l__spintent_sp #1 _read_arg_tl }
5276     }
5277     \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
5278     {
5279         \__spintent_luafun_mrow_intent_arg:nn
5280         { } { intent = "_ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix" }
5281         { \l__spintent_sp_rad_diam_letra_tl }
5282     }
5283     {
5284         \MathMLintent
5285         { _ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix }
5286         {
5287             \__spintent_inner_sep:
5288             \l__spintent_sp_rad_diam_letra_tl
5289             \c_math_subscript_token
5290             {
5291                 \MathMLintent { :silent }
5292                 { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5293             }
5294         }
5295     }

```

```
5296 }
```

(End of definition for `\__spintent_sp_rad_diam_read_arg_print:n`.)

```
\__spintent_sp_rad_diam_exec:nn
```

The function `\__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘`#1`’ and the optional (*keys*) passed in ‘`#2`’ then delegating the rendering to `\__spintent_sp_rad_diam_print:n`.

```
5297 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
5298 {
5299   \keys_set:nn { spintent / sp #1 } {#2}
5300   \__spintent_sp_rad_diam_print:n {#1}
5301 }
```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`.)

```
\sprad
\spdiam
```

Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```
5302 \NewDocumentCommand \sprad { 0{} }
5303 {
5304   \mode_if_math:TF
5305   {
5306     \group_begin:
5307       \attribute \__spintent_scope_attr_int = 1
5308       \__spintent_sp_rad_diam_exec:nn { rad } {#1}
5309     \group_end:
5310   }
5311   { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
5312 }
5313 \NewDocumentCommand \spdiam { 0{} }
5314 {
5315   \mode_if_math:TF
5316   {
5317     \group_begin:
5318       \attribute \__spintent_scope_attr_int = 1
5319       \__spintent_sp_rad_diam_exec:nn { diam } {#1}
5320     \group_end:
5321   }
5322   { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
5323 }
```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page 29.)

11.46 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside`/`\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.46.1 Definition of variables for `\spcirc`

```
\__spintent_geo_luaset_circ_radio_type_str
\__spintent_geo_luaset_circ_radio_raw_str
```

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. The variable `\__spintent_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, *segment*, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```
5324 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
5325 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str
```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

```
\__spintent_spcirc_mode_str
\__spintent_spcirc_prefix_tl
\__spintent_spcirc_print_sym_str
\__spintent_spcirc_read_sty_str
\__spintent_spcirc_radio_sty_str
\__spintent_spcirc_sub_index_str
\__spintent_spcirc_articulo_str
```

The variable `\__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (*usual* or *odot*). The variable `\__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (*circun* or *circle*). The variable `\__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to *la* or *el*, matching the grammatical gender of the chosen word.

```
5326 \str_new:N \__spintent_spcirc_mode_str
5327 \tl_new:N \__spintent_spcirc_prefix_tl
5328 \str_new:N \__spintent_spcirc_print_sym_str
5329 \str_new:N \__spintent_spcirc_read_sty_str
```

```

5330 \str_new:N \__spintent_spcirc_radio_sty_str
5331 \str_new:N \__spintent_spcirc_sub_index_str
5332 \str_new:N \__spintent_spcirc_articulo_str
5333 \tl_new:N \__spintent_spcirc_read_arg_tl

```

(End of definition for `\__spintent_spcirc_mode_str` and others.)

`\__spintent_spcirc_word_tl` Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (circunferencia/círculo or the `read-arg` override) and the final literal `intent` string.

```

5334 \tl_new:N \__spintent_spcirc_word_tl

```

(End of definition for `\__spintent_spcirc_word_tl`.)

11.46.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg` Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

5335 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
5336 {
5337   Invalid~argument~format~in~\token_to_str:N \spcirc:~'#1'~\msg_line_context:.
5338 }
5339 {
5340   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
5341 }

```

(End of definition for `spcirc-invalid-arg`.)

11.46.3 Definition of keys for `\spcirc`

The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «la circunferencia», `circle` for «el círculo». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (`side`, via `\spside`) or its *measure* (`mside`, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```

5342 \keys_define:nn { spintent / spcirc }
5343 {
5344   read-cmd      .choices:nn = { school , mathcat }
5345   {
5346     \int_case:nn { \_keys_choice_int }
5347     {
5348       {1}
5349       {
5350         \str_set:Nn \__spintent_spcirc_mode_str { school }
5351         \keys_set:nn { spintent / spside } { read-cmd = school }
5352         \keys_set:nn { spintent / spmside } { read-cmd = school }
5353       }
5354       {2}
5355       {
5356         \str_set:Nn \__spintent_spcirc_mode_str { mathcat }
5357         \keys_set:nn { spintent / spside } { read-cmd = mathcat }
5358         \keys_set:nn { spintent / spmside } { read-cmd = mathcat }
5359       }
5360     }
5361   },
5362   read-cmd      .initial:n = school,
5363   read-cmd      .value_required:n = true,
5364   prefix        .code:n =
5365   { \__spintent_sanitizé_affix:Nn \__spintent_spcirc_prefix_tl {#1} },
5366   prefix        .initial:n = ,
5367   prefix        .value_required:n = true,
5368   print-sym     .choices:nn = { usual , odot }
5369   {
5370     \int_case:nn { \_keys_choice_int }
5371     {
5372       {1} { \str_set:Nn \__spintent_spcirc_print_sym_str { usual } }
5373       {2} { \str_set:Nn \__spintent_spcirc_print_sym_str { odot } }
5374     }
5375   },
5376   print-sym     .initial:n = odot,
5377   print-sym     .value_required:n = true,
5378   read-sty      .choices:nn = { circun , circle }
5379   {

```

```

5380     \int_case:nn { \l_keys_choice_int }
5381     {
5382         {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
5383         {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
5384     }
5385 },
5386 read-sty      .initial:n = circun,
5387 read-sty      .value_required:n = true,
5388 diameter      .bool_set:N = \l__spintent_spcirc_diameter_bool,
5389 diameter      .initial:n = false,
5390 diameter      .value_required:n = true,
5391 radio-sty      .choices:nn = { side , mside }
5392     {
5393         \int_case:nn { \l_keys_choice_int }
5394         {
5395             {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
5396             {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
5397         }
5398     },
5399 radio-sty      .initial:n = mside,
5400 radio-sty      .value_required:n = true,
5401 sub-index      .code:n =
5402     { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
5403 sub-index      .initial:n = ,
5404 sub-index      .value_required:n = true,
5405 read-sub       .bool_set:N = \l__spintent_spcirc_read_sub_bool,
5406 read-sub       .initial:n = false,
5407 read-sub       .value_required:n = true,
5408 read-arg       .code:n =
5409     { \__spintent_sanitiz_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
5410 read-arg       .initial:n = ,
5411 read-arg       .value_required:n = true,
5412 silent         .bool_set:N = \l__spintent_spcirc_silent_bool,
5413 silent         .initial:n = false,
5414 silent         .value_required:n = true,
5415 concept        .bool_set:N = \l__spintent_spcirc_concept_bool,
5416 concept        .initial:n = true,
5417 concept        .value_required:n = true,
5418 spside         .meta:nn = { spintent / spside } { #1 },
5419 spside         .value_required:n = true,
5420 spmside        .meta:nn = { spintent / spmside } { #1 },
5421 spmside        .value_required:n = true,
5422 unknown        .code:n =
5423     {
5424         \str_set:Nn \l__spintent_command_name_str { spcirc }
5425         \__spintent_unknown_keys_cmd:n {#1}
5426     },
5427 }

```

(End of definition for read-cmd and others.)

11.46.4 Internal functions for \spcirc

```

\l__spintent_spcirc_has_arg_bool
\l__spintent_spcirc_intent_tl
\l__spintent_spcirc_intent_arg_tl
\l__spintent_spcirc_connector_tl
\l__spintent_spcirc_connector_arg_tl
\l__spintent_spcirc_radio_arg_tl

```

The variable `\l__spintent_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l__spintent_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional `sub-index` suffix) paired with the symbol. The variable `\l__spintent_spcirc_connector_tl` holds de-centro-en, paired with the (center) group; the center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`), as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintent_spcirc_radio_arg_tl`, is spoken directly on the , delimiter after the center, replacing its own reading rather than pairing with a group.

```

5428 \bool_new:N \l__spintent_spcirc_has_arg_bool
5429 \tl_new:N \l__spintent_spcirc_intent_tl
5430 \tl_new:N \l__spintent_spcirc_intent_arg_tl
5431 \tl_new:N \l__spintent_spcirc_connector_tl
5432 \tl_new:N \l__spintent_spcirc_connector_arg_tl
5433 \tl_new:N \l__spintent_spcirc_radio_arg_tl

```

(End of definition for \l__spintent_spcirc_has_arg_bool and others.)

`\__spintent_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

5434 \cs_new_protected:Nn \__spintent_spcirc_sym:

```

```

5435 {
5436   \str_if_eq:VnTF \l__spintent_spcirc_print_sym_str { odot }
5437   {
5438     \group_begin:
5439     \textstyle\bigodot
5440     \group_end:
5441   }
5442   { C }
5443 }

```

(End of definition for `\l__spintent_spcirc_sym:`.)

`\l__spintent_spcirc_word_and_article:` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```

5444 \cs_new_protected:Nn \l__spintent_spcirc_word_and_article:
5445 {
5446   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5447   {
5448     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5449     {
5450       \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia }
5451       \str_set:Nn \l__spintent_spcirc_articulo_str { la }
5452     }
5453     {
5454       \tl_set:Nn \l__spintent_spcirc_word_tl { círculo }
5455       \str_set:Nn \l__spintent_spcirc_articulo_str { el }
5456     }
5457   }
5458   {
5459     \tl_set:Nn \l__spintent_spcirc_word_tl { \l__spintent_spcirc_read_arg_tl }
5460     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5461     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5462     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5463   }
5464 }

```

(End of definition for `\l__spintent_spcirc_word_and_article:`.)

`\l__spintent_spcirc_radio_print:` Dispatches on `\l__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```

5465 \cs_new_protected:Nn \l__spintent_spcirc_radio_print:
5466 {
5467   \str_case:Vn \l__spintent_geo_luaset_circ_radio_type_str
5468   {
5469     { number } { \exp_args:Ne \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
5470     { measure } { \exp_args:Ne \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
5471     { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
5472     { segment }
5473     {
5474       \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
5475       { \exp_args:Ne \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5476       { \exp_args:Ne \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5477     }
5478   }
5479 }

```

(End of definition for `\l__spintent_spcirc_radio_print:`.)

`\l__spintent_spcirc_build_concept:` Builds two words: `\l__spintent_spcirc_intent_tl` (via `\l__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (`de-centro-en`), paired with the (center) group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\l__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

```

5480 \cs_new_protected:Nn \l__spintent_spcirc_build_concept:
5481 {
5482   \l__spintent_spcirc_word_and_article:
5483   \tl_set:Ne \l__spintent_spcirc_intent_tl { \l__spintent_spcirc_word_tl }
5484   \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5485   {

```

```

5486     \tl_put_right:Ne \l__spintent_spcirc_intent_tl
5487     {
5488         -
5489         \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
5490         \l__spintent_spcirc_sub_index_str
5491     }
5492 }
5493 \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }

```

The key `silent`, when `true`, overrides `\l__spintent_spcirc_intent_arg_tl`/`\l__spintent_spcirc_radio_arg_tl` with `_-` and `\l__spintent_spcirc_connector_arg_tl` with `_( $pt )` (the center is always read), regardless of `read-arg`.

```

5494     \bool_if:NTF \l__spintent_spcirc_silent_bool
5495     {
5496         \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5497         \tl_set:Ne \l__spintent_spcirc_connector_arg_tl { _( $ pt ) }
5498         \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5499     }

```

With `concept true`, `\l__spintent_spcirc_intent_arg_tl` is `_``\l__spintent_spcirc_intent_tl` alone (bare — the group it wraps has nothing to reference, only a silenced symbol), with `prefix` placed after the leading `_` when given; `\l__spintent_spcirc_connector_arg_tl` is `\l__spintent_spcirc_connector_tl:prefix` and `\l__spintent_spcirc_radio_arg_tl` — spoken directly on the `,` delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\l__spintent_spcirc_radio_print:`’s independent reading follows as the next sibling.

```

5500     {
5501         \bool_if:NTF \l__spintent_spcirc_concept_bool
5502         {
5503             \tl_set:Ne \l__spintent_spcirc_intent_arg_tl
5504             {
5505                 -
5506                 \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
5507                 { \tl_use:N \l__spintent_spcirc_prefix_tl - }
5508                 \l__spintent_spcirc_intent_tl
5509             }
5510             \tl_set:Ne \l__spintent_spcirc_connector_arg_tl
5511             { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5512             \bool_if:NTF \l__spintent_spcirc_diameter_bool
5513             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5514             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5515         }

```

With `concept false` and `read-arg` empty, the same `_-`/`_( $pt )` silencing applies as `silent`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio`/`y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\l__spintent_spside_process:nn`).

```

5516     {
5517         \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5518         {
5519             \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5520             \tl_set:Ne \l__spintent_spcirc_connector_arg_tl { _( $ pt ) }
5521             \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5522         }
5523         {
5524             \tl_set:Ne \l__spintent_spcirc_intent_arg_tl { _ \l__spintent_spcirc_intent_tl }
5525             \tl_set:Ne \l__spintent_spcirc_connector_arg_tl
5526             { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5527             \bool_if:NTF \l__spintent_spcirc_diameter_bool
5528             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5529             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5530         }
5531     }
5532 }
5533 }

```

(End of definition for `\l__spintent_spcirc_build_concept:`)

`\l__spintent_spcirc_process:n`

Receives the `d()` argument in ‘#1’, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\l__spintent_spcirc_exec:nn`) — `\l__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\l__spintent_luafun_geo_circ_parse_and_set:n` never called at all —

there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

5534 \cs_new_protected:Nn \__spintent_spcirc_process:n
5535 {
5536   \tl_if_empty:nTF {#1}
5537   {
5538     \bool_set_false:N \l__spintent_spcirc_has_arg_bool
5539   }
5540   {
5541     \bool_set_true:N \l__spintent_spcirc_has_arg_bool
5542     \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5543     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5544     {
5545       \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5546     }
5547   }
5548   \__spintent_spcirc_mathml:
5549 }

```

(End of definition for `\__spintent_spcirc_process:n`)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\l__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\l__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\l__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol's own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`'s symbol and `\sparc`'s accent. When a radius is present, the `,` delimiter after the center carries `\l__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — except with `silent=true`, when it is wrapped in a literal `_\MathMLintent`, which silences whatever it produces. Otherwise, wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_mrow_intent_arg_tl` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back's recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter's own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

5550 \cs_new_protected:Nn \__spintent_spcirc_mathml:
5551 {
5552   \__spintent_invisible_sep: % (luamml issues #26)
5553   \__spintent_spcirc_build_concept:
5554   \MathMLintent { \l__spintent_spcirc_intent_arg_tl }
5555   {
5556     \__spintent_inner_sep:
5557     \__spintent_luafun_mrow_intent_arg:nn
5558     { } { intent = ":silent" }
5559     { \__spintent_spcirc_sym: }
5560     \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5561     {
5562       \c_math_subscript_token
5563       {
5564         \__spintent_luafun_mrow_intent_arg:nn
5565         { } { intent = ":silent" }
5566         { \l__spintent_spcirc_sub_index_str }
5567       }
5568     }
5569   }
5570   \bool_if:NT \l__spintent_spcirc_has_arg_bool
5571   {
5572     \str_if_empty:NTF \l__spintent_geo_luaset_circ_radio_type_str
5573     {

```

```

5574         \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5575     {
5576         \__spintent_inner_sep:
5577         \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5578         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaseta_points_
5579         \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
5580     }
5581 }
5582 {
5583     \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5584     {
5585         \__spintent_inner_sep:
5586         \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5587         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaseta_points_
5588         )
5589     \MathMLintent { \l__spintent_spcirc_radio_arg_tl } { , }
5590     \bool_if:NTF \l__spintent_spcirc_silent_bool
5591     {
5592         \MathMLintent { _- }
5593         {
5594             \__spintent_inner_sep:
5595             \__spintent_spcirc_radio_print:
5596         }
5597     }
5598     { \__spintent_spcirc_radio_print: }
5599     \MathMLintent { :silent } { \rparen }
5600 }
5601 }
5602 }

```

(End of definition for `\__spintent_spcirc_mathml:`)

`\__spintent_spcirc_exec:nn`

The function `\__spintent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spcirc_print:n`.

```

5603 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
5604 {
5605     \keys_set:nn { spintent / spcirc } { #1 }
5606     \__spintent_spcirc_process:n { #2 }
5607 }

```

(End of definition for `\__spintent_spcirc_exec:nn`)

`\spcirc`

The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcd/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

5608 \NewDocumentCommand \spcirc { 0{} d() }
5609 {
5610     \mode_if_math:TF
5611     {
5612         \group_begin:
5613         \attribute \l__spintent_scope_attr_int = 1
5614         \tl_if_novalue:nTF {#2}
5615         { \__spintent_spcirc_exec:nn { #1 } { } }
5616         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
5617     \group_end:
5618     }
5619     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
5620 }

```

(End of definition for `\spcirc`. This function is documented on page 30.)

11.47 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.47.1 Definition of variables for \spAcirc and \spPcirc

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```
\__spinttent_spAcirc_mode_str
\__spinttent_spPcirc_mode_str
\__spinttent_spAcirc_print_sym_str
\__spinttent_spPcirc_print_sym_str
\__spinttent_spAcirc_read_sty_str
\__spinttent_spPcirc_read_sty_str
\__spinttent_spAcirc_radio_sty_str
\__spinttent_spPcirc_radio_sty_str
\__spinttent_spAcirc_sub_index_str
\__spinttent_spPcirc_sub_index_str
\__spinttent_spAcirc_prefix_tl
\__spinttent_spPcirc_prefix_tl
\__spinttent_spAcirc_read_arg_tl
\__spinttent_spPcirc_read_arg_tl
\__spinttent_geo_circfig_mode_str
```

```
5621 \clist_map_inline:nn { Acirc , Pcirc }
5622 {
5623   \str_new:c { \__spinttent_sp #1 _mode_str      }
5624   \str_new:c { \__spinttent_sp #1 _print_sym_str }
5625   \str_new:c { \__spinttent_sp #1 _read_sty_str  }
5626   \str_new:c { \__spinttent_sp #1 _radio_sty_str }
5627   \str_new:c { \__spinttent_sp #1 _sub_index_str }
5628   \tl_new:c { \__spinttent_sp #1 _prefix_tl      }
5629   \tl_new:c { \__spinttent_sp #1 _read_arg_tl    }
5630 }
```

(End of definition for __spinttent_spAcirc_mode_str and others.)

The variable `\__spinttent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (`bare`).

```
5631 \str_new:N \__spinttent_geo_circfig_mode_str
```

(End of definition for __spinttent_geo_circfig_mode_str.)

11.47.2 Definition of error messages for \spAcirc and \spPcirc

```
spAcirc-invalid-arg
spPcirc-invalid-arg
```

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```
5632 \clist_map_inline:nn { Acirc , Pcirc }
5633 {
5634   \msg_new:nnnn { spinttent } { sp #1 -invalid-arg }
5635   {
5636     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
5637   }
5638   {
5639     See~the~documentation~of~##1~for~the~accepted~argument.
5640   }
}
```

(End of definition for spAcirc-invalid-arg and spPcirc-invalid-arg.)

11.47.3 Definition of keys for \spAcirc and \spPcirc

```
read-cmd
prefix
print-sym
read-sty
concept
sub-index
read-sub
read-arg
silent
unknown
```

We define the shared (*keys*) `read-cmd`, `prefix`, `print-sym`, `read-sty`, `concept`, `sub-index`, `read-sub`, `read-arg` and `silent`.

```
5641 \keys_define:nn { spinttent / sp #1 }
5642 {
5643   read-cmd .choices:nn = { school , mathcat }
5644   {
5645     \int_case:nn { \l_keys_choice_int }
5646     {
5647       {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
5648       {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
5649     }
5650   },
5651   read-cmd .initial:n = school,
5652   read-cmd .value_required:n = true,
5653   read-cmd / unknown .code:n =
5654     \msg_error:nneee { spinttent } { unknown-choice }
5655     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5656   prefix .code:n =
5657     { \__spinttent_sanitize_affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
5658   prefix .initial:n = ,
5659   prefix .value_required:n = true,
5660   print-sym .choices:nn = { usual , odot }
5661   {
5662     \int_case:nn { \l_keys_choice_int }
5663     {
5664       {1} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { usual } }
5665       {2} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { odot } }
5666     }
5667   },
5668   print-sym .initial:n = odot,
5669   print-sym .value_required:n = true,
5670   print-sym / unknown .code:n =
5671     \msg_error:nneee { spinttent } { unknown-choice }
```

```

5672         { print-sym } { usual,~odot } { \exp_not:n {##1} },
5673 read-sty             .choices:nn = { circun , circle }
5674     {
5675         \int_case:nn { \l_keys_choice_int }
5676         {
5677             {1} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { circun } }
5678             {2} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { circle } }
5679         }
5680     },
5681 read-sty             .initial:n = circun,
5682 read-sty             .value_required:n = true,
5683 concept             .bool_set:c = { \l__spintent_sp #1 _concept_bool },
5684 concept             .initial:n = true,
5685 concept             .value_required:n = true,
5686 sub-index           .code:n =
5687     { \str_set:cn { \l__spintent_sp #1 _sub_index_str } {##1} },
5688 sub-index           .initial:n = ,
5689 sub-index           .value_required:n = true,
5690 read-sub            .bool_set:c = { \l__spintent_sp #1 _read_sub_bool },
5691 read-sub            .initial:n = false,
5692 read-sub            .value_required:n = true,
5693 read-arg            .code:n =
5694     { \l__spintent_sanitize_affix:cn { \l__spintent_sp #1 _read_arg_tl } {##1} },
5695 read-arg            .initial:n = ,
5696 read-arg            .value_required:n = true,
5697 silent             .bool_set:c = { \l__spintent_sp #1 _silent_bool },
5698 silent             .initial:n = false,
5699 silent             .value_required:n = true,
5700 unknown            .code:n =
5701     {
5702         \str_set:Nn \l__spintent_command_name_str { sp #1 }
5703         \l__spintent_unknown_keys_cmd:n {##1}
5704     },
5705 }
5706 }

```

(End of definition for `read-cmd` and others.)

```

\l__spintent_spcircfig_short_bool
\l__spintent_spcircfig_op_tl
\l__spintent_spcircfig_intent_tl
\l__spintent_spcircfig_intent_arg_tl

```

The variable `\l__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\l__spintent_luafun_geo_afig_parse_and_set:n/\l__spintent_luafun_geo_pfifg_p` the same entry point `\spAfig/\spPfifg` use) rather than a real circle. `\l__spintent_spcircfig_op_tl` holds *área* or *perímetro*. The variable `\l__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\l__spintent_spcircfig_del_word_tl` (via `\l__spintent_spcircfig_del_word:n`: the word, contracted with `de` `-/de` `-la`-, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con` `centro` `-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. `O_{1}`), as in the Euler circle) is never an issue.

```

5707 \bool_new:N \l__spintent_spcircfig_short_bool
5708 \tl_new:N \l__spintent_spcircfig_op_tl
5709 \tl_new:N \l__spintent_spcircfig_del_word_tl
5710 \tl_new:N \l__spintent_spcircfig_intent_tl
5711 \tl_new:N \l__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\l__spintent_spcircfig_short_bool` and others.)

```
\l__spintent_spcircfig_sym:n
```

Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

5712 \cs_new_protected:Nn \l__spintent_spcircfig_sym:n
5713 {
5714     \str_if_eq:vnTF { \l__spintent_sp #1 _print_sym_str } { odot }
5715     { \bigodot }
5716     { C }
5717 }

```

(End of definition for `\l__spintent_spcircfig_sym:n`.)

```
\l__spintent_spcircfig_word_and_article:n
```

Sets `\l__spintent_spcirc_word_tl` to `read-arg` or *circunferencia/círculo* from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

5718 \cs_new_protected:Nn \l__spintent_spcircfig_word_and_article:n
5719 {
5720     \str_if_eq:vnTF { \l__spintent_sp #1 _read_sty_str } { circun }

```

```

5721     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5722     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5723     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5724     {
5725         \str_if_eq:vnTF { l__spintent_sp #1 _read_sty_str } { circun }
5726         { \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia } }
5727         { \tl_set:Nn \l__spintent_spcirc_word_tl { círculo } }
5728     }
5729     {
5730         \tl_set:Ne \l__spintent_spcirc_word_tl
5731         {
5732             \tl_use:c { l__spintent_sp #1 _read_arg_tl }
5733         }
5734     }
5735 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n`

Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`'s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in `'#1'`.

```

5736 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n
5737 {
5738     \__spintent_spcircfig_word_and_article:n {#1}
5739     \str_if_eq:VnTF \l__spintent_spcirc_articulo_str { el }
5740     {
5741         \tl_set:Ne \l__spintent_spcircfig_del_word_tl { del- \l__spintent_spcirc_word_tl }
5742     }
5743     {
5744         \tl_set:Ne \l__spintent_spcircfig_del_word_tl { de-la- \l__spintent_spcirc_word_tl }
5745     }
5746 }

```

(End of definition for `\__spintent_spcircfig_del_word:n`.)

`\__spintent_spcircfig_build_short:n`

Handles the short number/letter case: builds `\l__spintent_spcircfig_intent_arg_tl` as `\l__spintent_spcircfig_intent_arg_tl` (*ber word*), with `prefix` placed after the leading `_` when given, with no `:prefix` (no reference — there is nothing to point to), or `_-` when `silent` is `true` or `concept` is `false`. Receives the short command name in `'#1'`.

```

5747 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
5748 {
5749     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5750     {
5751         \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5752     }
5753     {
5754         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5755         {
5756             \__spintent_spcircfig_del_word:n {#1}
5757             \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5758             {
5759                 -
5760                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5761                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } } - }
5762             \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl -
5763             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5764             \l__spintent_geo_luaset_sub_spoken_str
5765         }
5766     }
5767     {
5768         \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5769     }
5770 }
5771 }

```

(End of definition for `\__spintent_spcircfig_build_short:n`.)

`\__spintent_spcircfig_finish_intent:nn`

Appends an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` to `\l__spintent_spcircfig_intent_arg_tl` — shared by both branches of `\__spintent_spcircfig_build_concept:nn`, after either one has set the word itself. Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5772 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn

```

```

5773 {
5774   \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5775   {
5776     \tl_put_right:Ne \l__spintent_spcircfig_intent_tl
5777     {
5778       -
5779       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5780       \str_use:c { l__spintent_sp #1 _sub_index_str }
5781     }
5782   }
5783   \tl_if_empty:nF {#2}
5784   {
5785     \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
5786   }
5787 }

```

(End of definition for `\__spintent_spcircfig_finish_intent:nn`.)

`\__spintent_spcircfig_build_concept:nn`

Builds `\__spintent_spcircfig_intent_arg_tl`. `_-` when `silent` is `true`, or when `false` and `concept` is `false` with `read-arg` empty (`_( $pt )` instead when `#2` is non-empty — the center is still read either way). Otherwise `\__spintent_spcircfig_intent_tl` — with `:prefix( $pt )` appended only when `#2` is non-empty — with `prefix` placed after the leading `_` when given, where `\__spintent_spcircfig_intent_tl` is: with `concept` `true`, `\__spintent_spcircfig_op_tl- \__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with `\__spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user's to write, same convention as `\__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5788 \cs_new_protected:Nn \__spintent_spcircfig_build_concept:nn
5789 {
5790   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5791   {
5792     \tl_if_empty:nTF {#2}
5793     { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5794     { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5795   }
5796   {
5797     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5798     {
5799       \__spintent_spcircfig_del_word:n {#1}
5800       \tl_set:Ne \l__spintent_spcircfig_intent_tl
5801       {
5802         \__spintent_spcircfig_op_tl - \__spintent_spcircfig_del_word_tl
5803       }
5804       \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5805       \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5806       {
5807         -
5808         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5809         {
5810           \tl_use:c { l__spintent_sp #1 _prefix_tl } -
5811         }
5812         \__spintent_spcircfig_intent_tl
5813         \tl_if_empty:nF {#2} { :prefix ( $ pt ) }
5814       }
5815     }
5816     {
5817       \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5818       {
5819         \tl_if_empty:nTF {#2}
5820         { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5821         { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5822       }
5823       {
5824         \tl_set:Ne \l__spintent_spcircfig_intent_tl
5825         { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
5826         \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5827         \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5828         {

```

```

5829         -
5830         \l__spintent_spcircfig_intent_tl
5831         \tl_if_empty:nF {#2} { :prefix ( $ pt ) }
5832     }
5833 }
5834 }
5835 }
5836 }

```

(End of definition for `\__spintent_spcircfig_build_concept:nn`)

`\__spintent_spcircfig_process:nn`

Receives the short command name in ‘#1’ and the argument in ‘#2’. Sets `\l__spintent_spcircfig_op_tl` to `área/perímetro`. With #2 empty, `\l__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in #2, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

5837 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5838 {
5839     \str_if_eq:eeTF {#1} { Acirc }
5840     { \tl_set:Nn \l__spintent_spcircfig_op_tl { área } }
5841     { \tl_set:Nn \l__spintent_spcircfig_op_tl { perímetro } }
5842     \tl_if_empty:nTF {#2}
5843     {
5844         \bool_set_false:N \l__spintent_spcircfig_short_bool
5845     }
5846     {
5847         \bool_set_false:N \l__spintent_spcircfig_short_bool
5848         \tl_if_in:nnTF {#2} { , }
5849         {
5850             \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5851             { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5852         }
5853         {
5854             \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5855             \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5856             {
5857                 \str_if_eq:eeTF {#1} { Acirc }
5858                 { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5859                 { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5860                 \str_if_eq:VnF \l__spintent_geo_luaset_error_str { true }
5861                 {
5862                     \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
5863                     { \bool_set_true:N \l__spintent_spcircfig_short_bool }
5864                     { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
5865                 }
5866             }
5867             \bool_if:NF \l__spintent_spcircfig_short_bool
5868             {
5869                 \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5870                 {
5871                     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5872                     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5873                 }
5874             }
5875         }
5876     }
5877     \bool_if:NTF \l__spintent_spcircfig_short_bool
5878     { \__spintent_spcircfig_build_short:n {#1} }
5879     { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
5880     \__spintent_spcircfig_mathml:nn { #1 } { #2 }
5881 }

```

(End of definition for `\__spintent_spcircfig_process:nn`)

`\__spintenc_spcircfig_mathml:nn`

Renders `\l__spintenc_spcircfig_intent_arg_tl`, already built by `\__spintenc_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintenc_mathcal:n{⟨A|P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintenc_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5882 \cs_new_protected:Nn \__spintenc_spcircfig_mathml:nn
5883 {
5884   \__spintenc_invisible_sep: % (luamml issues #26)
5885   \MathMLintent { \l__spintenc_spcircfig_intent_arg_tl }
5886   {
5887     \__spintenc_inner_sep:
5888     \__spintenc_luafun_mrow_intent_arg:nn
5889     { } { intent = ":silent" }
5890     {
5891       \str_if_eq:eeTF {#1} { Acirc } { \__spintenc_mathcal:n { A } } { \__spintenc_mathcal:n
5892     }
5893     \bool_if:NTF \l__spintenc_spcircfig_short_bool
5894     {
5895       \c_math_subscript_token
5896       {
5897         \__spintenc_spcircfig_sym:n {#1}
5898         \c_math_subscript_token { \l__spintenc_geo_luaset_print_tl }
5899       }
5900     }
5901     {
5902       \c_math_subscript_token
5903       {
5904         \MathMLintent { :silent } { \__spintenc_spcircfig_sym:n {#1} }
5905         \str_if_empty:cF { \l__spintenc_sp #1 _sub_index_str }
5906         {
5907           \c_math_subscript_token
5908           {
5909             \MathMLintent { :silent }
5910             { \str_use:c { \l__spintenc_sp #1 _sub_index_str } }
5911           }
5912         }
5913         \tl_if_empty:nF {#2}
5914         {
5915           \MathMLintent { :silent } { \lparen }
5916           \__spintenc_mathmlarg_point:nnV { pt } { sp #1 } \l__spintenc_geo_luaset_poin
5917           \MathMLintent { :silent } { \rparen }
5918         }
5919       }
5920     }
5921   }
5922 }

```

(End of definition for `\__spintenc_spcircfig_mathml:nn`.)

`\__spintenc_spcircfig_exec:nnn`

The function `\__spintenc_spcircfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintenc_spcircfig_process:nn`.

```

5923 \cs_new_protected:Nn \__spintenc_spcircfig_exec:nnn
5924 {
5925   \keys_set:nn { spintenc / sp #1 } { #2 }
5926   \__spintenc_spcircfig_process:nn { #1 } { #3 }
5927 }

```

(End of definition for `\__spintenc_spcircfig_exec:nnn`.)

`\spAcirc`
`\spPcirc`

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintenc_spcircfig_exec:nnn`.

```

5928 \NewDocumentCommand \spAcirc { 0{} d() }
5929 {
5930   \mode_if_math:TF
5931   {
5932     \group_begin:
5933     \attribute \l__spintenc_scope_attr_int = 1
5934     \tl_if_novalue:nTF {#2}

```

```

5935         { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
5936         { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5937     \group_end:
5938 }
5939 { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }
5940 }
5941 \NewDocumentCommand \spPcirc { 0{} d() }
5942 {
5943     \mode_if_math:TF
5944     {
5945         \group_begin:
5946         \attribute \l__spintent_scope_attr_int = 1
5947         \tl_if_novalue:nTF {#2}
5948         { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5949         { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5950         \group_end:
5951     }
5952     { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }
5953 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page 31.)

11.48 The command `\spVol`

The user-level command `\spVol` provides a semantic interface for the volume of a solid, always typesetting `\mathcal{V}`.

11.48.1 Definition of variables for `\spVol`

The variable `\l__spintent_spVol_mode_str` holds the `read-cmd` choice (`school` or `mathcat`); `\l__spintent_spVol_sub_index_str` the raw `sub-index` text; `\l__spintent_spVol_read_arg_tl` the sanitized `read-arg` text. `\l__spintent_spVol_read_arg_tl` and `\l__spintent_spVol_suffix_tl` are scratch variables holding the word to voice (volumen) and the text to attach after it; `\l__spintent_spVol_native_bool` records whether that text should be combined natively with the subscript (silenced) or left for MathCAT to read on its own. The variable `\l__spintent_spVol_read_arg_full_tl` holds the text to voice when `read-arg` is given.

```

5954 \str_new:N \l__spintent_spVol_mode_str
5955 \str_new:N \l__spintent_spVol_sub_index_str
5956 \tl_new:N \l__spintent_spVol_read_arg_tl
5957 \tl_new:N \l__spintent_spVol_word_tl
5958 \tl_new:N \l__spintent_spVol_suffix_tl
5959 \bool_new:N \l__spintent_spVol_native_bool
5960 \tl_new:N \l__spintent_spVol_read_arg_full_tl

```

(End of definition for `\l__spintent_spVol_mode_str` and others.)

11.48.2 Definition of keys for `\spVol`

We define the `(keys)` `read-cmd`, `sub-index`, `read-sub`, `read-arg`, `concept` and `silent`.

```

read-cmd 5961 \keys_define:nn { spintent / spVol }
sub-index 5962 {
read-sub 5963     read-cmd .choices:nn = { school , mathcat }
read-arg 5964     {
concept 5965         \int_case:nn { \l_keys_choice_int }
silent 5966         {
unknown 5967             {1} { \str_set:Nn \l__spintent_spVol_mode_str { school } }
5968             {2} { \str_set:Nn \l__spintent_spVol_mode_str { mathcat } }
5969         }
5970     },
5971     read-cmd .initial:n = school,
5972     read-cmd .value_required:n = true,
5973     read-cmd / unknown .code:n =
5974         \msg_error:nneee { spintent } { unknown-choice }
5975         { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
5976     sub-index .code:n =
5977         { \str_set:Nn \l__spintent_spVol_sub_index_str {#1} },
5978     sub-index .initial:n = ,
5979     sub-index .value_required:n = true,
5980     read-sub .bool_set:N = \l__spintent_spVol_read_sub_bool,
5981     read-sub .initial:n = false,
5982     read-sub .value_required:n = true,
5983     read-arg .code:n =
5984         { \__spintent_sanitize_affix:Nn \l__spintent_spVol_read_arg_tl {#1} },
5985     read-arg .initial:n = ,
5986     read-arg .value_required:n = true,

```

```

5987     concept      .bool_set:N = \l__spintent_spVol_concept_bool,
5988     concept      .initial:n = true,
5989     concept      .value_required:n = true,
5990     silent       .bool_set:N = \l__spintent_spVol_silent_bool,
5991     silent       .initial:n = false,
5992     silent       .value_required:n = true,
5993     unknown      .code:n =
5994     {
5995         \str_set:Nn \l__spintent_command_name_str { spVol }
5996         \__spintent_unknown_keys_cmd:n {#1}
5997     },
5998 }

```

(End of definition for read-cmd and others.)

11.48.3 Internal functions for \spVol

`\__spintent_spVol_print:` The function `\__spintent_spVol_print:` dispatches to `\__spintent_spVol_silent:` when `silent` is `true`, or to `\__spintent_spVol_verbal:` otherwise.

```

5999 \cs_new_protected:Nn \__spintent_spVol_print:
6000 {
6001     \__spintent_invisible_sep:
6002     \bool_if:NTF \l__spintent_spVol_silent_bool
6003     { \__spintent_spVol_silent: }
6004     { \__spintent_spVol_verbal: }
6005 }

```

(End of definition for `\__spintent_spVol_print:`.)

`\__spintent_spVol_silent:` The function `\__spintent_spVol_silent:` typesets `\mathcal{V}` with the intent `_` — the symbol alone through the shared queue when there is no `sub-index`, or symbol and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

6006 \cs_new_protected:Nn \__spintent_spVol_silent:
6007 {
6008     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6009     {
6010         \__spintent_luafun_mrow_intent_arg:nn
6011         { } { intent = "_" }
6012         { \mathcal{V} }
6013     }
6014     {
6015         \MathMLintent { _- }
6016         {
6017             \__spintent_inner_sep:
6018             \mathcal{V}
6019             \c_math_subscript_token
6020             {
6021                 \MathMLintent { :silent }
6022                 { \l__spintent_spVol_sub_index_str }
6023             }
6024         }
6025     }
6026 }

```

(End of definition for `\__spintent_spVol_silent:`.)

`\__spintent_spVol_verbal:` The function `\__spintent_spVol_verbal:` delegates to `\__spintent_spVol_read_arg_print:` when `read-arg` is given, and otherwise, with `concept false`, to `\__spintent_spVol_symbol_print:`. With `concept true` it sets `\l__spintent_spVol_word_tl` to `volumen` and `\l__spintent_spVol_suffix_tl` to `sub-` (`value`)/(`value`) (per `read-sub`), combined natively only under `read-cmd=school` — under `read-cmd=mathcat` the suffix is left for MathCAT to read on its own. Delegates the actual typesetting to `\__spintent_spVol_attach_subscript:`

```

6027 \cs_new_protected:Nn \__spintent_spVol_verbal:
6028 {
6029     \tl_if_empty:NTF \l__spintent_spVol_read_arg_tl
6030     {
6031         \bool_if:NTF \l__spintent_spVol_concept_bool
6032         {
6033             \tl_set:Nn \l__spintent_spVol_word_tl { volumen }
6034             \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
6035             { \bool_set_true:N \l__spintent_spVol_native_bool }
6036             { \bool_set_false:N \l__spintent_spVol_native_bool }
6037             \tl_set:Ne \l__spintent_spVol_suffix_tl

```

```

6038         {
6039             \bool_if:NT \l__spintrent_spVol_read_sub_bool { sub- }
6040             \l__spintrent_spVol_sub_index_str
6041         }
6042         \__spintrent_spVol_attach_subscript:
6043     }
6044     { \__spintrent_spVol_symbol_print: }
6045 }
6046 { \__spintrent_spVol_read_arg_print: }
6047 }

```

(End of definition for `\__spintrent_spVol_verbal:`.)

`\__spintrent_spVol_symbol_print:`

With `concept false`, the symbol itself is read instead — but only when `read-cmd` is `school`: with no `sub-index`, the shared queue tags the symbol with the intent `_V-`; with one, symbol and subscript are wrapped together in one native `\MathMLintent` (subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the symbol and its subscript, unmarked by us.

```

6048 \cs_new_protected:Nn \__spintrent_spVol_symbol_print:
6049 {
6050     \str_if_empty:NTF \l__spintrent_spVol_sub_index_str
6051     {
6052         \str_if_eq:VnTF \l__spintrent_spVol_mode_str { school }
6053         {
6054             \__spintrent_luafun_mrow_intent_arg:nn
6055             { } { intent = "_V-" }
6056             { \mathcal{V} } }
6057         { \mathcal{V} } }
6058     }
6059     {
6060         \str_if_eq:VnTF \l__spintrent_spVol_mode_str { school }
6061         {
6062             \MathMLintent
6063             {
6064                 _V-
6065                 \bool_if:NT \l__spintrent_spVol_read_sub_bool { sub- }
6066                 \l__spintrent_spVol_sub_index_str
6067             }
6068             {
6069                 \__spintrent_inner_sep:
6070                 \mathcal{V}
6071                 \c_math_subscript_token
6072                 {
6073                     \MathMLintent { :silent }
6074                     { \l__spintrent_spVol_sub_index_str }
6075                 }
6076             }
6077         }
6078     }
6079     {
6080         \mathcal{V}
6081         \c_math_subscript_token
6082         { \l__spintrent_spVol_sub_index_str }
6083     }
6084 }
6085 }

```

(End of definition for `\__spintrent_spVol_symbol_print:`.)

`\__spintrent_spVol_attach_subscript:`

The function `\__spintrent_spVol_attach_subscript:` typesets `\mathcal{V}`. An empty `sub-index` tags the symbol alone through the shared queue (`\l__spintrent_spVol_suffix_tl` is discarded, since there's no subscript to attach it to). A given one combines symbol, subscript, and `\l__spintrent_spVol_suffix_tl` in one native `\MathMLintent` (subscript silenced) when `\l__spintrent_spVol_native_bool` is `true`, or tags the symbol alone through the shared queue with the subscript left bare when `false`.

```

6086 \cs_new_protected:Nn \__spintrent_spVol_attach_subscript:
6087 {
6088     \str_if_empty:NTF \l__spintrent_spVol_sub_index_str
6089     {
6090         \__spintrent_luafun_mrow_intent_arg:nn
6091         { } { intent = "_ \l__spintrent_spVol_word_tl :nofix" }
6092         { \mathcal{V} } }
6093     }

```

```

6094     {
6095         \bool_if:NTF \l__spintent_spVol_native_bool
6096         {
6097             \MathMLintent
6098             { _ \l__spintent_spVol_word_tl - \l__spintent_spVol_suffix_tl :nofix }
6099             {
6100                 \__spintent_inner_sep:
6101                 \mathcal{V}
6102                 \c_math_subscript_token
6103                 {
6104                     \MathMLintent { :silent }
6105                     { \l__spintent_spVol_sub_index_str }
6106                 }
6107             }
6108         }
6109     {
6110         \__spintent_luafun_mrow_intent_arg:nn
6111         { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
6112         { \mathcal{V} }
6113         \c_math_subscript_token
6114         { \l__spintent_spVol_sub_index_str }
6115     }
6116 }
6117 }

```

(End of definition for `\__spintent_spVol_attach_subscript:`.)

`\__spintent_spVol_read_arg_print:`

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true`, `\l__spintent_spVol_read_arg_full_tl` becomes *volumen* followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively regardless of `read-cmd:` with no `sub-index`, the shared queue tags the symbol alone; with one, symbol and subscript are wrapped together in one native `\MathMLintent`.

```

6118 \cs_new_protected:Nn \__spintent_spVol_read_arg_print:
6119 {
6120     \bool_if:NTF \l__spintent_spVol_concept_bool
6121     {
6122         \tl_set:Nx \l__spintent_spVol_read_arg_full_tl
6123         { volumen- \tl_use:N \l__spintent_spVol_read_arg_tl }
6124     }
6125     { \tl_set_eq:NN \l__spintent_spVol_read_arg_full_tl \l__spintent_spVol_read_arg_tl }
6126     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6127     {
6128         \__spintent_luafun_mrow_intent_arg:nn
6129         { } { intent = "_ \l__spintent_spVol_read_arg_full_tl :nofix" }
6130         { \mathcal{V} }
6131     }
6132     {
6133         \MathMLintent
6134         { _ \l__spintent_spVol_read_arg_full_tl :nofix }
6135         {
6136             \__spintent_inner_sep:
6137             \mathcal{V}
6138             \c_math_subscript_token
6139             {
6140                 \MathMLintent { :silent }
6141                 { \l__spintent_spVol_sub_index_str }
6142             }
6143         }
6144     }
6145 }

```

(End of definition for `\__spintent_spVol_read_arg_print:`.)

`\spVol` The public command `\spVol` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spVol_print:`.

```

6146 \NewDocumentCommand \spVol { 0{ } }
6147 {
6148     \mode_if_math:TF
6149     {
6150         \group_begin:
6151         \attribute \l__spintent_scope_attr_int = 1

```

```

6152         \keys_set:nn { spintent / spVol } {#1}
6153         \__spintent_spVol_print:
6154     \group_end:
6155 }
6156 { \msg_error:nnn { spintent } { need-math-mode } { \spVol } }
6157 }

```

(End of definition for \spVol. This function is documented on page 33.)

11.49 Finish package

Finish package implementation.

```

6158 \file_input_stop:
6159 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
\angle	125
autofit	<u>3233</u>
C	
cifras-sep	<u>289</u> , <u>574</u> , <u>847</u> , <u>1884</u> , <u>1955</u> , <u>2574</u>
Document class:	
l3doc	40
cmd-key-unknown	<u>208</u>
cmd-key-value-unknown	<u>208</u>
Commands provide by spintent :	
\spmoney	58
\spnum	48
\spsiglo	67, 70
\spunit	53
concept <u>2046</u> , <u>2256</u> , <u>3233</u> , <u>3858</u> , <u>4067</u> , <u>4354</u> , <u>4508</u> , <u>4693</u> , <u>5086</u> , <u>5342</u> , <u>5641</u> , <u>5961</u>	
concept*	<u>4693</u>
currency	<u>847</u>
currency-no-subunits	<u>816</u>
D	
diameter	<u>5342</u>
I	
inline	<u>2436</u>
inline-numeric-denominator	<u>552</u>
invalid-angle-format	<u>2524</u>
invalid-century-format	<u>1242</u>
invalid-currency	<u>816</u>
invalid-date-format	<u>1472</u>
invalid-time-format	<u>1393</u>
J	
join-word	<u>2652</u>
K	
Keys for \spAcirc provide by spintent :	
concept	155
prefix	155
print-sym	155, <u>156</u>
radio-sty	155
read-arg	155
read-cmd	155
read-sty	155
read-sub	155
silent	155
sub-index	155
Keys for \spAfig provide by spintent :	
prefix	136
print-sym	136, <u>137</u>
read-arg	136–138
read-cmd	136
read-sub	137
Keys for \spangle provide by spintent :	
concept	124
prefix	124
print-m	124
read-cmd	124
recto	124
silent	124
sub-index	124
upper-case	124
wrap-parens	80
Keys for \spBdiv provide by spintent :	
concept	80
notation-sym	80
read-parens	80
read-period-sep	80
read-sign	80
read-sym	80
set-sym	80
wrap-parens	80
Keys for \spdsym provide by spintent :	
prefix	78
read-sym	78
set-sym	78
suffix	78
Keys for \spmangle provide by spintent :	
concept	124
prefix	124
print-m	124
read-cmd	124
recto	124
silent	124
Keys for \spmarc provide by spintent :	
concept	130
prefix	129, <u>130</u>
read-cmd	129, <u>130</u>
silent	130
Keys for \spmcid provide by spintent :	
concept	84
prefix	84
read-cmd	84
result	84
silent	84
upper-case	84
Keys for \spmcm provide by spintent :	
concept	84
prefix	84
read-cmd	84
result	84
silent	84
upper-case	84
Keys for \spmcirc provide by spintent :	
concept	130
prefix	129, <u>130</u>
read-cmd	129, <u>130</u>
silent	130
Keys for \spncirc provide by spintent :	
concept	130
prefix	129, <u>130</u>
read-cmd	129, <u>130</u>
silent	130
Keys for \spncirc provide by spintent :	
concept	148
print-sym	148–150
radio-sty	148, <u>149</u>
read-arg	149
read-cmd	148
read-sty	148, <u>149</u>
read-sub	149
sub-index	148, <u>149</u>
Keys for \spcoord provide by spintent :	
silent	101
Keys for \spdiam provide by spintent :	
concept	144
read-arg	144
read-cmd	144
read-sub	144
silent	144
sub-index	144
upper-case	144
Keys for \spdiv provide by spintent :	
concept	144
read-arg	144
read-cmd	144
read-sub	144
silent	144
sub-index	144
upper-case	144
Keys for \spdiv provide by spintent :	
concept	80
notation-sym	80
read-parens	80
read-period-sep	80
read-sign	80
read-sym	80
set-sym	80
wrap-parens	80
Keys for \spdsym provide by spintent :	
prefix	78
read-sym	78
set-sym	78
suffix	78
Keys for \spmangle provide by spintent :	
concept	124
prefix	124
print-m	124
read-cmd	124
recto	124
silent	124
Keys for \spmarc provide by spintent :	
concept	130
prefix	129, <u>130</u>
read-cmd	129, <u>130</u>
silent	130
Keys for \spmcid provide by spintent :	
concept	84
prefix	84
read-cmd	84
result	84
silent	84
upper-case	84
Keys for \spmcm provide by spintent :	
concept	84
prefix	84
read-cmd	84
result	84
silent	84
upper-case	84
Keys for \spmcirc provide by spintent :	
concept	130
prefix	129, <u>130</u>
read-cmd	129, <u>130</u>
silent	130
Keys for \spncirc provide by spintent :	
concept	148
print-sym	148–150
radio-sty	148, <u>149</u>
read-arg	149
read-cmd	148
read-sty	148, <u>149</u>
read-sub	149
sub-index	148, <u>149</u>

prefix	84	print-sym	155, 156
read-cmd	84	radio-sty	155
result	84	read-arg	155
silent	84	read-cmd	155
upper-case	84	read-sty	155
Keys for \spmoney provide by spintent :		read-sub	155
cifras-sep	60	silent	155
currency	60	sub-index	155
money-math	60	Keys for \spPfig provide by spintent :	
sub-units	60	prefix	136
Keys for \spmQ provide by spintent :		print-sym	136, 137
join-word	96, 98	read-arg	136–138
read-parens	96, 97	read-cmd	136
use-dfrac	96–100	read-sub	137
use-spnZ	96, 99, 100	Keys for \spPoint provide by spintent :	
wrap-parens	96, 97	concept	106
Keys for \spmside provide by spintent :		prefix	106
concept	120	read-cmd	106
prefix	120	silent	106
print-m	120	Keys for \spPoly provide by spintent :	
print-sym	144	concept	133
read-arg	120	prefix	133
read-cmd	120, 123	print-sym	133
read-sty	120	read-cmd	133
silent	120, 123	silent	133
Keys for \spmsym provide by spintent :		Keys for \sprad provide by spintent :	
not-print	79	concept	144
prefix	79	read-arg	144
read-sym	79	read-cmd	144
set-sym	79	read-sub	144
suffix	79	silent	144
Keys for \spmul provide by spintent :		sub-index	144
cifras-sep	82	upper-case	144
notation-sym	82	Keys for \sprayo provide by spintent :	
read-parens	82	concept	117
read-period-sep	82	prefix	117
read-sign	82	read-cmd	117, 118
read-sym	82	read-sty	117
set-sym	82	silent	117, 118
wrap-parens	82	Keys for \sprazon provide by spintent :	
Keys for \spnD provide by spintent :		inline	91
concept	88	silent	91
prefix	88	use-dfrac	91
result-num	88	Keys for \spread provide by spintent :	
result	88	read-arg	76
silent	88	silent	76
Keys for \spnM provide by spintent :		Keys for \sprecta provide by spintent :	
concept	88	prefix	113
prefix	88	read-cmd	113
result-num	88	silent	113
result	88	Keys for \spshort provide by spintent :	
silent	88	read-arg	65
Keys for \spnum provide by spintent :		small-caps	65
cifras-sep	47–49	Keys for \spside provide by spintent :	
notation-sym	48	concept	120
read-period-sep	48, 50	prefix	120
read-sign	48	read-arg	120
Keys for \spnZ provide by spintent :		read-cmd	120, 123
cifras-sep	94	read-sty	120
read-parens	94, 95	silent	120, 123
read-sign	94	Keys for \spsiglo provide by spintent :	
wrap-parens	94, 95	print-era	68, 69
Keys for \spPcirc provide by spintent :		Keys for \spunit provide by spintent :	
concept	155	cifras-sep	54
prefix	155	notation-sym	54

print-cdot	54	right-angle-sqr-fallback	80
read-cdot	54	\rightangle	125
read-period-sep	54		
read-slash	54	S	
Keys for \spusep provide by spintent :		seq commands:	
silent	74	\seq_set_from_clist:Nn	62
size	74	set-sym	1738, 1809, 1884, 1955
Keys for \spvec provide by spintent :		\setspintent	3, 44, 196
autofit	109	silent	1520, 1659, 2046, 2256, 2436, 2866, 3114, 3233, 3483, 3718, 3858, 4067, 4354, 4508, 4693, 5086, 5342, 5641, 5961
concept	109	size	1520
prefix	108, 109	small-caps	1120
read-cmd	108, 109	\spAcirc	31, 154, 5928
silent	109	spAcirc-invalid-arg	5632
Keys for \spVol provide by spintent :		\spAfig	26, 136, 5057
concept	161	spAfig-invalid-arg	4655
read-arg	161	spAfig-invalid-points	4655
read-cmd	161	spAfig-read-arg-key	4655
read-sub	161	spAfig-sym-cardinality-conflict	4655
silent	161	spAfig-sym-letra-conflict	4655
sub-index	161	\spang	24, 93, 2564
		\spangle	23, 124, 4317
M		spangle-invalid-arg	4057
mathcal-fallback	80	\sparc	25, 129, 4477
\MathMLintent	128	sparc-invalid-points	4344
mc-invalid-arguments	2039	\spcirc	30, 148, 5608
mc-too-few-arguments	2282	spcirc-invalid-arg	5335
md-invalid-argument	2282	\spcoord	19, 101, 3003
Lua module provide by spintent :		spcoord-invalid-arg	2862
spintent.lua 40, 41, 43, 46–48, 53, 57–60, 64–70, 72, 83, 87, 93, 95, 100, 106		\spdate	11, 72, 1496
money-math	847	\spdiam	29, 144, 5302
		\spdiv	13, 80, 1945
N		spdiv-empty-argument	1904
not-print	1809	\spdsym	12, 78, 1785
notation-sym	289, 574, 1884, 1955	spintent internal commands:	
P		_spintent_collapse_hyphens:N	128
Packages :		_l_spintent_command_name_str	208
expl3	46, 48	_l_spintent_geo_circfig_mode_str	5631
lua-unicode-math	40	_l_spintent_geo_final_print_tl	4647
luamml	43, 96, 98	_l_spintent_geo_luaset_angulo_case_-str	2835
tagpdf	64, 65, 69	_l_spintent_geo_luaset_circ_radio_raw_-str	5324
unicode-math	40, 75	_l_spintent_geo_luaset_circ_radio_type_-str	5324
prefix	1738, 1809, 2046, 2256, 3114, 3233, 3483, 3718, 3858, 4067, 4354, 4508, 4693, 5342, 5641	_l_spintent_geo_luaset_concept_str	2835
print-era	1250	_l_spintent_geo_luaset_error_str	2835
print-m	3858, 4067	_l_spintent_geo_luaset_intent_str	2835
print-sym	4508, 4693, 5086, 5342, 5641	_l_spintent_geo_luaset_letra_str	2835
		_l_spintent_geo_luaset_letra_word_str	2835
R		_l_spintent_geo_luaset_letra_word_str	2835
radio-sty	5342	_l_spintent_geo_luaset_nombre_es_num_-str	2835
read-arg ..	1120, 1659, 3858, 4693, 5086, 5342, 5641, 5961	_l_spintent_geo_luaset_points_-clist__l_spintent_geo_luaset_is_-name_str	2835
read-cdot	574	_l_spintent_geo_luaset_poly_sym_str ..	2835
read-cmd ..	2046, 3114, 3233, 3483, 3718, 3858, 4067, 4354, 4508, 4693, 5086, 5342, 5641, 5961	_l_spintent_geo_luaset_print_tl	2835
read-parens	1884, 1955, 2574, 2652	_l_spintent_geo_luaset_sub_spoken_str	2835
read-period-sep	289, 574, 1884, 1955	_l_spintent_geo_luaset_sub_str	2835
read-sign	289, 1884, 1955, 2574	_l_spintent_geo_luaset_tipo_str	2835
read-slash	574	_l_spintent_geo_print_sym_str	4763
read-sty	3718, 3858, 5342, 5641		
read-sub	4693, 5086, 5342, 5641, 5961		
read-sym	1738, 1809, 1884, 1955		
recto	4067		
result	2046, 2256		
result-num	2256		

<code>__spintent_inner_sep:</code>	120	<code>__spintent_right_angle_sqr:</code>	107
<code>__spintent_invisible_sep:</code>	116	<code>__spintent_sanitize_affix:Nn</code>	136
<code>\c__spintent_invisible_sep_int</code>	124	<code>\l__spintent_saved_luaaml_flag_int</code> 98, 2633	
<code>\c__spintent_invisible_sep_tl</code>	98	<code>\l__spintent_scope_attr_int</code>	74
<code>__spintent_luafun_clean_split_and_set:n</code> 76, 246		<code>\l__spintent_setspintent_input_arg_cmd_-tl</code>	159
<code>__spintent_luafun_mrow_intent_arg:nn</code> .	116	<code>__spintent_setspintent_parse_args:nn</code> .	179
<code>__spintent_luafun_set_debug_ghost:n</code> ...	71	<code>__spintent_setspintent_set_keys:nn</code> ...	168
<code>__spintent_luafun_spang_parse:n</code>	93	<code>\l__spintent_smparc_prefix_tl</code>	4339
<code>__spintent_luafun_sptime_parse:n</code>	73	<code>__spintent_sp_rad_diam_exec:nn</code>	5297
<code>__spintent_luafun_spmoney_lookup_data:n</code> 76		<code>\l__spintent_sp_rad_diam_letra_tl</code>	5083
<code>__spintent_luafun_spmQ_scale_int:nn</code> ...	76	<code>__spintent_sp_rad_diam_print:n</code>	5133
<code>__spintent_luafun_sptime_parse:n</code>	72	<code>\l__spintent_sp_rad_diam_read_arg_full_-tl</code>	5083
<code>__spintent_luafun_spunit_lookup_alias:n</code> 76		<code>__spintent_sp_rad_diam_read_arg_-print:n</code>	5259
<code>\l__spintent_luaset_arg_above_tl</code>	265	<code>\l__spintent_sp_rad_diam_word_tl</code>	5083
<code>\l__spintent_luaset_arg_below_tl</code>	265	<code>\l__spintent_spAcirc_mode_str</code>	5621
<code>\l__spintent_luaset_cal_fam_ok_str</code>	67	<code>\l__spintent_spAcirc_prefix_tl</code>	5621
<code>\l__spintent_luaset_dec_is_all_zeros_-str</code>	258	<code>\l__spintent_spAcirc_print_sym_str</code> ...	5621
<code>\l__spintent_luaset_dec_sep_tl</code>	246	<code>\l__spintent_spAcirc_radio_sty_str</code> ...	5621
<code>\l__spintent_luaset_decimal_period_str</code> 258		<code>\l__spintent_spAcirc_read_arg_tl</code>	5621
<code>\l__spintent_luaset_denom_has_number_-str</code>	265	<code>\l__spintent_spAcirc_read_sty_str</code>	5621
<code>\l__spintent_luaset_exponent_tl</code>	258	<code>\l__spintent_spAcirc_sub_index_str</code> ...	5621
<code>\l__spintent_luaset_format_part_dec_str</code> 253		<code>\l__spintent_spAfig_mode_str</code>	4648
<code>\l__spintent_luaset_format_part_int_str</code> 253		<code>\l__spintent_spAfig_prefix_tl</code>	4648
<code>\l__spintent_luaset_has_exponent_str</code> ..	258	<code>\l__spintent_spAfig_print_sym_str</code>	4648
<code>\l__spintent_luaset_has_integer_str</code> . 98-100, 246		<code>\l__spintent_spAfig_read_arg_tl</code>	4648
<code>\l__spintent_luaset_has_natural_str</code> ...	246	<code>\l__spintent_spang_luaset_error_str</code> ...	2520
<code>\l__spintent_luaset_has_units_str</code>	265	<code>\l__spintent_spang_luaset_grado_str</code> ...	2520
<code>\l__spintent_luaset_millions_str</code>	258	<code>\l__spintent_spang_luaset_minuto_str</code> ..	2520
<code>\l__spintent_luaset_not_decimal_not_-period_str</code>	258	<code>\l__spintent_spang_luaset_segundo_str</code> .	2520
<code>\l__spintent_luaset_not_decimal_period_-str</code>	258	<code>__spintent_spang_process:n</code>	2529
<code>\l__spintent_luaset_only_part_dec_str</code> .	253	<code>__spintent_spangle_exec:nnn</code>	4312
<code>\l__spintent_luaset_only_part_int_str</code> .	253	<code>\l__spintent_spangle_intent_arg_tl</code> ...	4126
<code>\l__spintent_luaset_only_part_period_-str</code>	253	<code>\l__spintent_spangle_intent_tl</code>	4126
<code>\l__spintent_luaset_part_dec_tl</code>	246	<code>__spintent_spangle_mathml:n</code>	4168
<code>\l__spintent_luaset_part_int_tl</code> 98, 246		<code>\l__spintent_spangle_mode_str</code>	4052
<code>\l__spintent_luaset_part_period_tl</code> ...	246	<code>\l__spintent_spangle_prefix_tl</code>	4052
<code>\l__spintent_luaset_scope_attr_str</code>	67	<code>\l__spintent_spangle_print_m_now_bool</code> .	4126
<code>\l__spintent_luaset_sign_tl</code>	98, 246	<code>__spintent_spangle_process:nn</code>	4133
<code>\l__spintent_luaset_status_str</code>	265	<code>\l__spintent_spangle_pt_idx_int</code>	4126
<code>\l__spintent_luaset_sym_fam_ok_str</code>	67	<code>\l__spintent_spangle_pt_seq</code>	4126
<code>__spintent_mathcal:n</code>	94	<code>\l__spintent_spangle_refs_tl</code>	4126
<code>__spintent_mathmlarg_point:nnn</code>	3018	<code>__spintent_spangle_sym:n</code>	4107
<code>\l__spintent_mathmlarg_point_kv_tl</code> ...	3014	<code>__spintent_sparc_exec:nnn</code>	4472
<code>\l__spintent_mathmlarg_point_mode_str</code> .	3014	<code>\l__spintent_sparc_intent_arg_tl</code>	4398
<code>\l__spintent_mathmlarg_point_ref_tl</code> ...	3014	<code>\l__spintent_sparc_intent_tl</code>	4398
<code>\l__spintent_mathmlarg_point_silent_-bool</code>	3014	<code>__spintent_sparc_mathml:n</code>	4416
<code>\l__spintent_mc_args_seq</code>	2026	<code>\l__spintent_sparc_mode_str</code>	4339
<code>\l__spintent_mc_luaset_all_numeric_str</code> 2026		<code>\l__spintent_sparc_prefix_tl</code>	4339
<code>\l__spintent_mc_luaset_types_str</code>	2026	<code>__spintent_sparc_print_arc:n</code>	4392
<code>__spintent_mc_process_args:nn</code>	2087	<code>__spintent_sparc_process:nn</code>	4403
<code>\l__spintent_mc_raw_args_seq</code>	2026	<code>\l__spintent_sparc_pt_idx_int</code>	4398
<code>\l__spintent_mc_types_seq_{uuu}\l__spintent_-spmcd_print_result_bool</code>	2026	<code>\l__spintent_sparc_pt_seq</code>	4398
<code>\l__spintent_md_out_seq</code>	2247	<code>\l__spintent_sparc_refs_tl</code>	4398
<code>__spintent_print_spunit_hierarchy:</code> ...	675	<code>__spintent_sparc_wide_arc:n</code>	4388
		<code>\l__spintent_spcirc_articulo_str</code>	5326
		<code>__spintent_spcirc_build_concept:</code>	5480
		<code>\l__spintent_spcirc_connector_arg_tl</code> ..	5428
		<code>\l__spintent_spcirc_connector_tl</code>	5428
		<code>__spintent_spcirc_exec:nn</code>	5603

<code>\l__spintent_spcirc_has_arg_bool</code>	5428
<code>\l__spintent_spcirc_intent_arg_tl</code>	5428
<code>\l__spintent_spcirc_intent_tl</code>	5428
<code>__spintent_spcirc_mathml:</code>	5550
<code>\l__spintent_spcirc_mode_str</code>	5326
<code>\l__spintent_spcirc_prefix_tl</code>	5326
<code>\l__spintent_spcirc_print_sym_str</code>	5326
<code>__spintent_spcirc_process:n</code>	5534
<code>\l__spintent_spcirc_radio_arg_tl</code>	5428
<code>__spintent_spcirc_radio_print:</code>	5465
<code>\l__spintent_spcirc_radio_sty_str</code>	5326
<code>\l__spintent_spcirc_read_sty_str</code>	5326
<code>\l__spintent_spcirc_sub_index_str</code>	5326
<code>__spintent_spcirc_sym:</code>	5434
<code>__spintent_spcirc_word_and_article:</code> ..	5444
<code>\l__spintent_spcirc_word_tl</code>	5334
<code>__spintent_spcircfig_build_concept:nn</code>	5788
<code>__spintent_spcircfig_build_short:n</code> ...	5747
<code>__spintent_spcircfig_del_word:n</code>	5736
<code>__spintent_spcircfig_exec:nnn</code>	5923
<code>__spintent_spcircfig_finish_intent:nn</code>	5772
<code>\l__spintent_spcircfig_intent_arg_tl</code> ..	5707
<code>\l__spintent_spcircfig_intent_tl</code>	5707
<code>__spintent_spcircfig_mathml:nn</code>	5882
<code>\l__spintent_spcircfig_op_tl</code>	5707
<code>__spintent_spcircfig_process:nn</code>	5837
<code>\l__spintent_spcircfig_short_bool</code>	5707
<code>__spintent_spcircfig_sym:n</code>	5712
<code>__spintent_spcircfig_word_and_- article:n</code>	5718
<code>__spintent_spcoord_exec:nn</code>	2985
<code>__spintent_spcoord_intent_infix:</code>	2956
<code>\l__spintent_spcoord_intent_str</code>	2858
<code>\l__spintent_spcoord_luaset_error_str</code> .	2849
<code>\l__spintent_spcoord_luaset_sep_str</code> ...	2849
<code>\l__spintent_spcoord_luaset_x_has_frac_- str</code>	2849
<code>\l__spintent_spcoord_luaset_x_sign_str</code>	2849
<code>\l__spintent_spcoord_luaset_x_type_str</code>	2849
<code>\l__spintent_spcoord_luaset_x_value_- str</code>	2849
<code>\l__spintent_spcoord_luaset_y_has_frac_- str</code>	2849
<code>\l__spintent_spcoord_luaset_y_sign_str</code>	2849
<code>\l__spintent_spcoord_luaset_y_type_str</code>	2849
<code>\l__spintent_spcoord_luaset_y_value_- str</code>	2849
<code>__spintent_spcoord_mathml:</code>	2968
<code>__spintent_spcoord_print:n</code>	2933
<code>\l__spintent_spcoord_raw_arg_tl</code>	2858
<code>__spintent_spcoord_set_wrap_parens_- size:</code>	2877
<code>__spintent_spcoord_value:n</code>	2924
<code>\l__spintent_spcoord_wrap_parens_l_tl</code> .	2858
<code>\l__spintent_spcoord_wrap_parens_r_tl</code> .	2858
<code>\l__spintent_spdate_luaset_error_str</code> ..	1470
<code>\l__spintent_spdate_luaset_output_str</code> .	1470
<code>__spintent_spdate_process:n</code>	73, 1481
<code>__spintent_spdiv_process:nnn</code>	1930
<code>__spintent_spdiv_render_op:nn</code>	1909
<code>__spintent_spdiv_render_parens:nn</code> ...	1918
<code>\l__spintent_spdsym_prefix_tl</code>	1734
<code>\l__spintent_spdsym_read_sym_tl</code>	1734
<code>\l__spintent_spdsym_suffix_tl</code>	1734
<code>\l__spintent_spdsym_symbol_tl</code>	1734
<code>__spintent_spfig_build_concept_- vertices:n</code>	4928
<code>__spintent_spfig_build_refs:</code>	4915
<code>__spintent_spfig_build_simple:n</code>	4796
<code>__spintent_spfig_exec:nnn</code>	5052
<code>\l__spintent_spfig_intent_arg_tl</code>	4763
<code>\l__spintent_spfig_intent_tl</code>	4763
<code>__spintent_spfig_mathml:n</code>	5012
<code>\l__spintent_spfig_op_tl</code>	4763
<code>__spintent_spfig_points_loop:n</code>	5000
<code>__spintent_spfig_process:nn</code>	4772
<code>\l__spintent_spfig_pt_idx_int</code>	4763
<code>\l__spintent_spfig_pt_seq</code>	4763
<code>\l__spintent_spfig_refs_tl</code>	4763
<code>__spintent_spfig_sym:</code>	4755
<code>\l__spintent_spfig_symbol_tl</code>	4763
<code>\l__spintent_spmangle_mode_str</code>	4052
<code>\l__spintent_spmangle_prefix_tl</code>	4052
<code>\l__spintent_spmarc_mode_str</code>	4339
<code>__spintent_spmc_process:nnnn</code>	2141
<code>\l__spintent_spmcd_prefix_tl</code>	2026
<code>\l__spintent_spmcd_print_upper_bool</code> ...	2026
<code>\l__spintent_spmcd_read_terse_bool</code> ...	2026
<code>\l__spintent_spmcd_suffix_tl</code>	2026
<code>\l__spintent_spmcm_prefix_tl</code>	2026
<code>\l__spintent_spmcm_print_result_bool</code> ..	2026
<code>\l__spintent_spmcm_print_upper_bool</code> ...	2026
<code>\l__spintent_spmcm_read_terse_bool</code> ...	2026
<code>\l__spintent_spmcm_spmcd_luaset_error_- str</code>	2026
<code>\l__spintent_spmcm_suffix_tl</code>	2026
<code>__spintent_spmoney_build_tree:</code>	1032
<code>__spintent_spmoney_check_sub_units:</code> ..	983
<code>\l__spintent_spmoney_extracted_curr_str</code>	808
<code>\l__spintent_spmoney_extracted_num_tl</code> .	808
<code>__spintent_spmoney_intent_int:</code>	948
<code>\l__spintent_spmoney_intent_money_str</code> .	808
<code>\l__spintent_spmoney_intent_sign_str</code> ..	808
<code>\l__spintent_spmoney_intent_sub_unit_- str</code>	808
<code>\l__spintent_spmoney_luaset_conde_str</code> .	798
<code>\l__spintent_spmoney_luaset_currency_arg_- str</code>	798
<code>\l__spintent_spmoney_luaset_plur_str</code> ..	798
<code>\l__spintent_spmoney_luaset_position_- str</code>	798
<code>\l__spintent_spmoney_luaset_print_iso_- str</code>	798
<code>\l__spintent_spmoney_luaset_sing_str</code> ..	798
<code>\l__spintent_spmoney_luaset_status_str</code>	798
<code>\l__spintent_spmoney_luaset_subplur_str</code>	798
<code>\l__spintent_spmoney_luaset_subsing_str</code>	798
<code>\l__spintent_spmoney_luaset_symbol_str</code>	798
<code>__spintent_spmoney_normalize_key:n</code> ...	833
<code>__spintent_spmoney_parse_and_route:n</code> .	1073
<code>__spintent_spmoney_print_dec_sub_- unit:</code>	1032
<code>__spintent_spmoney_print_int:</code>	919

<code>__spintent_spmoney_print_sep_con: ...</code>	1032	<code>\l__spintent_spnD_intent_read_sign_str</code>	48 , 284
<code>__spintent_spmoney_print_symbol: ...</code>	881	<code>\l__spintent_spnD_notation_sym_tl ...</code>	284
<code>__spintent_spmoney_render_layout: ...</code>	898	<code>__spintent_spnD_print_integer: ...</code>	451
<code>__spintent_spmoney_render_number_node: ...</code>	910	<code>__spintent_spnD_print_num_start: ...</code>	451
<code>__spintent_spmoney_render_only_integer_node: ...</code>	910	<code>__spintent_spnD_print_number_core: ...</code>	451
<code>\l__spintent_spmoney_res_main_voice_str</code>	808	<code>__spintent_spnD_print_part_dec: ...</code>	430
<code>__spintent_spmoney_set_and_print_sub_units: ...</code>	983	<code>__spintent_spnD_print_part_period: ...</code>	430
<code>__spintent_spmoney_set_currency:n ...</code>	833	<code>__spintent_spnD_print_period_bar: ...</code>	359
<code>\l__spintent_spmoney_sub_units_bool ...</code>	808	<code>\l__spintent_spnD_read_period_sep_str</code>	48 , 284
<code>\l__spintent_spmoney_subnum_int ...</code>	808	<code>\l__spintent_spnD_read_terse_bool</code>	48 , 284
<code>__spintent_spmoney_validate_money: ...</code>	865	<code>__spintent_spnD_render_dec: ...</code>	342
<code>__spintent_spmQ_exec:nnnn ...</code>	2813	<code>__spintent_spnD_render_decimal_sep: ...</code>	396
<code>\l__spintent_spmQ_join_word_str ...</code>	98 , 2633	<code>__spintent_spnD_render_int: ...</code>	342
<code>__spintent_spmQ_print:nnn ...</code>	2805	<code>__spintent_spnD_render_only_period: ...</code>	359
<code>__spintent_spmQ_print_scale_int: ...</code>	2751	<code>__spintent_spnD_render_part:n ...</code>	342
<code>__spintent_spmQ_print_scale_int:n ...</code>	2760	<code>__spintent_spnD_render_period_sep: ...</code>	359
<code>__spintent_spmQ_print_spnZ_dfrac:nn ...</code>	2788	<code>__spintent_spnz_process:nn ...</code>	2607
<code>__spintent_spmQ_set_join_word: ...</code>	2742	<code>__spintent_spnz_render_parens: ...</code>	2595
<code>__spintent_spmQ_set_wrap_parens_size: ...</code>	2687	<code>\l__spintent_spPcirc_mode_str ...</code>	5621
<code>__spintent_spmQ_start_wrap_parens: ...</code>	2722	<code>\l__spintent_spPcirc_prefix_tl ...</code>	5621
<code>__spintent_spmQ_stop_wrap_parens: ...</code>	2722	<code>\l__spintent_spPcirc_print_sym_str ...</code>	5621
<code>\l__spintent_spmQ_use_dfrac_bool ...</code>	99	<code>\l__spintent_spPcirc_radio_sty_str ...</code>	5621
<code>\l__spintent_spmQ_use_spnZ_bool ...</code>	99	<code>\l__spintent_spPcirc_read_arg_tl ...</code>	5621
<code>\l__spintent_spmSide_mode_str ...</code>	3841	<code>\l__spintent_spPcirc_read_sty_str ...</code>	5621
<code>\l__spintent_spmSide_prefix_tl ...</code>	3841	<code>\l__spintent_spPcirc_sub_index_str ...</code>	5621
<code>\l__spintent_spmSide_read_arg_tl ...</code>	3841	<code>\l__spintent_spPfig_mode_str ...</code>	4648
<code>\l__spintent_spmSide_read_sty_str ...</code>	3841	<code>\l__spintent_spPfig_prefix_tl ...</code>	4648
<code>\l__spintent_spmSym_prefix_tl ...</code>	1805	<code>\l__spintent_spPfig_print_sym_str ...</code>	4648
<code>\l__spintent_spmSym_read_sym_tl ...</code>	1805	<code>\l__spintent_spPfig_read_arg_tl ...</code>	4648
<code>\l__spintent_spmSym_suffix_tl ...</code>	1805	<code>__spintent_spPoint_exec:nnn ...</code>	3187
<code>\l__spintent_spmSym_symbol_tl ...</code>	1805	<code>\l__spintent_spPoint_intent_arg_tl ...</code>	3103
<code>__spintent_spmul_process:nnn ...</code>	2001	<code>__spintent_spPoint_mathml:n ...</code>	3178
<code>__spintent_spmul_render_op:nn ...</code>	1980	<code>\l__spintent_spPoint_mode_str ...</code>	3103
<code>__spintent_spmul_render_parens:nn ...</code>	1989	<code>\l__spintent_spPoint_prefix_tl ...</code>	3103
<code>\l__spintent_spnD_luaset_is_truncated_str ...</code>	2247	<code>__spintent_spPoint_print:n ...</code>	3145
<code>\l__spintent_spnD_luaset_result_tl ...</code>	2247	<code>__spintent_spPoly_build_concept: ...</code>	4567
<code>\l__spintent_spnD_prefix_tl ...</code>	2247	<code>__spintent_spPoly_build_refs: ...</code>	4554
<code>\l__spintent_spnD_print_result_num_tl</code>	2247	<code>__spintent_spPoly_exec:nn ...</code>	4631
<code>\l__spintent_spnM_luaset_is_truncated_str ...</code>	2247	<code>\l__spintent_spPoly_intent_arg_tl ...</code>	4542
<code>\l__spintent_spnM_luaset_result_tl ...</code>	2247	<code>__spintent_spPoly_mathml: ...</code>	4611
<code>\l__spintent_spnM_prefix_tl ...</code>	2247	<code>\l__spintent_spPoly_mode_str ...</code>	4499
<code>\l__spintent_spnM_print_result_num_tl</code>	2247	<code>__spintent_spPoly_points_loop: ...</code>	4589
<code>__spintent_spnM_spnD_exec:nnnn ...</code>	2350	<code>\l__spintent_spPoly_prefix_tl ...</code>	4499
<code>__spintent_spnM_spnD_intent:nn ...</code>	2301	<code>__spintent_spPoly_process:n ...</code>	4601
<code>\l__spintent_spnM_spnD_luaset_error_str ...</code>	2247	<code>\l__spintent_spPoly_pt_idx_int ...</code>	4542
<code>__spintent_spnM_spnD_process:nnnn ...</code>	2296	<code>\l__spintent_spPoly_pt_seq ...</code>	4542
<code>__spintent_spnM_spnD_result:nn ...</code>	2318	<code>\l__spintent_spPoly_refs_tl ...</code>	4542
<code>__spintent_spnM_spnD_silent:nn ...</code>	2310	<code>__spintent_spPoly_sym: ...</code>	4546
<code>\l__spintent_spnmd_luaset_is_number_str ...</code>	2247	<code>__spintent_sprayo_exec:nn ...</code>	3825
<code>\l__spintent_spnQ_intent_read_sign_str</code>	2633	<code>\l__spintent_sprayo_intent_arg_tl ...</code>	3762
<code>\l__spintent_spnQ_read_parens_bool ...</code>	97	<code>__spintent_sprayo_mathml: ...</code>	3775
<code>\l__spintent_spnQ_wrap_parens_l_tl</code>	97 , 2633	<code>\l__spintent_sprayo_mode_str ...</code>	3708
<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	97 , 2633	<code>\l__spintent_sprayo_prefix_tl ...</code>	3708
<code>\l__spintent_spnnum_notation_sym_tl ...</code>	48	<code>__spintent_sprayo_print:n ...</code>	3766
<code>\l__spintent_spnnum_intent_read_dec_sep_str ...</code>	48 , 50 , 284	<code>\l__spintent_sprayo_pt_idx_int ...</code>	3762
		<code>\l__spintent_sprayo_pt_seq ...</code>	3762
		<code>\l__spintent_sprayo_read_sty_str ...</code>	3708
		<code>\l__spintent_sprayo_refs_tl ...</code>	3762
		<code>__spintent_sprazon_exec:nnn ...</code>	2453

<code>__spintent_spread_exec:nn</code>	1676
<code>\l__spintent_spread_luaset_reading_str</code>	1656
<code>\l__spintent_spread_luaset_status_str</code>	1656
<code>\l__spintent_spread_read_arg_tl</code>	1656
<code>__spintent_sprecta_exec:nn</code>	3692
<code>\l__spintent_sprecta_intent_arg_tl</code> ...	3469
<code>\l__spintent_sprecta_kv_tl</code>	3469
<code>__spintent_sprecta_mathml:n</code>	3523
<code>\l__spintent_sprecta_mode_str</code>	3469
<code>\l__spintent_sprecta_prefix_tl</code>	3469
<code>__spintent_sprecta_print:n</code>	3514
<code>\l__spintent_sprecta_pt_idx_int</code>	3469
<code>\l__spintent_sprecta_pt_seq</code>	3469
<code>\l__spintent_sprecta_refs_tl</code>	3469
<code>__spintent_spshort_execute_layout:...</code>	1181
<code>\l__spintent_spshort_luaset_base_str</code> ..	1098
<code>\l__spintent_spshort_luaset_expand_str</code>	1098
<code>\l__spintent_spshort_luaset_layout_str</code>	1098
<code>\l__spintent_spshort_luaset_output_str</code>	1098
<code>\l__spintent_spshort_luaset_status_str</code>	1098
<code>\l__spintent_spshort_luaset_suffix_str</code>	1098
<code>__spintent_spshort_print:nn</code>	1134
<code>__spintent_spshort_process_and_ render:n</code>	1146
<code>__spintent_spshort_render_bypass:n</code> ...	1157
<code>__spintent_spshort_render_error:n</code> ...	1157
<code>__spintent_spshort_render_fallback:..</code>	1157
<code>__spintent_spshort_render_success:...</code>	1157
<code>__spintent_spshort_tag_span:nn</code>	1107
<code>__spintent_spside_exec:nnn</code>	4025
<code>\l__spintent_spside_intent_arg_tl</code>	3915
<code>\l__spintent_spside_intent_tl</code>	3915
<code>__spintent_spside_mathml:n</code>	3948
<code>\l__spintent_spside_mode_str</code>	3841
<code>__spintent_spside_points_loop:n</code>	4013
<code>\l__spintent_spside_prefix_tl</code>	3841
<code>__spintent_spside_process:nn</code>	3920
<code>\l__spintent_spside_pt_idx_int</code>	3915
<code>\l__spintent_spside_pt_seq</code>	3915
<code>\l__spintent_spside_read_arg_tl</code>	3841
<code>\l__spintent_spside_read_sty_str</code>	3841
<code>\l__spintent_spside_refs_tl</code>	3915
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1238
<code>\l__spintent_spsiglo_luaset_error_str</code> ..	1238
<code>\l__spintent_spsiglo_luaset_roman_min_ str</code>	1238
<code>\l__spintent_spsiglo_print_era_str</code> ...	1238
<code>__spintent_spsiglo_print_math:n</code>	1322
<code>__spintent_spsiglo_print_text:n</code>	1322
<code>__spintent_spsiglo_render_simple:nn</code> ..	1273
<code>__spintent_spsiglo_render_with_ era:nnn</code>	1273
<code>__spintent_spsiglo_tag_span:nn</code>	1309
<code>__spintent_sptime_build_seconds:nn</code> ...	1402
<code>__spintent_sptime_build_tree:...</code> 72,	1420
<code>\l__spintent_sptime_luaset_base_str</code> ...	1388
<code>\l__spintent_sptime_luaset_error_str</code> ..	1388
<code>\l__spintent_sptime_luaset_has_seconds_ str</code>	1388
<code>\l__spintent_sptime_luaset_is_fraction_ str</code>	1388
<code>\l__spintent_sptime_luaset_seconds_str</code>	1388
<code>\l__spintent_spunit_exp_seq</code>	545
<code>\l__spintent_spunit_exponent_tl</code>	545
<code>__spintent_spunit_format_canonical:..</code>	608
<code>\l__spintent_spunit_is_mult_bool</code>	545
<code>\l__spintent_spunit_luaset_canonical_ str</code>	538
<code>\l__spintent_spunit_luaset_compact_exp_ str</code>	538
<code>\l__spintent_spunit_luaset_is_compact_ str</code>	538
<code>\l__spintent_spunit_luaset_is_ sexagesimal_str</code>	538
<code>\l__spintent_spunit_luaset_lookup_status_ str</code>	538
<code>\l__spintent_spunit_luaset_read_str</code> ...	538
<code>\l__spintent_spunit_luaset_status_str</code> ..	538
<code>\l__spintent_spunit_mult_seq</code>	545
<code>__spintent_spunit_process_base_exp:n</code> ..	616
<code>__spintent_spunit_process_mult:n</code>	668
<code>\l__spintent_spunit_read_cdot_bool</code> ...	545
<code>\l__spintent_spunit_read_slash_str</code> ...	545
<code>__spintent_spunit_safe_exec:n</code>	687
<code>\l__spintent_spunit_unit_name_tl</code>	545
<code>\l__spintent_spusep_arg_tl</code>	73, 75, 1517
<code>__spintent_spusep_convert:n</code>	1579
<code>\l__spintent_spusep_left_tl</code>	73, 74, 1517
<code>\l__spintent_spusep_right_tl</code>	73, 74, 1517
<code>__spintent_spusep_set_parens:nn</code>	1568
<code>\l__spintent_spusep_silent_bool</code>	74
<code>__spintent_sptvec_exec:nnn</code>	3431
<code>\l__spintent_sptvec_intent_arg_tl</code>	3222
<code>\l__spintent_sptvec_kv_tl</code>	3222
<code>__spintent_sptvec_mathml:n</code>	3284
<code>\l__spintent_sptvec_mode_str</code>	3222
<code>\l__spintent_sptvec_prefix_tl</code>	3222
<code>__spintent_sptvec_print:n</code>	3267
<code>\l__spintent_sptvec_pt_idx_int</code>	3222
<code>\l__spintent_sptvec_pt_seq</code>	3222
<code>\l__spintent_sptvec_refs_tl</code>	3222
<code>__spintent_sptVol_attach_subscript:...</code>	6086
<code>\l__spintent_sptVol_mode_str</code>	5954
<code>\l__spintent_sptVol_native_bool</code>	5954
<code>__spintent_sptVol_print:...</code>	5999
<code>\l__spintent_sptVol_read_arg_full_tl</code> ...	5954
<code>__spintent_sptVol_read_arg_print:...</code>	6118
<code>\l__spintent_sptVol_read_arg_tl</code>	5954
<code>__spintent_sptVol_silent:...</code>	6006
<code>\l__spintent_sptVol_sub_index_str</code>	5954
<code>\l__spintent_sptVol_suffix_tl</code>	5954
<code>__spintent_sptVol_symbol_print:...</code>	6048
<code>__spintent_sptVol_verbal:...</code>	6027
<code>\l__spintent_sptVol_word_tl</code>	5954
<code>__spintent_unknown_keys:n</code>	235
<code>__spintent_unknown_keys:nn</code>	235
<code>spintent.lua</code>	70
<code>\spmangle</code>	24, 124, 4317
<code>spmangle-invalid-arg</code>	4057
<code>\spmarc</code>	25, 129, 4477
<code>spmarc-invalid-points</code>	4344
<code>\spmcd</code>	15, 83, 2227
<code>\spmcm</code>	16, 83, 2227

<code>\spmoney</code>	6, 58, 1087
<code>\spmQ</code>	17, 95, 2825
<code>spmQ-not-integer</code>	2638
<code>spmQ-zero-denominator</code>	2638
<code>\spmside</code>	22, 120, 4030
<code>spmside</code>	5342
<code>spmside-invalid-points</code>	3848
<code>\spmsym</code>	13, 79, 1858
<code>\spmul</code>	14, 82, 2016
<code>spmul-empty-argument</code>	1975
<code>\spnD</code>	14, 87, 2416
<code>\spnewread</code>	11, 1715
<code>spnewread-duplicate</code>	1628
<code>spnewread-empty-args</code>	1628
<code>\spnewunit</code>	6, 57, 766
<code>spnewunit-duplicate</code>	745
<code>\spnM</code>	15, 87, 2416
<code>\spnum</code>	3, 48, 49, 520
<code>spnum-has-units</code>	506
<code>spnum-no-digits</code>	506
<code>\spnZ</code>	17, 94, 2623
<code>spnz-not-integer</code>	2590
<code>\spPcirc</code>	32, 154, 5928
<code>spPcirc-invalid-arg</code>	5632
<code>\spPfig</code>	28, 136, 5057
<code>spPfig-invalid-arg</code>	4655
<code>spPfig-invalid-points</code>	4655
<code>spPfig-read-arg-key</code>	4655
<code>spPfig-sym-cardinality-conflict</code>	4655
<code>spPfig-sym-letra-conflict</code>	4655
<code>\spPoint</code>	19, 106, 3211
<code>spPoint-invalid-point</code>	3106
<code>\spPoly</code>	26, 133, 4636
<code>spPoly-invalid-points</code>	4501
<code>\spProp</code>	18, 92, 2504
<code>\sprad</code>	29, 144, 5302
<code>\sprayo</code>	21, 117, 3830
<code>sprayo-invalid-points</code>	3711
<code>\sprazon</code>	18, 91, 2494
<code>\spread</code>	12, 44, 75, 1705
<code>spread-unknown-sym</code>	1628
<code>\sprecta</code>	20, 113, 3697
<code>sprecta-invalid-points</code>	3476
<code>\spshort</code>	8, 64, 1225
<code>\spside</code>	21, 120, 4030
<code>spside</code>	5342
<code>spside-invalid-points</code>	3848
<code>\spsiglo</code>	10, 67, 1375
<code>\sptime</code>	11, 70, 1452
<code>\spunit</code>	4, 53, 54, 57, 733
<code>spunit-invalid-unit</code>	552
<code>spunit-no-units</code>	552
<code>\spunitalias</code>	6, 57, 780
<code>spunitalias-duplicate</code>	745
<code>spunitalias-invalid-expr</code>	745
<code>\spusep</code>	12, 73, 1617
<code>spusep-empty-arg</code>	1506
<code>spusep-invalid-arg</code>	1506
<code>\spvec</code>	20, 108, 3458
<code>spvec-invalid-arg</code>	3229
<code>\spVol</code>	33, 161, 6146
str commands:	
<code>\str_if_eq:nnTF</code>	62
<code>\str_lowercase:n</code>	62
<code>\str_uppercase:n</code>	62
sub-index	5086 , 5342 , 5641 , 5961
sub-units	847
suffix	1738 , 1809
U	
unknown	1520 , 1659 , 1738 , 1809 , 2046 , 2256 , 2436 , 2652 , 3114 , 3233 , 3483 , 3718 , 3858 , 4067 , 4354 , 4508 , 4693 , 5086 , 5342 , 5641 , 5961
unknown-abbreviation	1104
unknown-choice	208
unknown-command-setup	159
upper-case	2046 , 5086
use-dfrac	2436 , 2652
use-spnZ	2652
W	
wrap-parens	1884 , 1955 , 2574 , 2652